CALDAN CONSTRUCTION

TECHNICAL SPECIFICATION • MODERN PLATFORM BLUEPRINT

Kenya-first construction management, commerce, analytics, and operations platform.

Projects • Inventory • Finance • Shop • Reports

Prepared for Dr. Daniel Ogwoka Siringi

Prepared by Mukoya Khisa | July 2026

"Building Kenya's Future, One Project at a Time."

Contents

List of Tables x

List of Figures xiv

List of Code Listings xvi

Important Notice 1

I Executive Summary & Brand Architecture 2

1 Executive Summary & Vision 4
- 1.1 The Vision . . . 4
- 1.2 The Problem Space . . . 4
- 1.3 The Solution . . . 5
- 1.4 Target Market . . . 6
- 1.5 Client Delivery and Cost Model . . . 6
- 1.6 Technology Foundation . . . 6
- 1.7 Architecture Philosophy . . . 7
  - 1.7.1 Service-Oriented Architecture . . . 7
  - 1.7.2 Dynamic by Design . . . 7
  - 1.7.3 Kenya-First, Single-Market Focus . . . 8
- 1.8 Scope of This Specification . . . 8
- 1.9 Developer's Note . . . 8

Chapter Summary . . . 9

2 Brand Architecture 10
- 2.1 The Name: Caldan Construction . . . 10
- 2.2 Tagline . . . 10
  - 2.2.1 The Tagline . . . 10
  - 2.2.2 Tagline Philosophy . . . 10
- 2.3 Color Palette . . . 11
  - 2.3.1 Color Psychology . . . 12
  - 2.3.2 Color Application Rules . . . 13
- 2.4 Logo Design . . . 13
  - 2.4.1 Design Principles . . . 14
- 2.5 Typography . . . 14
  - 2.5.1 Primary Font: Inter . . . 14
- 2.6 Iconography . . . 14
- 2.7 Tone of Voice . . . 15
- 2.8 Brand Assets Summary . . . 16

3 Platform Overview **17**
3.1 Platform Architecture Philosophy . . . 17
3.2 User Types and Role Hierarchy . . . 17
3.3 Corporate Website Architecture . . . 18
3.4 Dashboard Architecture . . . 18
3.4.1 Dashboard Routing Logic . . . 18
3.4.2 Platform Admin Dashboard – Full Access . . . 18
3.4.3 Feature Taxonomy – Complete . . . 20

II Technology Stack & Infrastructure **22**

4 Technology Stack **24**
4.1 Selection Philosophy . . . 24
4.2 Technology Stack Overview . . . 25
4.3 Backend: Django . . . 25
4.3.1 Why Django . . . 25
4.3.2 Django Apps Architecture . . . 26
4.4 Database: MySQL . . . 26
4.4.1 Why MySQL . . . 26
4.5 Frontend: HTML, CSS, JavaScript . . . 27
4.5.1 Design Principles . . . 27
4.5.2 CSS Architecture . . . 27
4.6 Payments: Paystack . . . 27
4.6.1 Why Paystack . . . 27
4.7 Email: Resend . . . 28
4.7.1 Why Resend . . . 28
4.8 SMS: Africa's Talking with Celcom Fallback . . . 28
4.8.1 Provider Strategy . . . 28
4.9 Media Storage: Cloudinary . . . 29
4.10 Task Scheduling: Celery + Redis . . . 29
4.10.1 Scheduled Tasks . . . 29

5 Django Application Architecture **31**
5.1 Overview . . . 31
5.2 Application Inventory . . . 32
5.3 Service-Oriented App Pattern . . . 32
5.4 Cross-App Interaction Rules . . . 33
5.5 Architecture Summary . . . 33

6 Domain, Infrastructure & Deployment **34**
6.1 Domain Strategy . . . 34
6.1.1 Primary Domain . . . 34
6.1.2 Domain Registration . . . 34
6.2 Email Configuration . . . 34
6.2.1 Transactional Email Address . . . 34
6.3 Environment Configuration . . . 34
6.3.1 The .env File . . . 34
6.4 Hosting Infrastructure . . . 35

6.4.1 Phase 1: Bootstrapped Hosting (Launch) . . . 35
6.4.2 Phase 2: Scaled Infrastructure (Post-Traction) . . . 36
6.5 SSL & Security . . . 36
6.5.1 HTTPS Everywhere . . . 36
6.6 Static & Media File Storage . . . 36
6.6.1 Static Files (CSS, JS, Images) . . . 36
6.6.2 Media Files (User Uploads) . . . 36
6.7 Backup Strategy . . . 37
6.7.1 Database Backups . . . 37
6.8 Monitoring & Logging . . . 37
6.8.1 Log File Inventory . . . 38
6.9 Deployment Workflow . . . 38
6.9.1 Deployment Process . . . 38
6.10 Infrastructure Cost Summary . . . 39

III Authentication & User Management 40

7 Authentication System 42
7.1 Overview . . . 42
7.2 Custom User Model . . . 42
7.3 OTP Verification Flow . . . 43
7.3.1 8-Digit OTP Generation . . . 43
7.4 Login Flow . . . 44
7.4.1 Complete Login Sequence . . . 44
7.5 Session Management . . . 44
7.5.1 Session Configuration . . . 44
7.5.2 Remember Me (Trusted Device) . . . 45
7.6 Login Attempt Limiting . . . 45
7.7 Authentication Security Measures . . . 46

8 User Management & Roles 47
8.1 Overview . . . 47
8.2 Role Hierarchy . . . 47
8.3 Role Definitions . . . 47
8.3.1 Chief Admin . . . 47
8.3.2 Platform Admin . . . 48
8.3.3 Client . . . 48
8.4 Internal User Creation Flow . . . 48
8.5 Invitation Model . . . 49
8.6 Permission Inventory . . . 49
8.6.1 Chief Admin Permissions . . . 50
8.6.2 Platform Admin Permissions . . . 51
8.7 Permission Enforcement . . . 51
8.8 Audit Trail for User Actions . . . 52

9 OTP & Security System **53**
9.1 Overview . . . 53
9.2 OTP Generation . . . 53
9.2.1 8-Digit Code Generation . . . 53
9.3 OTP Delivery . . . 53
9.3.1 Email Template . . . 53
9.4 OTP Verification Flow . . . 54
9.5 Security Monitoring . . . 54
9.5.1 Security Events Logged . . . 55

10 Admin Account Creation & Invitations **56**
10.1 Overview . . . 56
10.2 First Admin: The Setup URL . . . 56
10.2.1 Setup Flow . . . 57
10.3 Inviting Other Admins . . . 57
10.4 Force Password Change Middleware . . . 58
10.5 Admin Management Dashboard . . . 58
10.6 Admin Permission Management . . . 59
10.7 Admin ID System . . . 59
10.8 Security Summary . . . 60

IV Core Business Modules **62**

11 Public Website Architecture **64**
11.1 Overview . . . 64
11.2 Page Inventory . . . 64
11.3 Landing Page Architecture . . . 65
11.3.1 Section Architecture . . . 65
11.3.2 Hero Section Specifications . . . 65
11.3.3 Trust Counter . . . 65
11.3.4 Landing Page Template . . . 66
11.4 Services Page . . . 67
11.5 Contact Page . . . 68
11.6 SEO & Metadata . . . 68

12 Projects Management **70**
12.1 Overview . . . 70
12.2 Project Hierarchy . . . 70
12.3 Project Model . . . 70
12.4 Project Milestones . . . 71
12.5 Progress Tracking . . . 72
12.6 Project Views . . . 73

13 Inventory Management **76**
13.1 Overview . . . 76
13.2 Inventory Control Loop . . . 76
13.3 Material Model . . . 76
13.4 Stock Transactions . . . 77

13.5 Stock Service 79

14 Financial Management **82**
14.1 Overview 82
14.2 Payment Model 82
14.3 Expense Model 83
14.4 Budget Tracking 84
14.5 Financial Service 85

15 Client Management **87**
15.1 Overview 87
15.2 Client Model 87
15.3 Lead Capture 88

16 Team Management **90**
16.1 Overview 90
16.2 Team Member Model 90
16.3 Team Management Views 91
16.4 Public Team Display 91

17 Services Management **93**
17.1 Overview 93
17.2 Service Model 93
17.3 Service Management Views 94

V Online Shop **96**

18 Shop & Product Catalog **97**
18.1 Overview 97
18.2 Product Model 97
18.3 Product Views 98

19 Cart & Checkout **100**
19.1 Overview 100
19.2 Cart Models 100
19.3 Cart Service 101
19.4 Checkout Views 101
19.5 Checkout Service 102

20 Order Management **104**
20.1 Overview 104
20.2 Order Models 104
20.3 Order Management Views 105
20.4 Order Status Workflow 106
20.5 Order Tracking (Client Side) 106

VI Admin Dashboard 108

21 Dashboard Architecture 109
21.1 Overview . . . 109
21.2 Base Dashboard Layout . . . 109
21.3 Context Processors . . . 109
21.4 Base Dashboard Template . . . 110
21.5 Sidebar Navigation . . . 110
21.6 Theme System . . . 112
21.6.1 CSS Custom Properties . . . 112

22 Dashboard Pages & Navigation 114
22.1 Overview . . . 114
22.2 Complete Page Inventory . . . 114
22.3 Dashboard Overview Page . . . 116

23 Data Visualization & Analytics 119
23.1 Overview . . . 119
23.2 Analytics Dashboard . . . 119
23.3 Analytics Service . . . 119
23.4 Analytics Views . . . 120
23.5 Export Options . . . 121

VII Notification System 122

24 Notification System Overview 123
24.1 Overview . . . 123
24.2 Three-Channel Architecture . . . 123
24.3 Notification Model . . . 123
24.4 Notification Service . . . 124
24.5 Notification Types . . . 125

25 Email System (Resend) 127
25.1 Overview . . . 127
25.2 Resend Setup . . . 127
25.2.1 Environment Configuration . . . 127
25.2.2 Django Settings . . . 127
25.3 Email Service . . . 127
25.4 Email Log Model . . . 129
25.5 Email Templates Inventory . . . 130

26 SMS System (Africa's Talking + Celcom) 131
26.1 Overview . . . 131
26.2 Provider Strategy . . . 131
26.3 SMS Service . . . 131
26.4 SMS Log Model . . . 133

27 Notification Types & Templates **135**
27.1 Overview . . . 135
27.2 Complete Notification Type Inventory . . . 135
27.3 Email Template Inventory . . . 135
27.4 SMS Template Inventory . . . 136
27.5 Email Template Example . . . 136

28 Reminder Engine & Automation **138**
28.1 Overview . . . 138
28.2 Reminder Service . . . 138
28.3 Celery Task Configuration . . . 139

29 Reports Engine **141**
29.1 Overview . . . 141
29.2 Report Architecture . . . 141
29.3 Report Inventory . . . 142
29.4 Report Service . . . 142
29.5 Report Views . . . 143

30 PDF Generation (Playwright) **145**
30.1 Overview . . . 145
30.2 PDF Service . . . 145
30.3 PDF Template Example . . . 146
30.4 Installation & Configuration . . . 147

31 Data Export & Backup **149**
31.1 Overview . . . 149
31.2 Data Export Center . . . 149
31.3 Export Service . . . 149
31.4 Backup Service . . . 150
31.5 Data Deletion . . . 153

32 Bi-Weekly Audit Reports **155**
32.1 Overview . . . 155
32.2 Audit Report Contents . . . 155
32.3 Audit Report Service . . . 155
32.4 Celery Task . . . 157

33 Security Architecture **158**
33.1 Overview . . . 158
33.2 Threat Model . . . 158
33.3 Network Security . . . 158
33.3.1 HTTPS Everywhere . . . 158
33.3.2 Content Security Policy . . . 159
33.4 Application Security . . . 159
33.4.1 Authentication Security . . . 159
33.4.2 Input Validation . . . 159
33.5 Data Security . . . 160
33.5.1 Encryption at Rest . . . 160

33.6 Audit Logging . . . 160
33.7 Incident Response Plan . . . 161

34 Audit & Compliance **162**
34.1 Overview . . . 162
34.2 AuditLog Model . . . 162
34.3 Audit Service . . . 163

35 Database Schema **165**
35.1 Overview . . . 165
35.2 Entity-Relationship Overview . . . 165
35.3 Core Tables . . . 165
35.3.1 Users Table . . . 166
35.3.2 Projects Table . . . 166
35.3.3 Materials Table . . . 167
35.3.4 Shop Orders Table . . . 167
35.4 Global Indexes . . . 168

36 API Architecture **169**
36.1 Overview . . . 169
36.2 Technology Stack . . . 169
36.3 Authentication . . . 169
36.4 API Endpoints Reference . . . 170
36.5 Error Handling . . . 170

37 Testing Strategy **172**
37.1 Overview . . . 172
37.2 Testing Pyramid . . . 172
37.3 Coverage Requirements . . . 172
37.4 Unit Test Examples . . . 172
37.5 Integration Tests . . . 173
37.6 End-to-End Tests (Playwright) . . . 174
37.7 Continuous Integration . . . 174

38 Deployment & DevOps **176**
38.1 Overview . . . 176
38.2 Deployment Environments . . . 176
38.3 Docker Configuration . . . 176
38.4 Deployment Script . . . 177
38.5 Health Check Endpoint . . . 177
38.6 Rollback Strategy . . . 178
38.7 Monitoring Metrics . . . 179

Final Statement **180**

A Modern Visual Atlas **181**
A.1 Strategic and Brand Context . . . 181
A.2 Platform Architecture . . . 182
A.3 Security and Governance . . . 183

A.4 Operations and Finance . . . 185
A.5 Delivery and Launch . . . 186

B Complete Project Directory **188**
B.1 Overview . . . 188
B.2 Directory Summary . . . 196

C API Endpoints Reference **197**

D Email Template Inventory **198**

E Permission Matrix **199**

F Testing Checklist **201**

G Deployment Checklist **202**

H Caldan Builder Company Profile Details **203**
H.1 Profile Header and Company Identity . . . 203
H.2 Source Profile Contents and Annexures . . . 204
H.3 Introduction, Vision, and Mission . . . 204
H.4 Services . . . 205
H.5 Ownership, Management, and Approach . . . 205
H.6 Experience and Project References . . . 206
H.7 Resources . . . 207
H.8 Contact Details . . . 207
H.9 Conclusion from Source Profile . . . 208
H.10 Shareholding, Credentials, References, and Banking Details . . . 208

Final Statement **209**

List of Tables

1.1 Platform Value Chain – From Construction Intent to Operational Excellence . . 4
1.2 Current State of Construction Management 5
1.3 Caldan Construction Core Capabilities 5
1.4 Target User Personas 6
1.5 Internal System Cost Responsibilities 6
1.6 Core Technology Stack 7
1.7 Document Structure Overview 8
1.8 Executive Summary – Key Specifications 9

2.1 Brand Name Components 10
2.2 Tagline Meaning Layers 10
2.3 Official Caldan Brand Color Palette 11
2.4 Color Psychology 12
2.5 Color Usage Guidelines 13
2.6 Logo Variants 13
2.7 Typography Hierarchy 14
2.8 Icon Mapping – Dashboard Navigation 15
2.9 Icon Mapping – Actions & States 15
2.10 Brand Voice Attributes 16
2.11 Complete Brand Asset Inventory 16
2.12 Brand Architecture – Key Specifications 16

3.1 Platform Environment Architecture 17
3.2 User Role Definitions 17
3.3 Corporate Website Page Inventory 18
3.4 Complete Feature Inventory by Module 20
3.5 Platform Overview – Key Specifications 21

4.1 Technology Selection Criteria 24
4.2 Complete Technology Stack – All Layers 25
4.3 Django Application Architecture 26
4.4 SMS Provider Strategy 28
4.5 Cloudinary Benefits 29
4.6 Automated Scheduled Tasks 29
4.7 Technology Stack – Key Specifications 30

5.1 Complete Django App Summary 32
5.2 Standard App Structure Pattern 33
5.3 Django Application Architecture Summary 33

6.1 Domain Registration Details 34
6.2 Hosting Options – Bootstrapped Phase 35
6.3 Hosting Evolution Path 36
6.4 Media Storage Strategy 37

6.5 Backup Schedule . . . 37
6.6 Log Files . . . 38
6.7 Estimated Monthly Infrastructure Costs – Bootstrapped Phase . . . 39
6.8 Domain, Infrastructure & Deployment – Key Specifications . . . 39

7.1 Authentication Security Measures . . . 46
7.2 Authentication System – Key Specifications . . . 46

8.1 Chief Admin Permissions . . . 50
8.2 Platform Admin Permissions (Configurable) . . . 51
8.3 User Management – Key Specifications . . . 52

9.1 Security Events Logged . . . 55
9.2 OTP & Security – Key Specifications . . . 55

10.1 Admin ID Format . . . 60
10.2 Admin Creation Security Features . . . 60
10.3 Admin Account Creation – Key Specifications . . . 61

11.1 Complete Public Website Page Inventory . . . 64
11.2 Hero Section Specifications . . . 65
11.3 Service Categories . . . 67
11.4 Contact Page Elements . . . 68
11.5 Public Website Architecture — Key Specifications . . . 69

12.1 Projects Management — Key Specifications . . . 75

13.1 Inventory Management — Key Specifications . . . 81

14.1 Financial Management — Key Specifications . . . 86

15.1 Client Management — Key Specifications . . . 89

16.1 Team Management — Key Specifications . . . 92

17.1 Services Management — Key Specifications . . . 95

18.1 Shop & Product Catalog — Key Specifications . . . 99

19.1 Cart & Checkout — Key Specifications . . . 103

20.1 Order Management — Key Specifications . . . 107

21.1 Dashboard Architecture — Key Specifications . . . 113

22.1 Complete Dashboard Page Inventory . . . 115
22.2 Dashboard Pages & Navigation — Key Specifications . . . 118

23.1 Export Options . . . 121
23.2 Data Visualization & Analytics — Key Specifications . . . 121

24.1 Complete Notification Type Inventory . . . 125
24.2 Notification System — Key Specifications . . . 126

25.1 Email Template Inventory . . . 130
25.2 Email System — Key Specifications . . . 130

26.1 SMS Provider Strategy . . . 131
26.2 SMS System — Key Specifications . . . 134

27.1 Complete Notification Type Inventory . . . 135
27.2 Email Template Inventory . . . 135
27.3 SMS Template Inventory . . . 136
27.4 Notification Types & Templates — Key Specifications . . . 137

28.1 Reminder Engine — Key Specifications . . . 140

29.1 Complete Report Inventory . . . 142
29.2 Reports Engine — Key Specifications . . . 144

30.1 PDF Generation — Key Specifications . . . 148

31.1 Data Export & Backup — Key Specifications . . . 154

32.1 Audit Report Contents . . . 155
32.2 Bi-Weekly Audit Reports — Key Specifications . . . 157

33.1 Threat Model — Identified Risks . . . 158
33.2 Authentication Security Measures . . . 159
33.3 Encryption at Rest . . . 160
33.4 Security Events Logged . . . 160
33.5 Incident Severity Levels . . . 161
33.6 Security Architecture — Key Specifications . . . 161

34.1 Audit & Compliance — Key Specifications . . . 164

35.1 users Table . . . 166
35.2 projects Table . . . 166
35.3 materials Table . . . 167
35.4 shop_orders Table . . . 167
35.5 Critical Database Indexes . . . 168
35.6 Database Schema — Key Specifications . . . 168

36.1 API Technology Stack . . . 169
36.2 API Endpoints Reference . . . 170
36.3 API Architecture — Key Specifications . . . 171

37.1 Minimum Code Coverage by Module . . . 172
37.2 Testing Strategy — Key Specifications . . . 175

38.1 Deployment Environments . . . 176
38.2 Production Monitoring Metrics . . . 179
38.3 Deployment & DevOps — Key Specifications . . . 179

B.1 Project Directory Summary . . . 196

C.1 Complete API Endpoints Reference . . . 197

D.1 Complete Email Template Inventory 198

E.1 Complete Permission Matrix 200

F.1 Testing Checklist — Critical Paths 201

G.1 Deployment Checklist — Pre-Launch 202

H.1 Company Profile Header Details from April 2021 Source Profile 203
H.2 Uploaded Company Profile Structure 204
H.3 Uploaded Profile Annexures 204
H.4 Caldan Builder Service Categories from Source Profile 205
H.5 Selected Projects Listed in the Uploaded Company Profile 206
H.6 Resource Details from Uploaded Company Profile 207
H.7 Contact Details from Uploaded Company Profile 207
H.8 Shareholding Details from Annexure A 208
H.9 Credentials, References, and Banking Details from Annexures 208

List of Figures

1 Executive Vision Map — brand architecture and strategic outcomes 3

2.1 Caldan brand colors displayed as document swatches 12

3.1 Infrastructure Blueprint — secure cloud-ready delivery stack 23

6.1 Identity Control Layer — roles, OTP, permissions, and auditability 41

7.1 Authentication Flow – OTP Verification 43

8.1 Role Hierarchy 47
8.2 Chief Admin Creation Flow 48

9.1 OTP Verification Flow 54

10.1 Operations Engine — projects, inventory, finance, clients, and shop 63

11.1 Landing Page Section Flow 65

12.1 Project Hierarchy 70

13.1 Inventory Control Loop 76

20.1 Order Status Workflow 106

21.1 Base Dashboard Layout 109

23.1 Analytics Dashboard Layout 119

24.1 Three-Channel Notification Architecture 123

29.1 Report Generation Architecture 141

31.1 Data Export Flow 149

35.1 High-Level Entity Relationship Diagram 165

37.1 Testing Pyramid 172

A.1 Executive Command Center — leadership decisions flowing into delivery control 181
A.2 Brand Experience Board — voice, colors, typography, and trust signals 181
A.3 Delivery Roadmap — phased implementation from foundation to launch 182
A.4 Technology Layer Map — frontend, services, data, and external integrations . . 182
A.5 Django Service Mesh — domain apps connected through service classes and shared data 182
A.6 Database Relationship Lens — core entities, ownership, and operational boundaries 183
A.7 Security Trust Boundary — authentication, sessions, OTP, permissions, and audit zones 183

A.8 Security Defense Model — prevention, detection, recovery, audit, and continuity 184
A.9 Permission Governance Matrix — chief admin, admin, client, and system rights 184
A.10 Inventory Control Tower — materials, thresholds, stock movement, and alerts . 185
A.11 Expense Review Board — expense capture, categorization, approval, reporting 185
A.12 Revenue Snapshot — orders, payments, balances, and growth signals 186
A.13 Reports Factory — query, aggregate, render, export, and archive 186
A.14 Deployment Runway — environment setup through production handover . . . 186
A.15 Launch Readiness Radar — production criteria before public release 187

List of Code Listings

4.1 CSS Custom Properties for Theming . . . 27
6.1 .env.example – Environment Variable Template . . . 35
6.2 Security Middleware Configuration – settings.py . . . 36
6.3 Static Files Configuration – settings.py . . . 36
6.4 Automated Database Backup Script – backup-db.sh . . . 37
7.1 Custom User Model – accounts/models/user.py . . . 42
7.2 OTP Service – accounts/services/otp-service.py . . . 43
7.3 Session Settings – settings.py . . . 44
7.4 Trusted Device Implementation . . . 45
7.5 Login Rate Limiting – accounts/services/auth-service.py . . . 45
8.1 Invitation Model – accounts/models/invitation.py . . . 49
8.2 Permission Decorators – accounts/permissions/decorators.py . . . 51
8.3 Audit Logging – accounts/services/audit-service.py . . . 52
9.1 OTP Generation – accounts/services/otp-service.py . . . 53
9.2 OTP Email Template – templates/email/otp-email.html . . . 54
10.1 Setup View – accounts/views/-setup-views.py . . . 56
10.2 Invitation Service – accounts/services/invitation-service.py . . . 57
10.3 Force Password Change Middleware – accounts/middleware/force-password-change-middleware.py . . . 58
10.4 Admin Management View – accounts/views/-admin-views.py . . . 58
10.5 Admin Permissions View – accounts/views/-admin-views.py . . . 59
11.1 Trust Counter Context Processor – website/context-processors.py . . . 65
11.2 Landing Page Template – website/templates/website/index.html . . . 66
11.3 Service Model – services/models/service.py . . . 67
11.4 SEO Meta Tags – website/templates/website/base.html . . . 68
12.1 Project Model – projects/models/project.py . . . 70
12.2 Milestone Model – projects/models/milestone.py . . . 71
12.3 Progress Service – projects/services/progress-service.py . . . 72
12.4 Project Views – projects/views/-admin-views.py . . . 73
13.1 Material Model – inventory/models/material.py . . . 76
13.2 Stock Transaction Model – inventory/models/stock-transaction.py . . . 77
13.3 Stock Service – inventory/services/stock-service.py . . . 79
14.1 Payment Model – finances/models/payment.py . . . 82
14.2 Expense Model – finances/models/expense.py . . . 83
14.3 Budget Model – finances/models/budget.py . . . 84
14.4 Financial Service – finances/services/financial-service.py . . . 85
15.1 Client Model – clients/models/client.py . . . 87
15.2 Lead Service – clients/services/lead-service.py . . . 88
16.1 Team Member Model – team/models/team-member.py . . . 90
16.2 Team Management Views – team/views/-admin-views.py . . . 91
16.3 Public Team Display – website/templates/website/about.html . . . 91
17.1 Service Model – services/models/service.py . . . 93
17.2 Service Management Views – services/views/-admin-views.py . . . 94

18.1 Product Model – shop/models/product.py . . . 97
18.2 Product Views – shop/views/-public-views.py . . . 98
19.1 Cart Models – shop/models/cart.py . . . 100
19.2 Cart Service – shop/services/cart-service.py . . . 101
19.3 Checkout Views – shop/views/-checkout-views.py . . . 101
19.4 Checkout Service – shop/services/checkout-service.py . . . 102
20.1 Order Models – shop/models/order.py . . . 104
20.2 Order Management Views – shop/views/-admin-views.py . . . 105
20.3 Order Tracking – shop/views/-order-views.py . . . 106
21.1 Dashboard Context Processor – dashboard/context-processors.py . . . 109
21.2 Base Dashboard Template – dashboard/templates/dashboard/base-dashboard.html 110
21.3 Sidebar Menu Data – dashboard/data/sidebar-menu.py . . . 110
21.4 Theme System CSS – static/css/theme.css . . . 112
22.1 Dashboard Overview Template . . . 116
23.1 Analytics Service – dashboard/services/analytics-service.py . . . 119
23.2 Analytics Views – dashboard/views/-analytics-views.py . . . 120
24.1 In-App Notification Model – notifications/models/notification.py . . . 123
24.2 Notification Service – notifications/services/notification-service.py . . . 124
25.1 .env Configuration for Resend . . . 127
25.2 settings.py – Email Configuration . . . 127
25.3 Email Service – notifications/services/email-service.py . . . 127
25.4 Email Log Model – notifications/models/email-log.py . . . 129
26.1 SMS Service – notifications/services/sms-service.py . . . 131
26.2 SMS Log Model – notifications/models/sms-log.py . . . 133
27.1 Stock Alert Email – templates/email/stock-alert.html . . . 136
28.1 Reminder Service – notifications/services/reminder-service.py . . . 138
28.2 Celery Beat Schedule – celery.py . . . 139
29.1 Report Service – reports/services/report-service.py . . . 142
29.2 Report Views – reports/views/-admin-views.py . . . 143
30.1 PDF Service – reports/services/pdf-service.py . . . 145
30.2 Receipt PDF Template – templates/pdf/receipt.html . . . 146
30.3 Playwright Installation – requirements.txt . . . 147
31.1 Export Service – reports/services/export-service.py . . . 149
31.2 Backup Service – reports/services/backup-service.py . . . 151
31.3 Deletion Service – reports/services/deletion-service.py . . . 153
32.1 Audit Report Service – notifications/services/audit-report-service.py . . . 156
32.2 Bi-Weekly Audit Task – notifications/tasks.py . . . 157
33.1 HTTPS Configuration – settings.py . . . 158
33.2 CSP Configuration – settings.py . . . 159
33.3 Input Validation Example . . . 159
34.1 AuditLog Model – accounts/models/audit.py . . . 162
34.2 Audit Service – accounts/services/audit-service.py . . . 163
36.1 JWT Configuration – settings.py . . . 169
36.2 API Error Response Example . . . 170
37.1 OTP Service Tests – accounts/tests/test-otp-service.py . . . 173
37.2 API Integration Tests – api/tests/test-projects.py . . . 173
37.3 Login Flow E2E – tests/e2e/login.spec.js . . . 174
37.4 GitHub Actions CI – .github/workflows/ci.yml . . . 174

38.1 Dockerfile 176
38.2 Deployment Script – scripts/deploy.sh 177
38.3 Health Check – caldan-site/urls.py 177
38.4 Rollback Script – scripts/rollback.sh 178
B.1 Complete Project Directory 188

Important Notice

CONFIDENTIALITY AND PROPRIETARY NOTICE

This document contains confidential and proprietary information belonging to **Caldan Construction**. Any unauthorized use, disclosure, reproduction, or distribution is strictly prohibited and may result in legal action.

This document is intended solely for the internal use of the Caldan Construction development team and authorized stakeholders.

Document Control:

Document ID	CAL-SPEC-2026-07-V1.0
Classification	CONFIDENTIAL
Distribution	Internal Use Only – Do Not Distribute
Owner	Mukoya Khisa, Website Developer & Manager
Contact	sir.mukoyakhisa@gmail.com
Total Pages	~500
Last Updated	July 11, 2026
Status	Draft 1.0 – Complete Specification
Platform	Caldan Construction – Construction Management Platform
Target Market	Kenya
Tagline	"Building Kenya's Future, One Project at a Time."

Part I

Executive Summary & Brand Architecture

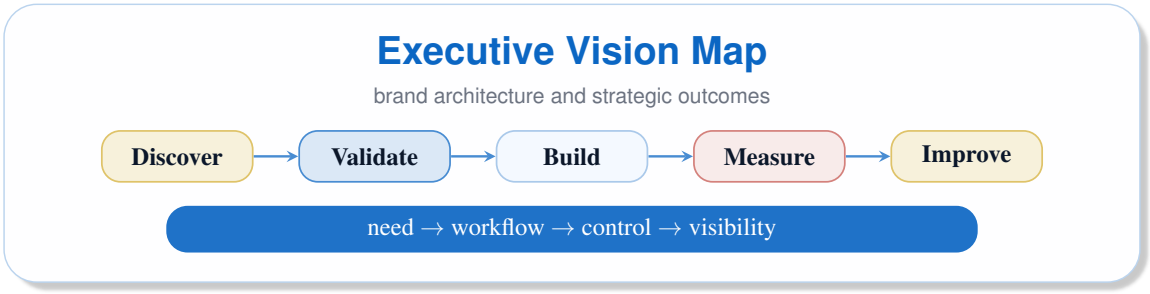


Figure 1: Executive Vision Map — brand architecture and strategic outcomes

Chapter 1

Executive Summary & Vision

1.1 The Vision

Caldan Construction is a comprehensive construction management and e-commerce platform designed and built in Kenya, for Kenya. The name Caldan represents the fusion of quality and craftsmanship—a name that stands for reliability, precision, and the enduring structures that shape communities.

Across civilizations, construction has represented far more than the assembly of materials. It has symbolized protection, permanence, and the places where lives, businesses, and communities flourish. At Caldan Construction, we carry that meaning into everything we build.

KEY INSIGHT Core Philosophy: Caldan Construction exists to bring order, accountability, and professionalism to the construction industry in Kenya—an industry that has long relied on paper records, verbal communication, and manual tracking.

Table 1.1: Platform Value Chain – From Construction Intent to Operational Excellence

Outcome	Platform Mechanism	Evidence
Project Visibility	Real-time progress tracking	Progress charts, milestone logs
Inventory Control	Automated stock management	Stock alerts, transaction records
Financial Clarity	Payment & expense tracking	Payment logs, budget reports
Team Accountability	Team management & tracking	Team profiles, activity logs
Operational Trust	Audit logs, secure access	Immutable activity trails
Client Satisfaction	Online shop, order tracking	Order history, delivery updates

1.2 The Problem Space

Kenya's construction industry is vast and growing. With rapid urbanization, infrastructure development, and a booming real estate sector, the demand for efficient construction management has never been higher.

Yet the tools available to many construction teams remain shockingly primitive:

Table 1.2: Current State of Construction Management

Challenge	Current Reality
Record-Keeping	Exercise books, Excel sheets on one phone, or no records at all
Material Tracking	No real-time quantities; theft and untracked movement
Team Management	No structured team tracking; manual oversight
Client Orders	Verbal requests; no order history
Project Progress	Verbal updates; no structured tracking
Communication	WhatsApp messages, phone calls, lost information
Reporting	Manual compilation; no analytics

The operating environment is fragmented and ready for an internal system that understands local site realities.

1.3 The Solution

Caldan Construction is a comprehensive internal web platform that digitizes construction management and e-commerce operations. It provides:

Table 1.3: Caldan Construction Core Capabilities

Capability	Description	Who Benefits
Project Management	Create and manage projects with milestones and progress tracking	Admin
Inventory Management	Track materials, stock in/out, low stock alerts	Admin
Team Management	Team profiles, roles, and activity tracking	Admin
Online Shop	Materials catalog, cart, checkout, orders	Admin, Clients
Financial Management	Payment and expense tracking	Admin
Client Management	Client database and lead tracking	Admin
Services Management	Manage services offered	Admin
Notifications	In-app, Email, SMS with Africa's Talking + Celcom	Admin
Reporting	Real-time reports, audit reports, exports	Admin
Data Backup	PDF, CSV, Excel, ZIP export, project archiving	Admin

1.4 Target Market

Caldan Construction serves the Kenyan construction ecosystem. The platform is built from the ground up for Kenyan realities:

Table 1.4: Target User Personas

User Type	Profile	Needs
Platform Admin	Company owner/manager overseeing operations	Full visibility, financial clarity, complete control
Client	Individuals or businesses ordering materials	Easy ordering, payment, order tracking

1.5 Client Delivery and Cost Model

Caldan Construction is a custom-built internal management system delivered to a single construction client.

Table 1.5: Internal System Cost Responsibilities

Item	Responsibility
Development Fee	One-time fee paid by the client for design, development, testing, deployment, and handover
Hosting	Paid directly by the client to the hosting provider
Domain	Purchased and renewed by the client
External APIs	Paid directly by the client (Paystack, Resend, SMS, Cloudinary)
System Users	Created internally by the Platform Admin; no self-service onboarding
Ongoing Support	Handled through a separate maintenance/support agreement if required

1.6 Technology Foundation

Caldan Construction is built on a modern, battle-tested technology stack:

Table 1.6: Core Technology Stack

Layer	Technology	Purpose
Backend Framework	Django 5.0+ (Python 3.11+)	Business logic, ORM, admin interface
Database	MySQL 8.0+	Relational data storage with full indexing
Frontend	HTML5, CSS3, JavaScript (ES6+)	Responsive, mobile-first user interface
Payments	Paystack	M-Pesa, card, and bank processing
Email	Resend	Transactional emails, receipts, OTP messages
SMS	Africa's Talking (primary) + Celcom (fallback)	Bulk SMS notifications with delivery logging
Media Storage	Cloudinary	Automatic CDN, on-the-fly transformations
PDF Generation	Playwright	Receipts, audit reports, exports
Task Scheduling	Celery + Redis	Automated reminders, weekly audit emails

1.7 Architecture Philosophy

1.7.1 Service-Oriented Architecture

Every business logic operation in Caldan Construction is encapsulated in service classes. Views are thin—they only handle request/response and call services. This ensures:

- **Separation of Concerns:** Business logic is isolated from presentation
- **Testability:** Services can be unit tested independently
- **Reusability:** Services can be called from multiple views/APIs
- **Maintainability:** Changes to business logic affect only services

1.7.2 Dynamic by Design

Every configurable element of Caldan Construction is stored as data, not hardcoded:

- System access settings – Internal records, editable via admin panel
- Notification templates – Editable templates, configurable per admin
- Theme preferences – Per-user light/dark/device default stored in profile

1.7.3 Kenya-First, Single-Market Focus

Caldan Construction is built exclusively for the Kenyan market. Every feature, every integration, every user experience decision is made with the client company and their clients in mind.

1.8 Scope of This Specification

This document provides the complete technical blueprint for Caldan Construction across 38 chapters:

Table 1.7: Document Structure Overview

Part	Chapters	Focus
Part I	1-3	Executive summary, brand architecture, platform overview
Part II	4-6	Technology stack, infrastructure, deployment
Part III	7-10	Authentication, user management, OTP, security
Part IV	11-17	Core business modules (Website, Projects, Inventory, etc.)
Part V	18-21	Online shop (Products, Cart, Orders, Payments)
Part VI	22-24	Admin dashboard (Architecture, Navigation, Analytics)
Part VII	25-28	Notification system (Email, SMS, Templates)
Part VIII	29-32	Reports, exports, PDF generation, audit reports
Part IX	33-34	Security architecture, audit & compliance
Part X	35-36	Database schema, API architecture
Part XI	37-38	Testing strategy, deployment & DevOps
Appendices	A-G	Directory, API endpoints, templates, matrices

1.9 Developer's Note

"Caldan Construction was conceived from a deep understanding of the Kenyan construction industry—an industry that has long relied on paper records, verbal communication, and manual tracking.

I witnessed firsthand how construction teams in Kenya managed their operations—with paper records, memory, and hope. The gap between what existed and what was possible was too large to ignore.

The name Caldan represents the fusion of quality and craftsmanship—a name that stands for reliability, precision, and the enduring structures that shape communities.

This platform exists to bring order, accountability, and professionalism to the construction industry in Kenya. Every project tracked, every material accounted for, every client served, and every payment recorded.

Behind every structure is a story. Caldan Construction is built for all of them."

Mukoya Khisa
Website Developer & Manager
Caldan Construction
Nairobi, Kenya
July 2026

Chapter Summary

Table 1.8: Executive Summary – Key Specifications

Property	**Value**
Platform Name	Caldan Construction
Tagline	"Building Kenya's Future, One Project at a Time."
Document ID	CAL-SPEC-2026-07-V1.0
Total Chapters	38
Total Pages	~500
Target Market	Kenya
Primary User	Platform Admin
Core Modules	8 (Website, Projects, Inventory, Shop, Finances, Clients, Reports, Notifications)

Chapter 2

Brand Architecture

2.1 The Name: Caldan Construction

The name **Caldan Construction** represents the fusion of quality and craftsmanship—a name that stands for reliability, precision, and the enduring structures that shape communities.

Table 2.1: Brand Name Components

Component	Origin	Intent
Caldan	Fusion of quality & craftsmanship	Emotional core – reliability, precision, trust
Construction	English	Functional clarity – what the company does
Full Name	Caldan Construction	Professional, established, industry-focused

2.2 Tagline

2.2.1 The Tagline

"Building Kenya's Future, One Project at a Time."

2.2.2 Tagline Philosophy

The tagline operates on multiple levels:

Table 2.2: Tagline Meaning Layers

Layer	Meaning	For Whom
Building Kenya's Future	National development, progress	All stakeholders
One Project at a Time	Attention to detail, quality focus	Clients, partners

2.3 Color Palette

The Caldan Construction color system now follows the official modern brand palette below. The specification uses this palette throughout covers, headings, links, cards, borders, diagrams, tables, and navigation elements so the document feels consistent with the Caldan identity rather than a generic traditional report.

Table 2.3: Official Caldan Brand Color Palette

Role	Swatch	Hex Code	Primary Usage
Primary	Royal Blue	#0B66C3	Main links, section accents, chapter bars, key diagrams, active states
Secondary	Deep Navy	#123B73	Structural headings, navigation blocks, executive panels, diagram anchors
Premium Accent	Gold	#D4AF37	Premium highlights, badges, CTA accents, divider lines, figure emphasis
Background	White	#FFFFFF	Main page surfaces, covers, cards, and clean reading areas
Alternate Background	Soft Slate	#F8FAFC	Light panels, appendix sections, diagram frames, subtle page contrast
Cards	White	#FFFFFF	Tables, content cards, boxed summaries, figure containers
Borders	Light Gray	#E5E7EB	Table rules, card outlines, figure frames, quiet separators
Primary Text	Dark Gray	#1F2937	Body copy, table content, captions, readable long-form text
Secondary Text	Medium Gray	#6B7280	Subtitles, metadata, helper text, muted explanatory copy
Footer	Deep Footer Navy	#0B172A	Footer styling, dark contrast elements, deep executive accents



Figure 2.1: Caldan brand colors displayed as document swatches

2.3.1 Color Psychology

Table 2.4: Color Psychology

Color	**Psychological Association**	**Application**
Royal Blue	Trust, clarity, modern digital confidence	Primary identity, links, chapter accents
Gold	Value, quality, premium experience	CTAs, badges, highlights
Red Accent	Urgency, action, warning	Alerts, errors, urgent notifications
Soft Slate	Cleanliness, breathing room, modern light interface	Alternate backgrounds, panels, figure frames
Dark Gray	Readability, clarity, professionalism	Body text, content

2.3.2 Color Application Rules

Table 2.5: Color Usage Guidelines

Context	Color Rule	Example
Dashboard (Light)	White background, royal-blue navigation, gold active items	Navigation, cards, content
Dashboard (Dark)	Deep-navy footer surfaces, white cards, gold CTAs	Dark mode toggle
Corporate Website	White background, royal-blue headers, gold accents	Services, about, contact
Alerts	Red accent for errors, Gold for warnings	Toast notifications, banners
Buttons	Royal Blue for primary actions, Deep Navy for secondary actions, Gold for premium emphasis	CTA buttons, form actions
Badges	Gold for verified, Red for urgent, Royal Blue for status	Status indicators

2.4 Logo Design

The Caldan Construction logo consists of two primary elements: an abstract geometric icon and the wordmark "Caldan Construction."

Table 2.6: Logo Variants

Variant	Usage	Background
Full Logo (Primary)	Primary usage	Light backgrounds (cream, white)
Full Logo (Light)	Dark mode interfaces	Dark backgrounds (teal, near-black)
Icon Only	Favicon, PWA manifest, app icon	Any
Wordmark Only	Limited space applications	Any

2.4.1 Design Principles

- **Abstract, Not Literal:** The logo icon uses abstract geometric shapes that subtly suggest shelter or construction
- **Scalable:** The mark remains legible from favicon size (16x16px) to hero banners (1200x630px)
- **Theme-Aware:** A single primary logo file is used across the internal access portal and dashboard
- **Modern Sans-Serif Typography:** Clean, geometric sans-serif consistent with modern software branding

2.5 Typography

2.5.1 Primary Font: Inter

Inter is a modern, open-source sans-serif typeface designed specifically for user interfaces.

Table 2.7: Typography Hierarchy

Element	Font	Weight	Size / Style
Page Headings	Inter	Bold (700)	2rem-3rem, dark teal
Section Headings	Inter	Semi-Bold (600)	1.5rem-1.75rem, dark teal
Body Text	Inter	Regular (400)	1rem, dark charcoal
Captions/Meta	Inter	Regular (400)	0.875rem, muted gray
Buttons	Inter	Medium (500)	0.875rem-1rem, white or dark teal
Code	Courier New / Fira Code	Regular (400)	0.875rem, monospace

2.6 Iconography

All icons throughout the Caldan Construction platform use **Font Awesome 6 (Free tier)**. No emoji characters are used.

Table 2.8: Icon Mapping – Dashboard Navigation

Navigation Item	Icon Class	Family
Dashboard Overview	fa-tachometer-alt	fas
Projects	fa-building	fas
Inventory	fa-boxes	fas
Shop Orders	fa-shopping-cart	fas
Finances	fa-money-bill-wave	fas
Clients	fa-users	fas
Team	fa-user-tie	fas
Reports	fa-chart-bar	fas
Settings	fa-cog	fas

Table 2.9: Icon Mapping – Actions & States

Action/State	Icon Class	Family
Add/Create	fa-plus	fas
Edit	fa-edit	fas
Delete	fa-trash	fas
Download	fa-download	fas
Export CSV	fa-file-csv	fas
Export PDF	fa-file-pdf	fas
Notifications	fa-bell	fas
Theme Toggle (Light)	fa-sun	fas
Theme Toggle (Dark)	fa-moon	fas
Verified Badge	fa-check-circle	fas

2.7 Tone of Voice

The Caldan Construction brand communicates with a distinct voice:

Table 2.10: Brand Voice Attributes

Attribute	Expression	Example
Professional	Precise, clear, trustworthy	"Track every project with real-time updates."
Human	Acknowledges human element	"Building Kenya's Future, One Project at a Time."
Confident	No hedging language	"Caldan Construction digitizes your operations."
Kenyan	Speaks naturally to Kenyan audience	"M-Pesa, Paystack, Kenyan construction reality."
Modern	Clean, contemporary, no bureaucracy	"One platform. Every project. Every client."

2.8 Brand Assets Summary

Table 2.11: Complete Brand Asset Inventory

Category	Asset	Status
Name	Caldan Construction	Finalized
Tagline	"Building Kenya's Future, One Project at a Time."	Finalized
Primary Domain	caldanconstruction.com	To Register
Contact Email	sir.mukoyakhisa@gmail.com	Active
Logo	Full logo for light/dark backgrounds	Draft Created
Favicon	32x32px icon	Created
Color Palette	8-color system	Finalized
Typography	Inter (UI), Times/Helvetica/Courier (Documentation)	Finalized
Icon Library	Font Awesome 6 (Free)	Finalized

Chapter Summary

Table 2.12: Brand Architecture – Key Specifications

Property	Value
Name	Caldan Construction
Tagline	"Building Kenya's Future, One Project at a Time."
Domain	caldanconstruction.com
Primary Colors	Dark Teal (#0D2B36), Gold (#D69723), Red Accent (#B9342B), Warm Cream (#F6F1E8), Dark Charcoal (#1F2937)
Typography	Inter (UI), Times New Roman (Documentation)
Iconography	Font Awesome 6 (Free)
Tone	Professional, Human, Confident, Kenyan, Modern

Chapter 3

Platform Overview

One platform. Every project. Every client.

3.1 Platform Architecture Philosophy

Caldan Construction is not a single application—it is an ecosystem of interconnected experiences serving two distinct user groups. The platform is architected as two primary environments sharing a common backend:

Table 3.1: Platform Environment Architecture

Environment	Description	Primary Users
Corporate Website	Hero, services, about, values, contact, footer	Visitors, Clients
Dashboard	Authenticated management interface with role-based views	Platform Admin

Both environments share the same Django backend, database, and API layer. The distinction is purely in presentation, authentication requirements, and available functionality.

3.2 User Types and Role Hierarchy

Caldan Construction defines user roles with specific access levels:

Table 3.2: User Role Definitions

Role	Access Level	Primary Functions
Platform Admin	Full (Superuser)	Manage all projects, inventory, shop, payments, reports, settings, team
Client	Limited (Shop only)	Browse products, place orders, track orders

KEY INSIGHT Core Principle: The Platform Admin is the sovereign administrator of their construction operations. They create all user accounts, configure permissions, and have full visibility across all modules. No other user can exceed the access level granted by the Platform Admin.

3.3 Corporate Website Architecture

The corporate website presents Caldan Construction services, story, values, and contact information.

Table 3.3: Corporate Website Page Inventory

Page	URL Pattern	Purpose
Landing Page	/	Hero section with company name, tagline, services overview
About	/about/	Company story, mission, and values
Services	/services/	Construction services and capabilities overview
Projects	/projects/	Project gallery with filters
Shop	/shop/	Materials catalog with categories and search
Contact	/contact/	Contact form, company contacts, and CTA section
Footer	Global	Company info, location, contact details
Staff Login	/login/	Discrete navbar text link for internal staff authentication
Legal	/legal/privacy/	Privacy policy, Terms of service

3.4 Dashboard Architecture

The dashboard is the authenticated operational environment. Upon login, the system detects the user's role and renders the appropriate dashboard view.

3.4.1 Dashboard Routing Logic

1. User submits login credentials
2. System validates credentials
3. 8-digit OTP sent to email
4. User verifies OTP
5. System detects user role
6. System renders appropriate dashboard

3.4.2 Platform Admin Dashboard – Full Access

The Platform Admin dashboard is the most comprehensive view in the platform. It provides:

- **Overview:** Stats cards (projects, stock, orders, revenue), charts, activity feed, quick actions

- **Sidebar Navigation:** 80+ items covering all platform modules
- **Projects:** Full CRUD with milestones, progress tracking, timeline views
- **Inventory:** Complete material management with stock in/out, alerts, suppliers
- **Shop:** Product catalog, orders, inventory integration
- **Finances:** Payment recording, expense tracking, budget management
- **Clients:** Client database and lead tracking
- **Team:** Team member management
- **Services:** Service management
- **Reports:** Project reports, inventory reports, financial reports
- **Data Management:** Backup, export, deletion
- **Settings:** General, appearance, admins, notifications, security

3.4.3 Feature Taxonomy – Complete

Table 3.4: Complete Feature Inventory by Module

Module	Features	Users
Authentication	Login, OTP, password reset, session management, admin invitations	Admin
Projects	CRUD, milestones, progress tracking, timeline, budget	Admin
Inventory	CRUD, stock in/out, alerts, suppliers, deliveries	Admin
Shop	Product catalog, cart, checkout, orders, tracking	Admin, Clients
Finances	Payment recording, expense tracking, budgets	Admin
Clients	CRUD, lead tracking, contact forms	Admin
Team	CRUD, profiles, roles	Admin
Services	CRUD, categories	Admin
Notifications	In-app, email (Resend), SMS (Africa's Talking), reminders	Admin
Reports	Project, inventory, financial, export	Admin
Data Management	Backup, export (PDF/CSV/Excel/ZIP), deletion	Admin
Settings	General, appearance, admins, notifications, security	Admin
Corporate Website	Services, about, values, contact, staff login	Visitors, Clients

Chapter Summary

Table 3.5: Platform Overview – Key Specifications

Property	**Value**
Roles	2: Platform Admin, Client
Dashboards	1: Platform Admin (80+ pages)
Authentication	Email + Password + 8-digit OTP
Notifications	3 channels: In-app, Email (Resend), SMS (Africa's Talking + Celcom)
Data Export	PDF, CSV, Excel, ZIP
Mobile	PWA with offline support
Storage	Cloudinary (media), MySQL (data)

Part II

Technology Stack & Infrastructure

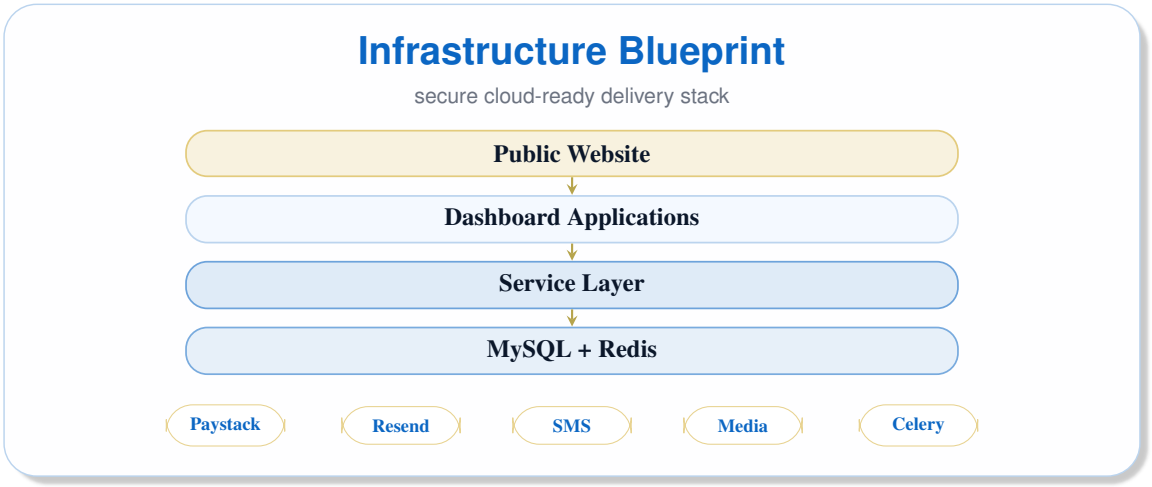


Figure 3.1: Infrastructure Blueprint — secure cloud-ready delivery stack

Chapter 4

Technology Stack

4.1 Selection Philosophy

Every technology in the Caldan Construction stack was chosen against four criteria:

Table 4.1: Technology Selection Criteria

Criterion	Definition	Why It Matters
Solo-Developer Friendly	Can be learned and maintained by one developer	Reduces development complexity
Kenyan-Market Appropriate	Integrates with Kenyan payment, SMS, and regulatory systems	M-Pesa, Paystack, local realities
Production-Proven	Track record of powering real businesses	Not experimental or hype-driven
Cost-Conscious	Free tiers, open-source, pay-as-you-grow	Near-zero operational costs

4.2 Technology Stack Overview

Table 4.2: Complete Technology Stack – All Layers

Layer	Technology	Purpose
Backend Framework	Django 5.0+ (Python 3.11+)	Business logic, ORM, admin interface
API Layer	Django REST Framework 3.14+	RESTful API endpoints
Database	MySQL 8.0+	Relational data storage with full indexing
Frontend	HTML5, CSS3, JavaScript (ES6+)	Responsive, mobile-first user interface
CSS Architecture	CSS Custom Properties	Theme system (Light/Dark/Device)
Icons	Font Awesome 6 (Free)	Professional iconography
Payments	Paystack	M-Pesa STK Push, card payments, bank transfers
Email	Resend	Transactional emails, receipts, OTP messages
SMS	Africa's Talking (primary) + Celcom (fallback)	Bulk SMS notifications with delivery logging
Media Storage	Cloudinary	Automatic CDN, on-the-fly transformations
PDF Generation	Playwright	Receipts, audit reports, exports
Task Scheduling	Celery + Redis	Automated reminders, weekly audit emails
Deployment	Railway	Simple deployment, auto-scaling
Domain	Namecheap	Domain registration (.com)

4.3 Backend: Django

4.3.1 Why Django

- **Batteries-Included:** ORM, admin interface, authentication, form handling, template engine—all built-in
- **Security by Default:** XSS protection, CSRF protection, SQL injection prevention—enabled out of the box
- **Admin Interface:** Automatic admin panel for platform managers
- **Migration System:** Version-controlled database schema changes
- **Python Ecosystem:** Full access to Python's library ecosystem
- **Kenyan Developer Community:** Large pool of Django developers in Kenya

4.3.2 Django Apps Architecture

The Caldan Construction backend is organized into 12 Django applications:

Table 4.3: Django Application Architecture

App	Responsibility	Key Models
accounts	User management, authentication, OTP, audit	User, Profile, Invitation, AuditLog
website	Corporate website, contact, testimonials	Contact, Testimonial
shop	Online store, products, cart, orders	Product, Cart, Order
services	Service management	Service
dashboard	Admin dashboard (80+ pages)	(No models – views only)
projects	Project management, milestones, progress	Project, Milestone, Gallery
inventory	Materials, stock in/out, alerts, suppliers, deliveries	Material, Supplier, StockTransaction, Delivery
finances	Payment recording, expenses, budgets	Payment, Expense, Budget
clients	Client management	Client
team	Team member management	TeamMember
reports	Reports, analytics, data export	ScheduledReport, BackupRecord
notifications	3-channel notifications, reminders	Notification, SMSLog, EmailLog

4.4 Database: MySQL

4.4.1 Why MySQL

- **Relational Integrity:** Foreign keys, transactions, constraints ensure data consistency
- **Performance:** Excellent read performance for dashboard queries
- **Ubiquity:** Supported by every major hosting provider
- **Django Compatibility:** Django ORM has first-class MySQL support
- **Cost:** Open source, no licensing fees

INFORMATION Character Set: All database tables use utf8mb4 encoding to support the full Unicode character set, including Kenyan shilling symbol (KSh) and any future multilingual content.

4.5 Frontend: HTML, CSS, JavaScript

4.5.1 Design Principles

- **No Frontend Framework:** Vanilla HTML, CSS, JavaScript—no React, Vue, or Angular
- **Server-Side Rendering:** Django templates render HTML on the server
- **Progressive Enhancement:** JavaScript enhances the experience but core CRUD works without it
- **Mobile-First CSS:** All layouts designed for mobile screens first, then enhanced for desktop
- **CSS Custom Properties:** Theme system implemented entirely through CSS variables

4.5.2 CSS Architecture

```
/* Light Mode (default) */
:root, [data-theme="light"] {
    --bg-primary: #FCF9F4;
    --bg-secondary: #F6F1E8;
    --bg-card: #FFFFFF;
    --text-primary: #1F2937;
    --text-secondary: #6B7280;
    --border-primary: #E5E7EB;
    --accent-primary: #0D2B36;
    --accent-secondary: #D69723;
    --accent-danger: #B9342B;
    --accent-success: #27AE60;
    --shadow: rgba(0,0,0,0.05);
}

/* Dark Mode */
[data-theme="dark"] {
    --bg-primary: #0D1F2A;
    --bg-secondary: #1A3A3A;
    --bg-card: #1E3A3A;
    --text-primary: #F1F5F9;
    --text-secondary: #94A3B8;
    --border-primary: #334155;
    --accent-primary: #2A5A5A;
    --accent-secondary: #D69723;
    --accent-danger: #E74C3C;
    --shadow: rgba(0,0,0,0.3);
}
```

Listing 4.1: CSS Custom Properties for Theming

4.6 Payments: Paystack

4.6.1 Why Paystack

- **Kenyan Market Presence:** Deep integration with M-Pesa STK Push, card networks, bank transfers

- **Developer Experience:** Clean REST API, Python SDK, well-documented webhook system
- **Client Cost Records:** Optional support for recording project payments and operating costs
- **Regulatory Compliance:** PCI-DSS Level 1 certified
- **Solo-Developer Friendly:** Straightforward API with sandbox testing environment

4.7 Email: Resend

4.7.1 Why Resend

- **Modern API:** Clean REST interface designed for transactional email
- **Generous Free Tier:** 3,000 emails/month free
- **Python SDK:** Official resend Python package
- **Template Support:** HTML email templates with variable interpolation
- **Deliverability:** Modern email infrastructure with high inbox placement

4.8 SMS: Africa's Talking with Celcom Fallback

4.8.1 Provider Strategy

Table 4.4: SMS Provider Strategy

Provider	Role	When Used
Africa's Talking	Primary	Development, sandbox testing, launch, normal transactional SMS
Celcom Africa	Fallback	Higher-volume operations, negotiated support, redundancy

4.9 Media Storage: Cloudinary

Table 4.5: Cloudinary Benefits

Benefit	Description	Why It Matters
Automatic CDN	Images served from global CDN	Fast loading for all users
Transformations	Resize, crop, optimize via URL	No need for multiple image sizes
Secure Storage	Private access controls	KYC documents, receipts
Cost-Effective	25GB free storage	Bootstrapped phase viability
Django Integration	Native django-cloudinary-storage	Seamless development experience

4.10 Task Scheduling: Celery + Redis

4.10.1 Scheduled Tasks

Table 4.6: Automated Scheduled Tasks

Task	Cadence	Description
Low Stock Alerts	Daily (morning)	Check all materials, send alerts below threshold
Order Confirmation Emails	Immediate	Send order confirmation to client
Bi-Weekly Audit Report	Every 14 days	Generate and email audit report
Database Backup	Daily (2 AM)	Automated database backup
Session Cleanup	Daily (night)	Clear expired user sessions

Chapter Summary

Table 4.7: Technology Stack – Key Specifications

Property	**Value**
Backend	Django 5.0+ (Python 3.11+)
Database	MySQL 8.0+
Frontend	HTML5, CSS3, JavaScript (ES6+)
Payments	Paystack (M-Pesa, Card, Bank)
Email	Resend (3,000/month free)
SMS	Africa's Talking + Celcom (fallback)
Media Storage	Cloudinary (25GB free)
PDF Generation	Playwright (headless Chromium)
Task Queue	Celery + Redis
Deployment	Railway
Domain	Namecheap (.com)

Chapter 5

Django Application Architecture

5.1 Overview

Caldan Construction is built using a modular Django application architecture consisting of 12 distinct apps. Each app is responsible for a specific platform domain and follows separation of concerns: models define data, views handle requests, forms validate input, services hold business logic, and templates render interfaces.

KEY INSIGHT Design Philosophy: Each app is self-contained with its own models, views, forms, services, and templates. Apps communicate through well-defined service interfaces instead of tightly coupled cross-app logic. This keeps the platform maintainable, testable, and easier to extend.

5.2 Application Inventory

Table 5.1: Complete Django App Summary

App	Domain	Primary Responsibility
accounts	Security	Internal users, invitations, OTP verification, sessions, audit logging
website	Public	Corporate website, services, contact, testimonials
shop	E-commerce	Products, cart, checkout, orders, payments
services	Services	Service catalog, categories, service display
dashboard	Interface	Admin dashboard, stats cards, sidebar, activity feeds
projects	Projects	Project CRUD, milestones, progress tracking, gallery
inventory	Materials	Materials, suppliers, stock IN/OUT, stock alerts, deliveries
finances	Finance	Payments, expenses, budgets, financial reports
clients	Clients	Client database, lead tracking, contact forms
team	Team	Team member profiles, roles, management
reports	Analytics	PDF/CSV exports, backups, scheduled reports
notifications	Messaging	In-app notifications, SMS logs, email logs, reminders

5.3 Service-Oriented App Pattern

Each app follows a standard structure:

Table 5.2: Standard App Structure Pattern

Layer	Purpose
models/	Database entities, relationships, indexes, model-level methods
views/	Request handling and page rendering (role-specific)
forms/	Input validation and form presentations
services/	Business rules, calculations, integrations, workflows
templates/	Dashboard pages, partials, shared UI components
urls.py	App-specific route definitions
admin.py	Admin registrations and internal management helpers

5.4 Cross-App Interaction Rules

- The dashboard reads app statistics through service classes, not raw scattered queries
- Notifications are routed through the notifications app so email, SMS, and in-app messages stay consistent
- Reports aggregate data through services and export layers rather than duplicating business logic
- Audit logging captures sensitive account, payment, and permission events
- Inventory integrates with shop for stock management

5.5 Architecture Summary

Table 5.3: Django Application Architecture Summary

Property	Value
Total Django Apps	12
Primary Pattern	Models + views + forms + services + templates per app
Authentication Boundary	Internal-only accounts created by Platform Admin
Core Dependency	accounts app provides auth, audit, invitations
Interface Boundary	dashboard app renders 80+ admin pages

Chapter 6

Domain, Infrastructure & Deployment

6.1 Domain Strategy

6.1.1 Primary Domain

The primary domain for Caldan Construction is:

caldanconstruction.com

6.1.2 Domain Registration

Table 6.1: Domain Registration Details

Domain	Registrar	Annual Cost (Approx.)
`caldanconstruction.com`	Namecheap	KES 1,500–2,000

6.2 Email Configuration

6.2.1 Transactional Email Address

For the initial launch phase, Caldan Construction will use:

sir.mukoyakhisa@gmail.com

> **INFORMATION** Design Decision: Custom domain email via Google Workspace or Zoho Mail is deferred until post-launch revenue justifies the additional cost (KES 500-1,500/month). Resend is configured to send from this address with proper SPF and DKIM authentication.

6.3 Environment Configuration

6.3.1 The .env File

All sensitive configuration values are stored in a single .env file at the project root:

```
# CALDAN CONSTRUCTION - ENVIRONMENT VARIABLES
# ======================================================================

# DJANGO CORE
SECRET_KEY=django-insecure-your-secret-key-here
DEBUG=True
ALLOWED_HOSTS=localhost,127.0.0.1

# DATABASE (MySQL)
DB_NAME=caldan_construction
DB_USER=caldan_user
DB_PASSWORD=your_strong_db_password
DB_HOST=localhost
DB_PORT=3306

# PAYSTACK
PAYSTACK_SECRET_KEY=sk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
PAYSTACK_PUBLIC_KEY=pk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
PAYSTACK_LIVE=False

# EMAIL (Resend)
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
SEND_REAL_EMAILS=False

# SMS PROVIDER (Options: africastalking OR celcom)
SMS_PROVIDER=africastalking

# AFRICA'S TALKING (Primary SMS Provider)
AFRICASTALKING_USERNAME=sandbox
AFRICASTALKING_API_KEY=atsk_sandbox_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
AFRICASTALKING_SANDBOX=True

# CELCOM AFRICA (Fallback SMS Provider)
CELCOM_API_URL=https://isms.celcomafrica.com/api/services/sendsms
CELCOM_API_KEY=your_celcom_live_api_key_here
CELCOM_PARTNER_ID=your_celcom_partner_id_here
CELCOM_SHORTCODE=CALDAN

# SMS DELIVERY SETTINGS
SEND_REAL_SMS=False

# CLOUDINARY (Media Storage)
CLOUDINARY_CLOUD_NAME=your_cloud_name_here
CLOUDINARY_API_KEY=0000000000000000
CLOUDINARY_API_SECRET=your_cloudinary_api_secret_here

# REDIS (Cache + Celery Broker)
REDIS_URL=redis://localhost:6379/0

# DEPLOYMENT
DJANGO_SETTINGS_MODULE=caldan_site.settings
ENVIRONMENT=development
BASE_URL=http://localhost:8000

# CALDAN-SPECIFIC CONFIGURATION
CALDAN_OTP_EXPIRY_MINUTES=10
CALDAN_GRACE_PERIOD_HOURS=48
```

Listing 6.1: .env.example – Environment Variable Template

6.4 Hosting Infrastructure

6.4.1 Phase 1: Bootstrapped Hosting (Launch)

For the initial launch phase, Caldan Construction will be deployed on Railway:

Table 6.2: Hosting Options – Bootstrapped Phase

Provider	**Monthly Cost**	**Notes**
Railway	$5-20/month	Simple deployment, auto-scaling, PostgreSQL/MySQL support

6.4.2 Phase 2: Scaled Infrastructure (Post-Traction)

Table 6.3: Hosting Evolution Path

Phase	Infrastructure	Trigger
Bootstrapped	Railway App Platform ($12/mo)	Launch
Growing	Railway + managed MySQL ($30-50/mo)	50+ projects
Scaling	Multiple instances + load balancer ($100-200/mo)	500+ projects

6.5 SSL & Security

6.5.1 HTTPS Everywhere

All traffic to caldanconstruction.com is encrypted via HTTPS:

```
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000  # 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

Listing 6.2: Security Middleware Configuration – settings.py

6.6 Static & Media File Storage

6.6.1 Static Files (CSS, JS, Images)

During the bootstrapped phase, static files are served by Whitenoise:

```
# Static files (CSS, JavaScript, Images)
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_DIRS = [BASE_DIR / 'static']

# Whitenoise
MIDDLEWARE = [
    'whitenoise.middleware.WhitenoiseMiddleware',
    # ... other middleware ...
]
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

Listing 6.3: Static Files Configuration – settings.py

6.6.2 Media Files (User Uploads)

User-uploaded files are stored using Cloudinary:

Table 6.4: Media Storage Strategy

Phase	Storage Solution	Notes
Production	Cloudinary primary storage	No local media/ folder
Growing	Cloudinary + DigitalOcean Spaces backup	250GB storage
Scale	AWS S3 or Google Cloud Storage	With CDN

6.7 Backup Strategy

6.7.1 Database Backups

Table 6.5: Backup Schedule

Backup Type	Frequency	Method
Full Database Dump	Daily (automated)	mysqldump via cron job, stored to cloud storage
Full Database Dump	Weekly (automated)	Compressed archive retained for 90 days
Manual Backup	Before deployments	Full dump triggered manually

```
#!/bin/bash
# /opt/scripts/backup_db.sh
# Run daily via cron: 0 2 * * * /opt/scripts/backup_db.sh

BACKUP_DIR="/backups/mysql"
DATE=$(date +%Y%m%d_%H%M%S)
DB_NAME="caldan_construction"
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_full_${DATE}.sql.gz"

# Create backup directory if it doesn't exist
mkdir -p $BACKUP_DIR

# Create backup with compression
mysqldump --single-transaction --quick \
    --routines --triggers \
    $DB_NAME | gzip > $BACKUP_FILE

# Keep only last 30 days of daily backups
find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete

# Log backup result
if [ -f $BACKUP_FILE ]; then
    SIZE=$(du -h $BACKUP_FILE | cut -f1)
    echo "[$(date)] Backup successful: $BACKUP_FILE ($SIZE)" >> /var/log/caldan_backup.log
else
    echo "[$(date)] Backup FAILED: $DB_NAME" >> /var/log/caldan_backup.log
    exit 1
fi
```

Listing 6.4: Automated Database Backup Script – backup-db.sh

6.8 Monitoring & Logging

6.8.1 Log File Inventory

Table 6.6: Log Files

Log File	Content	Retention
debug.log	General application debug information	30 days rolling
error.log	Application errors and exceptions	90 days rolling
audit.log	All user actions	365 days
payments.log	Payment transaction logs	365 days
shop.log	Shop order logs	90 days
sms.log	SMS sending logs	90 days
email.log	Email sending logs	90 days

6.9 Deployment Workflow

6.9.1 Deployment Process

1. **Local Testing:** Feature developed and tested locally with MySQL
2. **Static Collection:** `python manage.py collectstatic` – aggregates all static files
3. **Migration Check:** `python manage.py makemigrations –check`
4. **Git Push:** Code pushed to GitHub repository (private)
5. **Deploy Trigger:** Platform detects push to main branch and auto-deploys
6. **Post-Deploy:** `python manage.py migrate` runs automatically
7. **Smoke Test:** Manual verification of login, OTP, and key features

6.10 Infrastructure Cost Summary

Table 6.7: Estimated Monthly Infrastructure Costs – Bootstrapped Phase

Service	**Plan**	**Monthly Cost (KES)**
Hosting (Railway)	Starter	KES 1,500
Domain (caldanconstruction.com)	Namecheap (annual / 12)	KES 150
Database	Included in hosting	KES 0
SSL Certificate	Let's Encrypt (auto)	KES 0
Paystack	Per-transaction only	KES 0 (until first payment)
Resend (Email)	Free tier (3,000/month)	KES 0
SMS Provider	Pay-per-message	Variable
Cloudinary	Free tier (25GB)	KES 0
Total Fixed Monthly Cost		**KES 1,650**

Chapter Summary

Table 6.8: Domain, Infrastructure & Deployment – Key Specifications

Property	**Value**
Primary Domain	caldanconstruction.com
Hosting Provider	Railway (bootstrapped phase)
SSL	Let's Encrypt with Cloudflare
Database	MySQL 8.0 (managed)
Media Storage	Cloudinary (primary)
Backup Schedule	Daily full backups, 30-day retention
Logging	7 log files with 30-365 day retention
Deployment	Railway automatic deployment from GitHub
Fixed Monthly Cost	KES 1,650

Part III

Authentication & User Management

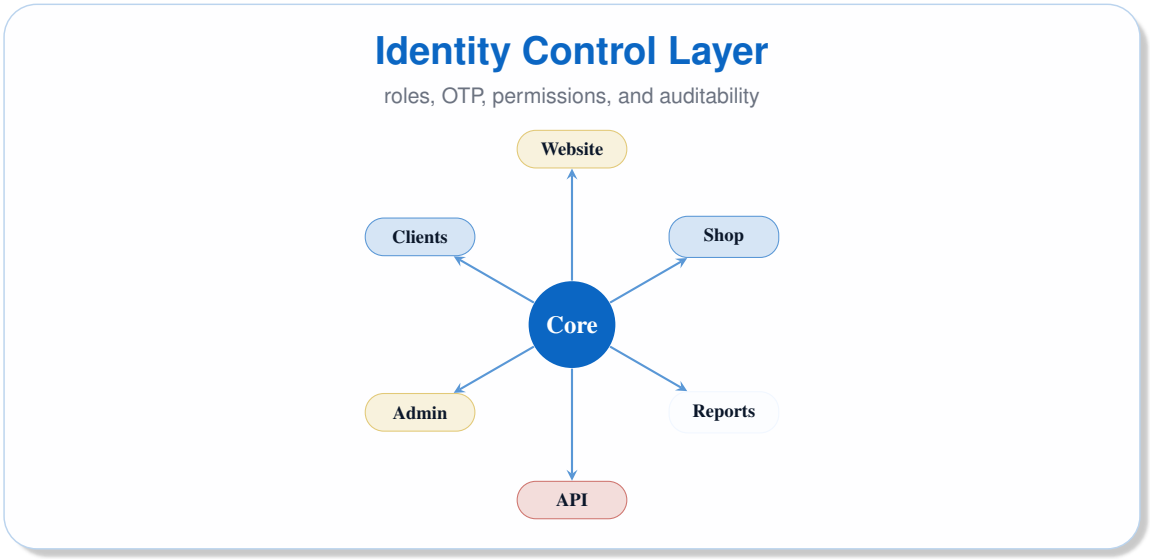


Figure 6.1: Identity Control Layer — roles, OTP, permissions, and auditability

Chapter 7

Authentication System

7.1 Overview

The Caldan Construction authentication system handles user identity verification, session management, and role-based access routing. Every login requires:

1. Email address
2. Password
3. 8-digit OTP sent to the user's email address

After verification, the system redirects the user to the Platform Admin dashboard.

7.2 Custom User Model

Caldan Construction uses a custom User model rather than Django's default:

```
from django.db import models
from django.contrib.auth.models import AbstractBaseUser, BaseUserManager
from django.utils import timezone

class UserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError('Email is required')
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        extra_fields.setdefault('is_chief', True)
        return self.create_user(email, password, **extra_fields)

class User(AbstractBaseUser):
    """Custom User model for Caldan Construction."""

    class Role(models.TextChoices):
        ADMIN = 'ADMIN', 'Platform Admin'

    # Primary identifier
    email = models.EmailField(unique=True, db_index=True)

    # Identity
    full_name = models.CharField(max_length=150)
    role = models.CharField(max_length=20, choices=Role.choices, default=Role.ADMIN)
    phone = models.CharField(max_length=15)

    # Status flags
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    is_superuser = models.BooleanField(default=False)
    is_chief = models.BooleanField(default=False)  # First admin - cannot be deleted
    is_email_verified = models.BooleanField(default=False)

    # OTP fields
    otp_code_hash = models.CharField(max_length=64, null=True, blank=True)
    otp_created_at = models.DateTimeField(null=True, blank=True)
```

```
    otp_attempts = models.PositiveSmallIntegerField(default=0)
    otp_resend_count = models.PositiveSmallIntegerField(default=0)
    otp_last_resent = models.DateTimeField(null=True, blank=True)

    # Trusted devices (JSON)
    trusted_devices = models.JSONField(default=dict)

    # Login tracking
    failed_login_attempts = models.PositiveSmallIntegerField(default=0)
    locked_until = models.DateTimeField(null=True, blank=True)
    last_login = models.DateTimeField(null=True, blank=True)

    # Timestamps
    date_joined = models.DateTimeField(default=timezone.now)
    updated_at = models.DateTimeField(auto_now=True)

    objects = UserManager()

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['full_name', 'phone']

    class Meta:
        db_table = 'users'
        verbose_name = 'User'
        verbose_name_plural = 'Users'

    def __str__(self):
        return f"{self.full_name} ({self.get_role_display()})"

    def get_short_name(self):
        return self.full_name.split()[0] if self.full_name else self.email

    def is_admin(self):
        return self.role == self.Role.ADMIN
```

Listing 7.1: Custom User Model – accounts/models/user.py

7.3 OTP Verification Flow

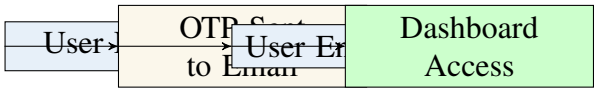


Figure 7.1: Authentication Flow – OTP Verification

7.3.1 8-Digit OTP Generation

```
import secrets
import hashlib
from django.utils import timezone
from datetime import timedelta

class OTPService:
    """Handles 8-digit OTP generation and verification."""

    OTP_LENGTH = 8
    OTP_EXPIRY_MINUTES = 5
    MAX_ATTEMPTS = 3
    MAX_RESENDS_PER_HOUR = 5
    RESEND_COOLDOWN_SECONDS = 60

    @classmethod
    def generate_otp(cls, user):
        """Generate and store 8-digit OTP for user."""
        # Generate random 8-digit code
        otp_code = ''.join(str(secrets.randbelow(10)) for _ in range(cls.OTP_LENGTH))

        # Hash and store
        user.otp_code_hash = hashlib.sha256(otp_code.encode()).hexdigest()
        user.otp_created_at = timezone.now()
        user.otp_attempts = 0
        user.save(update_fields=['otp_code_hash', 'otp_created_at', 'otp_attempts'])
        return otp_code

    @classmethod
    def verify_otp(cls, user, otp_code):
        """Verify OTP code."""
        # Check if OTP exists
        if not user.otp_code_hash or not user.otp_created_at:
```

```
            return False, "No OTP generated"

        # Check expiry (5 minutes)
        if timezone.now() > user.otp_created_at + timedelta(minutes=cls.OTP_EXPIRY_MINUTES):
            return False, "OTP expired"

        # Check attempts
        if user.otp_attempts >= cls.MAX_ATTEMPTS:
            return False, "Too many attempts"

        # Verify code
        hashed = hashlib.sha256(otp_code.encode()).hexdigest()
        if hashed == user.otp_code_hash:
            # Success - clear OTP
            user.otp_code_hash = None
            user.otp_created_at = None
            user.otp_attempts = 0
            user.is_email_verified = True
            user.save(update_fields=['otp_code_hash', 'otp_created_at', 'otp_attempts', 'is_email_verified'])
            return True, "OTP verified"

        # Failed attempt
        user.otp_attempts += 1
        user.save(update_fields=['otp_attempts'])
        remaining = cls.MAX_ATTEMPTS - user.otp_attempts
        return False, f"Invalid OTP. {remaining} attempts remaining"
```

Listing 7.2: OTP Service – accounts/services/otp-service.py

7.4 Login Flow

7.4.1 Complete Login Sequence

1. Admin navigates to the internal login URL: `/login/`
2. Admin enters email and password
3. System validates credentials
4. System sends an 8-digit OTP to the admin's email
5. Admin enters OTP
6. System verifies OTP and completes authentication
7. Admin is redirected to the Platform Admin dashboard
8. **No public signup or self-registration path exists anywhere in the system**

7.5 Session Management

7.5.1 Session Configuration

```
# Session configuration
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
SESSION_COOKIE_AGE = 86400  # 24 hours (default session)
SESSION_COOKIE_SECURE = True  # HTTPS only
SESSION_COOKIE_HTTPONLY = True  # Not accessible via JavaScript
SESSION_COOKIE_SAMESITE = 'Lax'
SESSION_EXPIRE_AT_BROWSER_CLOSE = False
SESSION_SAVE_EVERY_REQUEST = True
```

Listing 7.3: Session Settings – settings.py

7.5.2 Remember Me (Trusted Device)

When a user checks "Remember this device" during login, a trusted device fingerprint is stored:

```
def generate_device_fingerprint(request):
    """Generate a unique device fingerprint."""
    import hashlib
    components = [
        request.META.get('HTTP_USER_AGENT', ''),
        request.META.get('HTTP_ACCEPT_LANGUAGE', ''),
        request.META.get('HTTP_ACCEPT_ENCODING', ''),
        request.META.get('REMOTE_ADDR', '').split(',')[0],
        settings.SECRET_KEY[:10],
    ]
    return hashlib.sha256(''.join(components).encode()).hexdigest()

def is_trusted_device(request, user):
    """Check if device is trusted for user."""
    fingerprint = generate_device_fingerprint(request)
    devices = user.trusted_devices.get('devices', [])
    for device in devices:
        if device['fingerprint'] == fingerprint:
            if timezone.now() < datetime.fromisoformat(device['expires_at']):
                return True
    return False

def add_trusted_device(request, user):
    """Add current device as trusted for 30 days."""
    fingerprint = generate_device_fingerprint(request)
    devices = user.trusted_devices.get('devices', [])
    # Limit to 10 devices
    if len(devices) >= 10:
        devices = devices[-9:]  # Keep newest 9
    devices.append({
        'fingerprint': fingerprint,
        'expires_at': (timezone.now() + timedelta(days=30)).isoformat(),
        'created_at': timezone.now().isoformat(),
        'device_name': request.META.get('HTTP_USER_AGENT', 'Unknown')[:50],
        'ip_address': request.META.get('REMOTE_ADDR', ''),
    })
    user.trusted_devices['devices'] = devices
    user.save(update_fields=['trusted_devices'])
```

Listing 7.4: Trusted Device Implementation

7.6 Login Attempt Limiting

To prevent brute-force attacks, failed login attempts are tracked:

```
class AuthService:
    """Authentication business logic."""
    MAX_FAILED_ATTEMPTS = 5
    LOCKOUT_DURATION_MINUTES = 15

    @classmethod
    def handle_failed_login(cls, user):
        """Increment failed attempts and lock account if threshold reached."""
        user.failed_login_attempts += 1
        if user.failed_login_attempts >= cls.MAX_FAILED_ATTEMPTS:
            user.locked_until = timezone.now() + timedelta(
                minutes=cls.LOCKOUT_DURATION_MINUTES
            )
        user.save(update_fields=['failed_login_attempts', 'locked_until'])

    @classmethod
    def handle_successful_login(cls, user):
        """Reset failed attempts on successful login."""
        user.failed_login_attempts = 0
        user.locked_until = None
        user.last_login = timezone.now()
        user.save(update_fields=['failed_login_attempts', 'locked_until', 'last_login'])

    @classmethod
    def is_account_locked(cls, user):
        """Check if account is currently locked."""
        if user.locked_until and user.locked_until > timezone.now():
            remaining = (user.locked_until - timezone.now()).seconds // 60
            return True, f"Account locked for {remaining} minutes"
        if user.locked_until and user.locked_until <= timezone.now():
            user.failed_login_attempts = 0
            user.locked_until = None
            user.save(update_fields=['failed_login_attempts', 'locked_until'])
        return False, None
```

Listing 7.5: Login Rate Limiting – accounts/services/auth-service.py

7.7 Authentication Security Measures

Table 7.1: Authentication Security Measures

Measure	Implementation
Password Hashing	bcrypt with cost factor 12
HTTPS Enforcement	All authentication pages served exclusively over HTTPS
CSRF Protection	Django's built-in CSRF middleware on all forms
Session Security	HTTP-only cookies, SameSite=Lax, secure flag enabled
Login Rate Limiting	5 failed attempts = 15-minute account lockout
OTP Security	8-digit code, 5-minute expiry, 3 attempts max
Password Strength	Minimum 8 characters, mix of letters and numbers
Password Reset Token	Single-use, expires after 24 hours
Trusted Device	SHA-256 fingerprint, 30-day expiry, 10 device limit

Chapter Summary

Table 7.2: Authentication System – Key Specifications

Property	Value
Authentication Method	Email + Password + 8-digit OTP
OTP Delivery	Email (Resend)
OTP Expiry	5 minutes
OTP Attempts	3 attempts max
OTP Resend	60-second cooldown, 5/hour limit
Trusted Device	30-day bypass with SHA-256 fingerprint
Max Trusted Devices	10 per user
Session Duration	24 hours (default), 30 days (remember me)
Account Lockout	5 failed attempts = 15-minute lockout
Password Reset	Email with single-use token, 24-hour expiry

Chapter 8

User Management & Roles

8.1 Overview

Caldan Construction implements a hierarchical role-based access control (RBAC) system that governs what each user type can see and do within the platform.

KEY INSIGHT Design Principle: The Platform Admin is the sovereign administrator of their construction operations. They create all user accounts, configure permissions, and have full visibility across all modules.

8.2 Role Hierarchy

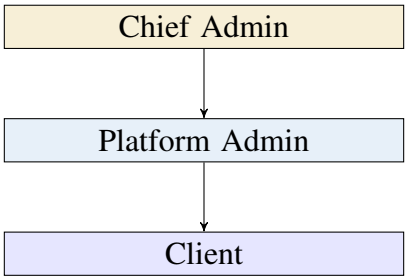


Figure 8.1: Role Hierarchy

8.3 Role Definitions

8.3.1 Chief Admin

The Chief Admin is the first account created during setup:

- **Full Access:** Everything across all modules
- **Cannot Be Deleted:** Protected from deletion (except manually from database)
- **User Creation:** Can create other admin accounts
- **Configuration:** System settings, appearance, notifications
- **Permission Management:** Can set permissions for other admins
- **Data Management:** Backup, export, deletion

8.3.2 Platform Admin

Platform Admins are created by the Chief Admin via invitation:

- **Full Access:** Everything across all modules (unless restricted by Chief)
- **User Creation:** Can invite other admins (if permitted)
- **Data Management:** Backup, export, deletion
- **Cannot Delete:** Cannot delete their own account
- **Audit:** All actions logged

8.3.3 Client

Clients have limited access:

- **Shop Access:** Browse products, place orders, track orders
- **No Dashboard Access:** Cannot access admin dashboard
- **Order History:** View past orders
- **No Data Access:** Cannot see admin data

8.4 Internal User Creation Flow

1. Client deploys project with empty database
2. Client visits `/setup/` (only available when no users exist)
3. Client creates Chief Admin account (email, password)
4. Account created with:
   - `is_superuser = True`
   - `is_staff = True`
   - `is_chief = True`
   - `admin_id = "ADMIN-001"`
5. `/setup/` URL is PERMANENTLY DISABLED
6. Chief Admin can invite other admins from dashboard
7. New admins get `ADMIN-002`, `ADMIN-003`, etc.



Figure 8.2: Chief Admin Creation Flow

8.5 Invitation Model

```
from django.db import models
from django.utils import timezone
import secrets

class AdminInvitation(models.Model):
    """Secure invitation for admin account creation."""

    email = models.EmailField(unique=True)
    token = models.CharField(max_length=64, unique=True)
    created_by = models.ForeignKey('accounts.User', on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    expires_at = models.DateTimeField()
    accepted_at = models.DateTimeField(null=True, blank=True)
    is_accepted = models.BooleanField(default=False)

    def save(self, *args, **kwargs):
        if not self.token:
            self.token = secrets.token_urlsafe(32)
            self.expires_at = timezone.now() + timedelta(hours=48)
        super().save(*args, **kwargs)

    def is_valid(self):
        """Check if invitation is still valid."""
        return not self.is_accepted and timezone.now() < self.expires_at

    def accept(self, user):
        """Mark invitation as accepted."""
        self.is_accepted = True
        self.accepted_at = timezone.now()
        self.save()
```

Listing 8.1: Invitation Model – accounts/models/invitation.py

8.6 Permission Inventory

8.6.1 Chief Admin Permissions

Table 8.1: Chief Admin Permissions

Module	Permission	Access
Dashboard	View dashboard	Yes
Projects	Create, read, update, delete	Yes
Inventory	Create, read, update, delete	Yes
Shop	Create, read, update, delete	Yes
Finances	Create, read, update, delete	Yes
Clients	Create, read, update, delete	Yes
Team	Create, read, update, delete	Yes
Services	Create, read, update, delete	Yes
Reports	View all reports, export data	Yes
Settings	Manage all settings	Yes
Admins	Create, manage, set permissions	Yes

8.6.2 Platform Admin Permissions

Table 8.2: Platform Admin Permissions (Configurable)

Module	Permission	Default Access
Dashboard	View dashboard	Yes
Projects	Create, read, update, delete	Yes
Inventory	Create, read, update, delete	Yes
Shop	Create, read, update, delete	Yes
Finances	Create, read, update, delete	Yes
Clients	Create, read, update, delete	Yes
Team	Create, read, update, delete	Yes
Services	Create, read, update, delete	Yes
Reports	View all reports, export data	Yes
Settings	Manage settings	Yes (limited)
Admins	View, invite new admins	Configurable

8.7 Permission Enforcement

```
from functools import wraps
from django.shortcuts import redirect
from django.core.exceptions import PermissionDenied

def admin_required(view_func):
    """Decorator: View accessible only to admins."""
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('accounts:login')
        if not request.user.is_admin():
            raise PermissionDenied("Access denied. Admin only.")
        return view_func(request, *args, **kwargs)
    return wrapper

def chief_admin_required(view_func):
    """Decorator: View accessible only to Chief Admin."""
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('accounts:login')
        if not request.user.is_chief:
            raise PermissionDenied("Access denied. Chief Admin only.")
        return view_func(request, *args, **kwargs)
    return wrapper

def setup_required(view_func):
    """Decorator: Setup view accessible only when no users exist."""
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        from accounts.models.user import User
        if User.objects.exists():
```

```
            raise PermissionDenied("Setup already completed.")
        return view_func(request, *args, **kwargs)
    return wrapper
```

Listing 8.2: Permission Decorators – accounts/permissions/decorators.py

8.8 Audit Trail for User Actions

Every user action is logged with full context:

```
from accounts.models.audit import AuditLog

class AuditService:
    """Centralized audit logging service."""

    @staticmethod
    def log_action(user, action, entity_type='', entity_id='', details=None, request=None):
        """Log an action to the immutable audit trail."""
        ip_address = None
        user_agent = ""
        if request:
            ip_address = AuditService._get_client_ip(request)
            user_agent = request.META.get('HTTP_USER_AGENT', '')[:500]

        log_entry = AuditLog.objects.create(
            user=user,
            user_role=user.role if user else 'SYSTEM',
            action=action,
            entity_type=entity_type,
            entity_id=str(entity_id) if entity_id else '',
            details=details or {},
            ip_address=ip_address,
            user_agent=user_agent,
        )
        return log_entry

    @staticmethod
    def _get_client_ip(request):
        """Extract client IP address from request."""
        x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
        if x_forwarded_for:
            return x_forwarded_for.split(',')[0].strip()
        return request.META.get('REMOTE_ADDR')
```

Listing 8.3: Audit Logging – accounts/services/audit-service.py

Chapter Summary

Table 8.3: User Management – Key Specifications

Property	Value
Roles	3: Chief Admin, Platform Admin, Client
User Creation	Chief Admin via /setup/ (one-time), Platform Admin via invitation
Chief Admin	Cannot be deleted, full permissions
Platform Admin	Configurable permissions, can self-delete
Permission Enforcement	Python decorators at view level
Data Scope	Full access for all admins
Audit Trail	All actions logged with user identity
Max Users	Unlimited
Account Deactivation	Soft delete with reactivation option

Chapter 9

OTP & Security System

9.1 Overview

The Caldan Construction OTP (One-Time Password) system provides an additional layer of security beyond username and password. Every login requires verification of an 8-digit OTP sent to the user's email address.

KEY INSIGHT Core Principle: Security is not an afterthought—it is embedded in every layer of the architecture. The OTP system provides defense-in-depth protection against credential theft and unauthorized access.

9.2 OTP Generation

9.2.1 8-Digit Code Generation

OTP codes are generated using cryptographically secure random number generation:

```
import secrets
import hashlib
from django.utils import timezone
from datetime import timedelta

class OTPService:
    """8-digit OTP generation and verification."""

    OTP_LENGTH = 8
    OTP_EXPIRY_MINUTES = 5
    MAX_ATTEMPTS = 3
    MAX_RESENDS_PER_HOUR = 5
    RESEND_COOLDOWN_SECONDS = 60

    @classmethod
    def generate_otp(cls, user):
        """Generate cryptographically secure 8-digit OTP."""
        # Generate random 8-digit code
        otp_code = ''.join(str(secrets.randbelow(10)) for _ in range(cls.OTP_LENGTH))

        # Hash for secure storage
        user.otp_code_hash = hashlib.sha256(otp_code.encode()).hexdigest()
        user.otp_created_at = timezone.now()
        user.otp_attempts = 0
        user.save(update_fields=['otp_code_hash', 'otp_created_at', 'otp_attempts'])
        return otp_code
```

Listing 9.1: OTP Generation – accounts/services/otp-service.py

9.3 OTP Delivery

9.3.1 Email Template

```
{% extends "email/base_email.html" %}

{% block content %}
<div style="text-align: center;">
    <h2 style="color: #0D2B36;">Your Verification Code</h2>
    <p style="color: #6B7280;">Enter the 8-digit code below to verify your identity.</p>

    <div style="background: #F6F1E8; padding: 20px; border-radius: 12px; margin: 24px 0; text-align: center;">
        <code style="font-size: 36px; font-weight: 700; color: #0D2B36; font-family: monospace; letter-spacing: 8px
    ↪ ;">
            {{ otp_code }}
        </code>
    </div>

    <div style="background: #FFF8E7; border-left: 4px solid #D69723; padding: 16px; margin: 20px 0; border-radius: 8
    ↪ px;">
        <p style="margin: 0; color: #2C2C2C;">
            <strong>Warning: This code expires in 5 minutes.</strong>
        </p>
        <p style="margin: 8px 0 0; font-size: 12px; color: #6B7280;">
            Never share this code with anyone. Caldan Construction will never ask for your OTP outside of the login
    ↪ page.
        </p>
    </div>

    <p style="color: #6B7280; font-size: 12px;">
        If you didn't request this code, please ignore this email.
    </p>

    <div style="margin-top: 24px; padding: 16px; background: #F6F1E8; border-radius: 8px;">
        <p style="margin: 0; font-size: 12px; color: #6B7280;">
            Device: {{ device_info }}
        </p>
        <p style="margin: 4px 0 0; font-size: 12px; color: #6B7280;">
            IP Address: {{ ip_address }}
        </p>
    </div>
</div>
{% endblock %}
```

Listing 9.2: OTP Email Template – templates/email/otp-email.html

9.4 OTP Verification Flow

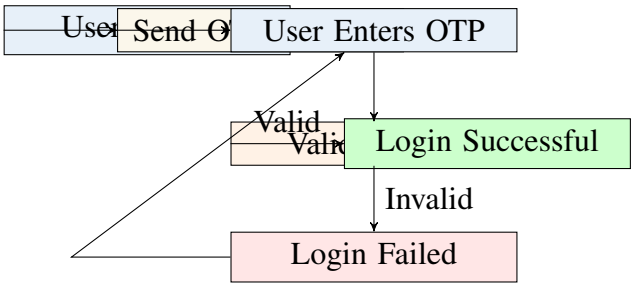


Figure 9.1: OTP Verification Flow

9.5 Security Monitoring

9.5.1 Security Events Logged

Table 9.1: Security Events Logged

Event	Log Level	Details Logged
Successful login	INFO	User, IP, device, timestamp
Failed login	WARNING	User, IP, attempt count
OTP sent	INFO	User, email, IP
OTP verified	INFO	User, IP, device
OTP failed	WARNING	User, IP, remaining attempts
OTP resend	INFO	User, IP, timestamp
Account lockout	WARNING	User, IP, duration
Trusted device added	INFO	User, device info, expiry
Password change	INFO	User, IP
Password reset	INFO	User, IP

Chapter Summary

Table 9.2: OTP & Security – Key Specifications

Property	Value
OTP Length	8 digits
OTP Expiry	5 minutes
OTP Attempts	3 attempts max
OTP Resend Cooldown	60 seconds
OTP Resends per Hour	5 max
OTP Delivery	Email (Resend)
OTP Storage	SHA-256 hash (plain text never stored)
Trusted Device	30-day bypass with SHA-256 fingerprint
Max Trusted Devices	10 per user
Account Lockout	5 failed attempts = 15 minutes
Password Strength	Min 8 chars, letters and numbers
Audit Logging	All security events logged

Chapter 10

Admin Account Creation & Invitations

10.1 Overview

The admin account creation system in Caldan Construction is designed to be secure, private, and invitation-only. There is no public signup page. Every admin account is created through a controlled process.

> **KEY INSIGHT** Core Principle: Security begins with access control. No one can create an admin account without a secure invitation from an existing admin. The first admin is created through a one-time setup URL.

10.2 First Admin: The Setup URL

When the project is deployed with an empty database, the `/setup/` URL is active. This is the ONLY way to create the first admin account.

```
from django.shortcuts import render, redirect
from django.contrib import messages
from django.views import View
from accounts.models.user import User
from accounts.services.audit_service import AuditService

class SetupView(View):
    """First-time setup view - only accessible when no users exist."""

    def dispatch(self, request, *args, **kwargs):
        # Check if setup has already been completed
        if User.objects.exists():
            return redirect('accounts:login')
        return super().dispatch(request, *args, **kwargs)

    def get(self, request):
        return render(request, 'accounts/setup/setup.html')

    def post(self, request):
        email = request.POST.get('email')
        full_name = request.POST.get('full_name')
        phone = request.POST.get('phone')
        password = request.POST.get('password')
        confirm = request.POST.get('confirm_password')

        # Validate
        if password != confirm:
            messages.error(request, "Passwords do not match.")
            return render(request, 'accounts/setup/setup.html')

        if User.objects.filter(email=email).exists():
            messages.error(request, "Email already exists.")
            return render(request, 'accounts/setup/setup.html')

        # Create Chief Admin
        user = User.objects.create_superuser(
            email=email,
            password=password,
            full_name=full_name,
            phone=phone,
            is_chief=True,
            is_active=True,
            is_staff=True,
            is_superuser=True
```

```
        )

        # Log audit
        AuditService.log_action(
            user=user,
            action='CHIEF_ADMIN_CREATED',
            details={
                'email': user.email,
                'setup_completed': True
            }
        )

        messages.success(request, "Chief Admin account created successfully! Please login.")
        return redirect('accounts:login')
```

Listing 10.1: Setup View – accounts/views/-setup-views.py

10.2.1 Setup Flow

1. Fresh install with empty database
2. Client visits `/setup/`
3. Client fills form: email, full name, phone, password
4. System creates Chief Admin account
5. `/setup/` URL is permanently disabled
6. Client logs in with credentials
7. Client is prompted to change password (first login)

10.3 Inviting Other Admins

Only the Chief Admin can invite new Platform Admins. The invitation process:

```
from django.core.mail import send_mail
from django.utils import timezone
from accounts.models.invitation import AdminInvitation
from accounts.models.user import User
from accounts.services.audit_service import AuditService

class InvitationService:
    """Handles admin invitation creation and acceptance."""

    @classmethod
    def create_invitation(cls, email, created_by):
        """Create a secure invitation link."""
        # Check if user already exists
        if User.objects.filter(email=email).exists():
            raise ValueError("User with this email already exists")

        # Check for existing valid invitation
        existing = AdminInvitation.objects.filter(
            email=email,
            is_accepted=False,
            expires_at__gt=timezone.now()
        ).first()

        if existing:
            raise ValueError("An invitation has already been sent to this email")

        # Create new invitation
        invitation = AdminInvitation.objects.create(
            email=email,
            created_by=created_by
        )

        # Send email with invitation link
        invitation_link = f"https://caldanconstruction.com/accept-invite/{invitation.token}"
        send_mail(
            subject="Invitation to join Caldan Construction Admin",
            message=f"Click this link to create your admin account: {invitation_link}",
            from_email="admin@caldanconstruction.com",
```

```
            recipient_list=[email]
        )

        AuditService.log_action(
            user=created_by,
            action='ADMIN_INVITATION_SENT',
            details={'email': email}
        )

        return invitation

    @classmethod
    def accept_invitation(cls, token, password):
        """Accept invitation and create admin account."""
        invitation = AdminInvitation.objects.get(token=token)

        if not invitation.is_valid():
            raise ValueError("Invitation has expired or already been used")

        # Create user
        user = User.objects.create_user(
            email=invitation.email,
            password=password,
            is_active=True,
            is_staff=True,
            is_superuser=False,
            is_chief=False
        )

        # Mark invitation as accepted
        invitation.accept(user)

        # Force password change on first login
        user.password_change_required = True
        user.save()

        AuditService.log_action(
            user=user,
            action='ADMIN_ACCOUNT_CREATED',
            details={'email': user.email}
        )

        return user
```

Listing 10.2: Invitation Service – accounts/services/invitation-service.py

10.4 Force Password Change Middleware

```
from django.shortcuts import redirect
from django.urls import reverse

class PasswordChangeRequiredMiddleware:
    """Force password change on first login."""

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        if request.user.is_authenticated:
            if getattr(request.user, 'password_change_required', False):
                # Don't intercept password change page itself
                if request.path != reverse('accounts:change_password'):
                    return redirect('accounts:change_password')

        return self.get_response(request)
```

Listing 10.3: Force Password Change Middleware – accounts/middleware/force-password-change-middleware.py

10.5 Admin Management Dashboard

```
@login_required
@chief_admin_required
def manage_admins(request):
    """List all admins with actions (Chief Admin only)."""
    admins = User.objects.filter(is_staff=True)

    if request.method == 'POST':
```

```
        admin_id = request.POST.get('admin_id')
        admin_to_delete = User.objects.get(id=admin_id)

        # Prevent self-deletion
        if admin_to_delete == request.user:
            messages.error(request, "You cannot delete your own account.")
            return redirect('settings:admins')

        # Prevent Chief Admin deletion
        if admin_to_delete.is_chief:
            messages.error(request, "Cannot delete Chief Admin account.")
            return redirect('settings:admins')

        admin_to_delete.delete()
        messages.success(request, f"Admin {admin_to_delete.email} deleted.")

        AuditService.log_action(
            user=request.user,
            action='ADMIN_DELETED',
            details={'deleted_email': admin_to_delete.email}
        )

        return redirect('settings:admins')

    return render(request, 'dashboard/settings/admins.html', {
        'admins': admins,
        'can_create_invitation': True
    })
```

Listing 10.4: Admin Management View – accounts/views/-admin-views.py

10.6 Admin Permission Management

```
@login_required
@chief_admin_required
def edit_admin_permissions(request, admin_id):
    """Edit admin permissions (Chief Admin only)."""
    admin = get_object_or_404(User, id=admin_id, is_staff=True)

    # Prevent editing Chief Admin permissions
    if admin.is_chief:
        messages.error(request, "Cannot modify Chief Admin permissions.")
        return redirect('settings:admins')

    if request.method == 'POST':
        permissions = {
            'can_invite_admins': request.POST.get('can_invite_admins') == 'on',
            'can_manage_settings': request.POST.get('can_manage_settings') == 'on',
            'can_manage_projects': request.POST.get('can_manage_projects') == 'on',
            'can_manage_inventory': request.POST.get('can_manage_inventory') == 'on',
            'can_manage_shop': request.POST.get('can_manage_shop') == 'on',
            'can_manage_finances': request.POST.get('can_manage_finances') == 'on',
            'can_manage_clients': request.POST.get('can_manage_clients') == 'on',
            'can_manage_team': request.POST.get('can_manage_team') == 'on',
            'can_manage_reports': request.POST.get('can_manage_reports') == 'on',
        }

        admin.profile.permissions = permissions
        admin.profile.save()

        AuditService.log_action(
            user=request.user,
            action='ADMIN_PERMISSIONS_UPDATED',
            details={'admin': admin.email, 'permissions': permissions}
        )

        messages.success(request, f"Permissions updated for {admin.email}.")
        return redirect('settings:admins')

    return render(request, 'dashboard/settings/admin_permissions.html', {
        'admin': admin,
        'permissions': admin.profile.permissions
    })
```

Listing 10.5: Admin Permissions View – accounts/views/-admin-views.py

10.7 Admin ID System

Each admin receives a unique identifier:

Table 10.1: Admin ID Format

Segment	Format	Example
Prefix	ADMIN	ADMIN
Sequential Number	3 digits, zero-padded	001, 002, 003
Full ID	ADMIN-001	Chief Admin
Full ID	ADMIN-002	First invited admin

10.8 Security Summary

Table 10.2: Admin Creation Security Features

Feature	Implementation
No public signup	All accounts created via secure invitation or setup URL
Invitation token	64-character URL-safe token, one-time use
Token expiry	48-hour expiration
Force password change	Middleware forces change on first login
Password strength	Minimum 8 characters, mix of letters and numbers
OTP verification	8-digit OTP on every login
Account lockout	5 failed attempts = 15-minute lockout
Session security	HTTP-only cookies, SameSite=Lax, secure flag
Audit trail	All account creations/deletions logged
Self-deletion prevention	Cannot delete your own account
Chief Admin protection	Chief Admin cannot be deleted except manually from database

Chapter Summary

Table 10.3: Admin Account Creation – Key Specifications

Property	**Value**
First Admin Creation	/setup/ URL (one-time use, only when no users exist)
Invitation Creation	Chief Admin only
Invitation Expiry	48 hours
Invitation Token	64-character URL-safe (one-time use)
Admin ID Format	ADMIN-001, ADMIN-002, etc.
Chief Admin	Cannot be deleted (except manually from database)
Password Change	Forced on first login
Permissions	Configurable per admin (Chief Admin only)
Audit Trail	All invitation/creation/deletion events logged

Part IV

Core Business Modules



Figure 10.1: Operations Engine — projects, inventory, finance, clients, and shop

Chapter 11

Public Website Architecture

11.1 Overview

The Caldan Construction public website (caldanconstruction.com) serves as the marketing storefront, project showcase, and e-commerce portal. It serves three distinct audiences — potential clients evaluating the company, visitors browsing projects, and customers ordering materials — all without requiring authentication.

KEY INSIGHT Design Philosophy: The public website is designed to convert visitors into clients. Every page, every section, and every call-to-action is crafted to either (a) convince a visitor to request a quote, (b) help a customer find and order materials, or (c) build trust in the Caldan brand.

11.2 Page Inventory

Table 11.1: Complete Public Website Page Inventory

Page	URL Pattern	Purpose
Landing Page	`/`	Hero, services overview, projects showcase, testimonials, CTA
About	`/about/`	Company story, mission, values, team introduction
Services	`/services/`	Full list of all 9 services with descriptions and icons
Service Detail	`/services/<slug>/`	Detailed service page with related projects
Projects	`/projects/`	Project gallery with filtering and search
Project Detail	`/projects/<id>/`	Single project with photos, details, and services used
Shop	`/shop/`	Materials catalog with categories and search
Product Detail	`/shop/product/<slug>/`	Product details, price, add to cart
Cart	`/shop/cart/`	Shopping cart with quantity updates
Checkout	`/shop/checkout/`	Address, payment, place order
Order Confirmation	`/shop/order-confirmation/<id>/`	Order placed successfully
Order Tracking	`/shop/track/<order_number>/`	Track order status
Contact	`/contact/`	Contact form, address, phone
Quote Request	`/quote/`	Detailed quote request form
Thank You	`/thank-you/`	Contact/quote success page
Legal: Privacy	`/legal/privacy/`	Privacy policy (DPA compliant)
Legal: Terms	`/legal/terms/`	Terms of service
Legal: Refunds	`/legal/refunds/`	Refund policy
Legal: Cookies	`/legal/cookies/`	Cookie policy

11.3 Landing Page Architecture

The landing page is the most important page on the entire website. It must capture attention within 3 seconds and communicate the company's value proposition instantly.

11.3.1 Section Architecture

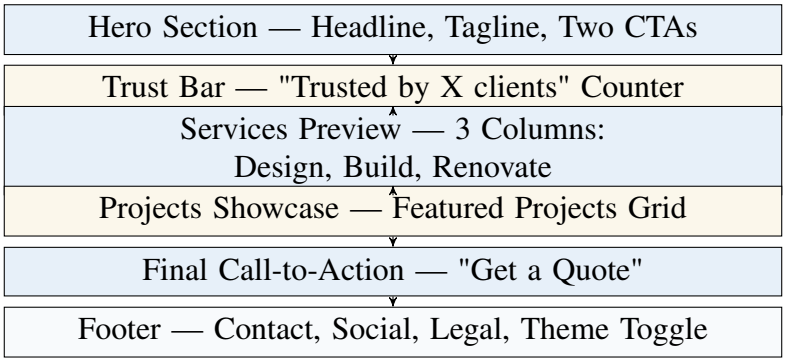


Figure 11.1: Landing Page Section Flow

11.3.2 Hero Section Specifications

Table 11.2: Hero Section Specifications

Element	Content / Behavior
Background	Dark teal gradient (#0D2B36 to #0A1F2A) with subtle geometric pattern
Headline	"Building Kenya's Future, One Project at a Time." — large, bold, white
Sub-headline	"Construction, renovations, interior design, and industrial solutions — all from one trusted partner."
Primary CTA	"Get a Quote" button — gold, large, links to /quote/
Secondary CTA	"View Our Projects" button — outlined white, links to /projects/
Visual Element	Subtle illustration or abstract geometric shapes suggesting construction

11.3.3 Trust Counter

A live counter displaying the number of satisfied clients:

```
from django.core.cache import cache
from clients.models.client import Client

def trust_counter(request):
    """
    Inject the dynamic trust counter into all website templates.
    Counter is cached for 60 seconds to minimize database queries.
    """
    count = cache.get('trust_counter')
    if count is None:
        count = Client.objects.filter(is_active=True).count()
```

```
        cache.set('trust_counter', count, 60)  # Cache for 60 seconds
    return {
        'trust_counter': count,
    }
```

Listing 11.1: Trust Counter Context Processor – website/context-processors.py

11.3.4 Landing Page Template

```
{% extends "website/base.html" %}
{% load static %}

{% block title %}Caldan Construction --- Building Kenya's Future{% endblock %}

{% block content %}

<!-- Hero Section -->
<section class="hero" style="background: linear-gradient(135deg, #0D2B36 0%, #0A1F2A 100%);">
    <div class="container">
        <div class="hero-content">
            <h1 class="hero-headline">
                Building Kenya's Future,<br>
                <span class="text-gold">One Project at a Time.</span>
            </h1>
            <p class="hero-subtitle">
                Construction, renovations, interior design, and industrial solutions ---
                all from one trusted partner.
            </p>
            <div class="hero-cta-group">
                <a href="{% url 'quote_request' %}" class="btn btn-gold btn-lg">
                    <i class="fas fa-paper-plane"></i> Get a Quote
                </a>
                <a href="{% url 'projects' %}" class="btn btn-outline-light btn-lg">
                    <i class="fas fa-building"></i> View Our Projects
                </a>
            </div>
        </div>
    </div>
</section>

<!-- Trust Counter -->
<section class="trust-bar">
    <div class="container">
        <p class="trust-text">
            <i class="fas fa-check-circle" style="color: #D69723;"></i>
            Trusted by <strong>{{ trust_counter }}</strong> clients across Kenya
        </p>
    </div>
</section>

<!-- Services Preview -->
<section class="services-preview">
    <div class="container">
        <h2 class="section-title">What We Offer</h2>
        <div class="three-col-grid">
            <div class="service-card">
                <i class="fas fa-drafting-compass service-icon"></i>
                <h3>Design & Construction</h3>
                <p>Full design and construction services from concept to completion.</p>
            </div>
            <div class="service-card">
                <i class="fas fa-hard-hat service-icon"></i>
                <h3>Renovations</h3>
                <p>Home, office, and commercial renovations with attention to detail.</p>
            </div>
            <div class="service-card">
                <i class="fas fa-paint-roller service-icon"></i>
                <h3>Interior Design</h3>
                <p>Beautiful interior fit-out and design for any space.</p>
            </div>
        </div>
        <div class="text-center mt-4">
            <a href="{% url 'services' %}" class="btn btn-outline-primary">
                View All Services <i class="fas fa-arrow-right"></i>
            </a>
        </div>
    </div>
</section>

<!-- Projects Showcase -->
<section class="projects-showcase" style="background: #F6F1E8;">
    <div class="container">
        <h2 class="section-title">Featured Projects</h2>
        <div class="project-grid">
            {% for project in featured_projects %}
            <div class="project-card">
                <img src="{{ project.cover_photo.url }}" alt="{{ project.name }}">
                <div class="project-card-body">
```

```
                    <h3>{{ project.name }}</h3>
                    <p>{{ project.town }}, {{ project.county }}</p>
                    <a href="{% url 'project_detail' project.id %}" class="btn btn-sm btn-primary">
                        View Project
                    </a>
                </div>
            </div>
            {% endfor %}
        </div>
        <div class="text-center mt-4">
            <a href="{% url 'projects' %}" class="btn btn-primary">
                View All Projects <i class="fas fa-arrow-right"></i>
            </a>
        </div>
    </div>
</section>

<!-- Final CTA -->
<section class="cta-banner" style="background: #0D2B36;">
    <div class="container">
        <h2>Ready to Build Your Project?</h2>
        <p>Contact us today for a free consultation and quote.</p>
        <a href="{% url 'quote_request' %}" class="btn btn-gold btn-lg">
            <i class="fas fa-paper-plane"></i> Get a Free Quote
        </a>
    </div>
</section>

{% endblock %}
```

Listing 11.2: Landing Page Template – website/templates/website/index.html

11.4 Services Page

The services page displays all 9 services Caldan offers:

Table 11.3: Service Categories

Service Name	Category	Description
Design & Construction	DESIGN	Full design and construction services
Renovations	RENOVATION	Home, office, commercial renovations
Interior Design	INTERIOR	Interior fit-out and design
Welding & Fabrication	WELDING	Metalwork, gates, railings, structural steel
Concrete Products	CONCRETE	Pre-cast concrete, blocks, slabs
Industrial Floor Solutions	INDUSTRIAL	Epoxy, polished concrete, industrial flooring
Waterproofing Works	WATERPROOFING	Roof, basement, tank waterproofing
Roads & Stormwater Infrastructure	INFRASTRUCTURE	Roads, drainage, culverts
Construction Chemicals	CHEMICALS	Admixtures, sealants, grouts

```
from django.db import models
from django.utils.text import slugify
```

```
class Service(models.Model):
    """Construction services offered by Caldan."""

    class Category(models.TextChoices):
        DESIGN = 'DESIGN', 'Design & Construction'
        RENOVATION = 'RENOVATION', 'Renovations'
        INTERIOR = 'INTERIOR', 'Interior Design'
        WELDING = 'WELDING', 'Welding & Fabrication'
        CONCRETE = 'CONCRETE', 'Concrete Products'
        INDUSTRIAL = 'INDUSTRIAL', 'Industrial Floor Solutions'
        WATERPROOFING = 'WATERPROOFING', 'Waterproofing Works'
        INFRASTRUCTURE = 'INFRASTRUCTURE', 'Roads & Stormwater Infrastructure'
        CHEMICALS = 'CHEMICALS', 'Construction Chemicals'

    name = models.CharField(max_length=200)
    category = models.CharField(max_length=20, choices=Category.choices, db_index=True)
    slug = models.SlugField(unique=True)
    description = models.TextField()
    icon = models.CharField(max_length=50, help_text="Font Awesome icon class")
    image = models.ImageField(upload_to='services/', blank=True, null=True)
    is_active = models.BooleanField(default=True)
    display_order = models.PositiveSmallIntegerField(default=0)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'services'
        ordering = ['display_order', 'name']

    def __str__(self):
        return self.name

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)
```

Listing 11.3: Service Model – services/models/service.py

11.5 Contact Page

The contact page provides multiple ways to reach Caldan:

Table 11.4: Contact Page Elements

Element	Specification
Contact Form	Name, Email, Phone, Subject, Message, Submit button
Email	sir.mukoyakhisa@gmail.com — displayed prominently
Phone	To be provided by client
Physical Address	Nairobi, Kenya
Response Time	"We respond to all inquiries within 24 hours."
Form Submission	Saved to ContactSubmission model; email notification sent to admin

11.6 SEO & Metadata

Every public page includes comprehensive metadata for search engines:

```
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Caldan Construction --- Building Kenya's Future{% endblock %}</title>
    <meta name="description" content="{% block meta_description %}Caldan Construction is a leading construction
     ↪ company in Kenya. We offer design, construction, renovations, interior design, and industrial solutions.{%
     ↪ endblock %}">
    <meta name="keywords" content="construction Kenya, building contractor, renovations Nairobi, interior design,
     ↪ industrial flooring, concrete products">
```

```
    <!-- Open Graph / Social Sharing -->
    <meta property="og:title" content="Caldan Construction --- Building Kenya's Future">
    <meta property="og:description" content="Trusted construction company in Kenya. Design, build, renovate, and more
     ↪ .">
    <meta property="og:image" content="{% static 'images/og-image.png' %}">
    <meta property="og:url" content="https://caldanconstruction.com">
    <meta property="og:type" content="website">

    <!-- Twitter Card -->
    <meta name="twitter:card" content="summary_large_image">
    <meta name="twitter:title" content="Caldan Construction --- Building Kenya's Future">
    <meta name="twitter:description" content="Trusted construction company in Kenya.">
    <meta name="twitter:image" content="{% static 'images/og-image.png' %}">

    <!-- Favicon -->
    <link rel="icon" type="image/png" sizes="32x32" href="{% static 'images/favicon.png' %}">
</head>
```

Listing 11.4: SEO Meta Tags – website/templates/website/base.html

Chapter Summary

Table 11.5: Public Website Architecture — Key Specifications

Property	**Value**
Total Pages	19 pages
Authentication Required	No (public website)
Theme Default	Light mode
Theme Toggle Location	Footer (sun/moon icon)
Theme Storage	localStorage (no database write)
Trust Counter	Live database count, cached 60 seconds
Landing Page Sections	6: Hero, Trust Bar, Services, Projects, CTA, Footer
Services	9 categories
Contact Email	sir.mukoyakhisa@gmail.com
Legal Pages	4: Privacy, Terms, Refunds, Cookies

Chapter 12

Projects Management

12.1 Overview

The Projects Management module is the structural foundation of Caldan Construction. It provides the Platform Admin with a comprehensive system for organizing, tracking, and managing construction projects from inception to completion.

KEY INSIGHT Design Principle: Every construction project has its own story. Caldan Construction provides the tools to track that story—from groundbreaking to completion—with milestones, progress tracking, and complete visibility.

12.2 Project Hierarchy

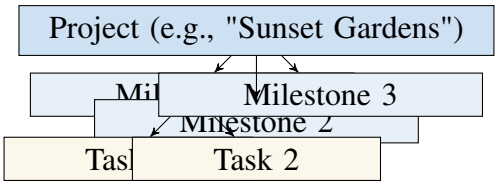


Figure 12.1: Project Hierarchy

12.3 Project Model

```
from django.db import models
from django.utils import timezone
from django.conf import settings

class Project(models.Model):
    """Construction project model."""

    class Status(models.TextChoices):
        PLANNING = 'PLANNING', 'Planning'
        ACTIVE = 'ACTIVE', 'Active'
        ON_HOLD = 'ON_HOLD', 'On Hold'
        COMPLETED = 'COMPLETED', 'Completed'
        ARCHIVED = 'ARCHIVED', 'Archived'

    # Identity
    name = models.CharField(max_length=200, db_index=True)
    description = models.TextField(blank=True, default='')
    project_code = models.CharField(max_length=20, unique=True, db_index=True)

    # Location
    location = models.CharField(max_length=200)
    county = models.CharField(max_length=100, db_index=True)
    town = models.CharField(max_length=100, db_index=True)
    estate = models.CharField(max_length=150, blank=True, default='')
```

```
    # Geocoding (for maps)
    latitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)
    longitude = models.DecimalField(max_digits=9, decimal_places=6, null=True, blank=True)

    # Timeline
    start_date = models.DateField()
    end_date = models.DateField(null=True, blank=True)
    estimated_completion = models.DateField(null=True, blank=True)
    actual_completion = models.DateField(null=True, blank=True)

    # Budget
    budget = models.DecimalField(max_digits=15, decimal_places=2, default=0)
    spent = models.DecimalField(max_digits=15, decimal_places=2, default=0)

    # Progress
    progress = models.DecimalField(max_digits=5, decimal_places=2, default=0)

    # Status
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PLANNING, db_index=True)

    # Services used (many-to-many)
    services = models.ManyToManyField('services.Service', related_name='projects', blank=True)

    # Soft delete
    is_archived = models.BooleanField(default=False, db_index=True)
    archived_at = models.DateTimeField(null=True, blank=True)

    # Cover photo
    cover_photo = models.ImageField(upload_to='projects/covers/', blank=True, null=True)

    # Timestamps
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'projects'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['status', 'created_at']),
            models.Index(fields=['county', 'town']),
        ]

    def __str__(self):
        return f"{self.project_code} --- {self.name}"

    def save(self, *args, **kwargs):
        if not self.project_code:
            self.project_code = self.generate_project_code()
        super().save(*args, **kwargs)

    def generate_project_code(self):
        """Generate unique project code: PRJ-2026-0001."""
        year = timezone.now().year
        count = Project.objects.filter(created_at__year=year).count() + 1
        return f"PRJ-{year}-{count:04d}"

    def update_progress(self):
        """Calculate overall progress from milestones."""
        milestones = self.milestones.all()
        if not milestones:
            return 0
        total_progress = sum(m.completion_percentage for m in milestones)
        self.progress = round(total_progress / milestones.count(), 2)
        self.save(update_fields=['progress'])
        return self.progress

    def archive(self):
        """Soft-delete this project."""
        self.is_archived = True
        self.archived_at = timezone.now()
        self.status = self.Status.ARCHIVED
        self.save()

    @property
    def days_active(self):
        """Calculate days since project started."""
        if not self.start_date:
            return 0
        end = self.actual_completion or timezone.now().date()
        return (end - self.start_date).days
```

Listing 12.1: Project Model – projects/models/project.py

12.4 Project Milestones

```
from django.db import models
from django.utils import timezone

class Milestone(models.Model):
    """Project milestone model."""

    class Status(models.TextChoices):
        PENDING = 'PENDING', 'Pending'
        IN_PROGRESS = 'IN_PROGRESS', 'In Progress'
        COMPLETED = 'COMPLETED', 'Completed'
        OVERDUE = 'OVERDUE', 'Overdue'

    project = models.ForeignKey(
        'projects.Project',
        on_delete=models.CASCADE,
        related_name='milestones'
    )

    title = models.CharField(max_length=200)
    description = models.TextField(blank=True, default='')
    target_date = models.DateField()
    completed_date = models.DateField(null=True, blank=True)

    completion_percentage = models.DecimalField(max_digits=5, decimal_places=2, default=0)
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING)

    # Evidence
    photo = models.ImageField(upload_to='projects/milestones/', blank=True, null=True)
    notes = models.TextField(blank=True, default='')

    # Order
    order = models.PositiveSmallIntegerField(default=0)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'project_milestones'
        ordering = ['order', 'target_date']

    def __str__(self):
        return f"{self.title} ({self.get_status_display()})"

    def save(self, *args, **kwargs):
        # Auto-update status based on completion
        if self.completion_percentage >= 100:
            self.status = self.Status.COMPLETED
            if not self.completed_date:
                self.completed_date = timezone.now().date()
        elif self.target_date and timezone.now().date() > self.target_date:
            if self.status != self.Status.COMPLETED:
                self.status = self.Status.OVERDUE

        super().save(*args, **kwargs)

        # Update project progress
        self.project.update_progress()

    def mark_complete(self):
        """Mark milestone as complete."""
        self.completion_percentage = 100
        self.completed_date = timezone.now().date()
        self.status = self.Status.COMPLETED
        self.save()
```

Listing 12.2: Milestone Model – projects/models/milestone.py

12.5 Progress Tracking

```
from django.db.models import Sum, Count
from django.utils import timezone
from datetime import timedelta

class ProgressService:
    """Service for project progress tracking."""

    @classmethod
    def get_project_progress(cls, project):
        """Get comprehensive progress data for a project."""
        milestones = project.milestones.all()

        if not milestones:
            return {
                'overall_progress': 0,
                'total_milestones': 0,
                'completed_milestones': 0,
```

```
                'pending_milestones': 0,
                'overdue_milestones': 0,
                'in_progress_milestones': 0,
                'days_active': project.days_active,
                'days_remaining': cls.get_days_remaining(project),
                'status': project.status
            }

        total = milestones.count()
        completed = milestones.filter(status='COMPLETED').count()
        pending = milestones.filter(status='PENDING').count()
        overdue = milestones.filter(status='OVERDUE').count()
        in_progress = milestones.filter(status='IN_PROGRESS').count()

        return {
            'overall_progress': project.progress,
            'total_milestones': total,
            'completed_milestones': completed,
            'pending_milestones': pending,
            'overdue_milestones': overdue,
            'in_progress_milestones': in_progress,
            'completion_rate': round((completed / total * 100), 1) if total > 0 else 0,
            'days_active': project.days_active,
            'days_remaining': cls.get_days_remaining(project),
            'status': project.status,
            'estimated_completion': project.estimated_completion,
            'actual_completion': project.actual_completion
        }

    @classmethod
    def get_days_remaining(cls, project):
        """Calculate days remaining until estimated completion."""
        if not project.estimated_completion:
            return None
        if project.status == 'COMPLETED':
            return 0
        remaining = (project.estimated_completion - timezone.now().date()).days
        return max(0, remaining)

    @classmethod
    def get_timeline_data(cls, project):
        """Generate timeline data for project."""
        milestones = project.milestones.all().order_by('target_date')
        timeline = []
        for milestone in milestones:
            timeline.append({
                'id': milestone.id,
                'title': milestone.title,
                'start': milestone.target_date.isoformat(),
                'end': milestone.completed_date.isoformat() if milestone.completed_date else None,
                'progress': float(milestone.completion_percentage),
                'status': milestone.status,
                'order': milestone.order,
            })
        return timeline
```

Listing 12.3: Progress Service – projects/services/progress-service.py

12.6 Project Views

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.core.paginator import Paginator
from accounts.permissions.decorators import admin_required
from projects.models.project import Project
from projects.services.progress_service import ProgressService

@login_required
@admin_required
def project_list(request):
    """List all projects with filters."""
    queryset = Project.objects.filter(is_archived=False).select_related()

    # Filters
    status = request.GET.get('status')
    county = request.GET.get('county')
    search = request.GET.get('search')

    if status:
        queryset = queryset.filter(status=status)
    if county:
        queryset = queryset.filter(county=county)
    if search:
        queryset = queryset.filter(
            models.Q(name__icontains=search) |
```

```
            models.Q(project_code__icontains=search) |
            models.Q(location__icontains=search)
        )

    # Pagination
    paginator = Paginator(queryset, 20)
    page = request.GET.get('page', 1)
    projects = paginator.get_page(page)

    # Stats
    stats = {
        'total': Project.objects.filter(is_archived=False).count(),
        'active': Project.objects.filter(status='ACTIVE').count(),
        'planning': Project.objects.filter(status='PLANNING').count(),
        'on_hold': Project.objects.filter(status='ON_HOLD').count(),
        'completed': Project.objects.filter(status='COMPLETED').count(),
        'total_budget': Project.objects.filter(is_archived=False).aggregate(
            total=models.Sum('budget')
        )['total'] or 0,
    }

    context = {
        'projects': projects,
        'stats': stats,
        'active_status': status,
        'active_county': county,
        'search': search,
    }
    return render(request, 'dashboard/admin/projects/list.html', context)

@login_required
@admin_required
def project_detail(request, project_id):
    """Project detail view."""
    project = get_object_or_404(Project, id=project_id, is_archived=False)
    progress = ProgressService.get_project_progress(project)
    timeline = ProgressService.get_timeline_data(project)

    context = {
        'project': project,
        'progress': progress,
        'timeline': timeline,
    }
    return render(request, 'dashboard/admin/projects/detail.html', context)

@login_required
@admin_required
def project_create(request):
    """Create a new project."""
    if request.method == 'POST':
        form = ProjectForm(request.POST, request.FILES)
        if form.is_valid():
            project = form.save()
            messages.success(request, f"Project {project.name} created successfully.")
            return redirect('projects:detail', project_id=project.id)
    else:
        form = ProjectForm()

    return render(request, 'dashboard/admin/projects/create.html', {'form': form})
```

Listing 12.4: Project Views – projects/views/-admin-views.py

Chapter Summary

Table 12.1: Projects Management — Key Specifications

Property	Value
Hierarchy	Project -> Milestones
Statuses	5: Planning, Active, On Hold, Completed, Archived
Project Code	PRJ-YYYY-XXXX (auto-generated)
Progress Tracking	Milestone-based with automatic calculation
Timeline	Gantt-style view with milestone dates
Budget Tracking	Planned vs spent tracking
Photos	Cover photo + milestone photos (Cloudinary)
Soft Delete	Archive with reactivation option
Services Linking	Many-to-many with services

Chapter 13

Inventory Management

13.1 Overview

The Inventory Management module is the operational heart of Caldan Construction's supply chain. It tracks all materials, manages stock levels, monitors suppliers, and provides real-time alerts when stock runs low.

> **KEY INSIGHT** Core Principle: Every material transaction is tracked, every receipt is uploaded, and every alert is actionable. No material is ever lost or unaccounted for.

13.2 Inventory Control Loop

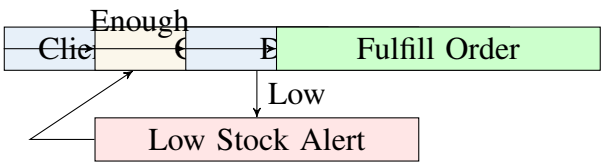


Figure 13.1: Inventory Control Loop

13.3 Material Model

```
from django.db import models
from django.utils import timezone
from django.conf import settings

class Material(models.Model):
    """Construction material model."""

    class Category(models.TextChoices):
        CEMENT = 'CEMENT', 'Cement'
        STEEL = 'STEEL', 'Steel'
        SAND = 'SAND', 'Sand'
        AGGREGATE = 'AGGREGATE', 'Aggregate'
        BLOCKS = 'BLOCKS', 'Blocks'
        PAINT = 'PAINT', 'Paint'
        TIMBER = 'TIMBER', 'Timber'
        PLUMBING = 'PLUMBING', 'Plumbing'
        ELECTRICAL = 'ELECTRICAL', 'Electrical'
        TOOLS = 'TOOLS', 'Tools'
        OTHER = 'OTHER', 'Other'

    class Unit(models.TextChoices):
        BAG = 'BAG', 'Bag'
        TONNE = 'TONNE', 'Tonne'
        PIECE = 'PIECE', 'Piece'
        LITRE = 'LITRE', 'Litre'
        METRE = 'METRE', 'Metre'
        SQUARE_METRE = 'SQUARE_METRE', 'Square Metre'
```

```
        CUBIC_METRE = 'CUBIC_METRE', 'Cubic Metre'
        KILOGRAM = 'KILOGRAM', 'Kilogram'
        OTHER = 'OTHER', 'Other'

    name = models.CharField(max_length=100, db_index=True)
    category = models.CharField(max_length=20, choices=Category.choices, db_index=True)
    unit = models.CharField(max_length=20, choices=Unit.choices)
    description = models.TextField(blank=True, default='')

    # Quantities
    current_quantity = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    minimum_threshold = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    reorder_quantity = models.DecimalField(max_digits=10, decimal_places=2, default=0)

    # Cost
    unit_cost = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    total_cost = models.DecimalField(max_digits=15, decimal_places=2, default=0)

    # Supplier
    supplier = models.ForeignKey(
        'inventory.Supplier',
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='materials'
    )

    # SKU (for shop integration)
    sku = models.CharField(max_length=50, blank=True, default='')

    # Status
    is_active = models.BooleanField(default=True)
    is_low_stock = models.BooleanField(default=False, db_index=True)

    # Shop integration
    is_available_in_shop = models.BooleanField(default=False)
    shop_price = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'materials'
        indexes = [
            models.Index(fields=['category']),
            models.Index(fields=['is_low_stock']),
            models.Index(fields=['is_available_in_shop']),
        ]

    def __str__(self):
        return f"{self.name} ({self.get_unit_display()})"

    def save(self, *args, **kwargs):
        # Check if stock is low
        self.is_low_stock = self.current_quantity <= self.minimum_threshold
        self.total_cost = self.current_quantity * self.unit_cost
        super().save(*args, **kwargs)

    def update_quantity(self, amount):
        """Update quantity and check alerts."""
        self.current_quantity += amount
        self.is_low_stock = self.current_quantity <= self.minimum_threshold
        self.total_cost = self.current_quantity * self.unit_cost
        self.save()

        if self.is_low_stock:
            self.create_alert()

    def create_alert(self):
        """Create low stock alert."""
        StockAlert.objects.get_or_create(
            material=self,
            status='OPEN',
            defaults={
                'message': f"Low stock: {self.name} ({self.current_quantity} {self.unit})",
                'threshold_quantity': self.minimum_threshold
            }
        )
```

Listing 13.1: Material Model – inventory/models/material.py

13.4 Stock Transactions

```
from django.db import models
from django.conf import settings
from django.utils import timezone
```

```
class StockTransaction(models.Model):
    """Stock IN/OUT transaction model."""

    class Type(models.TextChoices):
        IN = 'IN', 'Stock In'
        OUT = 'OUT', 'Stock Out'
        ADJUSTMENT = 'ADJUSTMENT', 'Adjustment'
        RETURN = 'RETURN', 'Return'

    class Status(models.TextChoices):
        PENDING = 'PENDING', 'Pending'
        CONFIRMED = 'CONFIRMED', 'Confirmed'
        CANCELLED = 'CANCELLED', 'Cancelled'

    material = models.ForeignKey(
        'inventory.Material',
        on_delete=models.CASCADE,
        related_name='transactions'
    )

    type = models.CharField(max_length=20, choices=Type.choices)
    quantity = models.DecimalField(max_digits=10, decimal_places=2)
    unit_cost = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)
    total_cost = models.DecimalField(max_digits=15, decimal_places=2, null=True, blank=True)

    # Who performed
    performed_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        related_name='stock_transactions'
    )

    # Reference
    reference = models.CharField(max_length=100, blank=True, default='')
    notes = models.TextField(blank=True, default='')

    # Receipt upload
    receipt = models.ImageField(
        upload_to='inventory/receipts/',
        blank=True,
        null=True
    )
    receipt_number = models.CharField(max_length=50, blank=True, default='')

    # Status
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING)
    confirmed_at = models.DateTimeField(null=True, blank=True)
    cancelled_at = models.DateTimeField(null=True, blank=True)
    cancelled_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        related_name='cancelled_transactions'
    )

    # Quantity before/after
    quantity_before = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)
    quantity_after = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)

    # Timestamps
    transaction_date = models.DateTimeField(default=timezone.now)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'stock_transactions'
        ordering = ['-transaction_date']
        indexes = [
            models.Index(fields=['material', '-transaction_date']),
            models.Index(fields=['status']),
        ]

    def __str__(self):
        return f"{self.get_type_display()} --- {self.material.name} ({self.quantity} {self.material.unit})"

    def save(self, *args, **kwargs):
        # Calculate total cost
        if self.unit_cost:
            self.total_cost = self.quantity * self.unit_cost

        # Get current quantity before update
        if not self.quantity_before:
            self.quantity_before = self.material.current_quantity

        # Update material quantity if confirmed
        if self.status == 'CONFIRMED' and not self.confirmed_at:
            if self.type == 'IN':
                self.material.update_quantity(self.quantity)
            elif self.type == 'OUT':
                self.material.update_quantity(-self.quantity)

            self.confirmed_at = timezone.now()
```

```
            self.quantity_after = self.material.current_quantity

        super().save(*args, **kwargs)

    def confirm(self):
        """Confirm and apply transaction."""
        self.status = self.Status.CONFIRMED
        self.confirmed_at = timezone.now()
        self.save()

    def cancel(self, user, reason=''):
        """Cancel transaction."""
        self.status = self.Status.CANCELLED
        self.cancelled_at = timezone.now()
        self.cancelled_by = user
        self.notes = f"{self.notes}\n\nCancelled: {reason}"
        self.save()
```

Listing 13.2: Stock Transaction Model – inventory/models/stock-transaction.py

13.5 Stock Service

```
from decimal import Decimal
from django.db import transaction
from django.utils import timezone
from inventory.models.material import Material
from inventory.models.stock_transaction import StockTransaction
from inventory.models.stock_alert import StockAlert
from notifications.services.notification_service import NotificationService
from accounts.services.audit_service import AuditService

class StockService:
    """Service for stock management operations."""

    @classmethod
    @transaction.atomic
    def record_stock_in(cls, material_id, quantity, performed_by,
                        unit_cost=None, notes='', receipt=None, receipt_number=''):
        """Record stock incoming."""
        material = Material.objects.get(id=material_id)

        if quantity <= 0:
            raise ValueError("Quantity must be positive")

        # Create transaction
        transaction_obj = StockTransaction.objects.create(
            material=material,
            type='IN',
            quantity=quantity,
            unit_cost=unit_cost or material.unit_cost,
            performed_by=performed_by,
            notes=notes,
            receipt=receipt,
            receipt_number=receipt_number,
            status='CONFIRMED'
        )

        # Update material
        material.current_quantity += quantity
        material.is_low_stock = material.current_quantity <= material.minimum_threshold
        material.total_cost = material.current_quantity * material.unit_cost
        material.save()

        # Update transaction with after quantity
        transaction_obj.quantity_after = material.current_quantity
        transaction_obj.save()

        # Check if alert should be cleared
        if not material.is_low_stock:
            StockAlert.objects.filter(material=material, status='OPEN').update(status='RESOLVED')

        # Log audit
        AuditService.log_action(
            user=performed_by,
            action='STOCK_IN_RECORDED',
            details={
                'material': material.name,
                'quantity': float(quantity),
                'new_balance': float(material.current_quantity)
            }
        )

        return transaction_obj

    @classmethod
    @transaction.atomic
```

```
    def record_stock_out(cls, material_id, quantity, performed_by, notes=''):
        """Record stock outgoing."""
        material = Material.objects.get(id=material_id)

        if quantity <= 0:
            raise ValueError("Quantity must be positive")

        if quantity > material.current_quantity:
            raise ValueError("Insufficient stock")

        # Create transaction
        transaction_obj = StockTransaction.objects.create(
            material=material,
            type='OUT',
            quantity=quantity,
            unit_cost=material.unit_cost,
            performed_by=performed_by,
            notes=notes,
            status='CONFIRMED'
        )

        # Update material
        material.current_quantity -= quantity
        material.is_low_stock = material.current_quantity <= material.minimum_threshold
        material.total_cost = material.current_quantity * material.unit_cost
        material.save()

        # Update transaction with after quantity
        transaction_obj.quantity_after = material.current_quantity
        transaction_obj.save()

        # Create alert if low stock
        if material.is_low_stock:
            StockAlert.objects.get_or_create(
                material=material,
                status='OPEN',
                defaults={
                    'message': f"Low stock: {material.name} ({material.current_quantity} {material.unit})",
                    'threshold_quantity': material.minimum_threshold
                }
            )

        # Log audit
        AuditService.log_action(
            user=performed_by,
            action='STOCK_OUT_RECORDED',
            details={
                'material': material.name,
                'quantity': float(quantity),
                'new_balance': float(material.current_quantity)
            }
        )

        return transaction_obj

    @classmethod
    def get_stock_summary(cls):
        """Get overall stock summary."""
        materials = Material.objects.filter(is_active=True)

        total_materials = materials.count()
        low_stock = materials.filter(is_low_stock=True).count()
        out_of_stock = materials.filter(current_quantity=0).count()
        total_value = materials.aggregate(
            total=models.Sum('total_cost')
        )['total'] or 0

        return {
            'total_materials': total_materials,
            'low_stock': low_stock,
            'out_of_stock': out_of_stock,
            'total_value': float(total_value),
            'low_stock_materials': materials.filter(is_low_stock=True)[:10],
            'recent_transactions': StockTransaction.objects.filter(
                status='CONFIRMED'
            ).order_by('-transaction_date')[:10]
        }
```

Listing 13.3: Stock Service – inventory/services/stock-service.py

Chapter Summary

Table 13.1: Inventory Management — Key Specifications

Property	**Value**
Material Categories	10 (Cement, Steel, Sand, Aggregates, etc.)
Material Units	8 (Bags, Tonnes, Pieces, Litres, etc.)
Transaction Types	4 (IN, OUT, Adjustment, Return)
Low Stock Alert	Automatic when current_qty <= minimum_threshold
Receipt Upload	Cloudinary storage
Supplier Management	Track supplier for each material
Stock Value	Automatic calculation (qty x unit_cost)
Shop Integration	Materials can be marked for online shop
Audit Trail	Every transaction logged
Notifications	Stock IN, Stock OUT, Low Stock alerts

Chapter 14

Financial Management

14.1 Overview

The Financial Management module handles payments, expenses, and budget tracking. It provides complete visibility into project finances and ensures every shilling is accounted for.

KEY INSIGHT Core Principle: Every financial transaction is tracked, categorized, and auditable. No money is ever unaccounted for.

14.2 Payment Model

```
from django.db import models
from django.conf import settings
from django.utils import timezone

class Payment(models.Model):
    """Payment record model."""

    class Type(models.TextChoices):
        MATERIAL = 'MATERIAL', 'Material'
        SALARY = 'SALARY', 'Salary'
        EQUIPMENT = 'EQUIPMENT', 'Equipment'
        SUBCONTRACTOR = 'SUBCONTRACTOR', 'Subcontractor'
        OTHER = 'OTHER', 'Other'

    class Method(models.TextChoices):
        CASH = 'CASH', 'Cash'
        MPESA = 'MPESA', 'M-Pesa'
        BANK = 'BANK', 'Bank Transfer'
        CHEQUE = 'CHEQUE', 'Cheque'
        PAYSTACK = 'PAYSTACK', 'Paystack'

    class Status(models.TextChoices):
        PENDING = 'PENDING', 'Pending'
        APPROVED = 'APPROVED', 'Approved'
        COMPLETED = 'COMPLETED', 'Completed'
        CANCELLED = 'CANCELLED', 'Cancelled'

    project = models.ForeignKey(
        'projects.Project',
        on_delete=models.CASCADE,
        related_name='payments'
    )

    # Payment details
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    type = models.CharField(max_length=20, choices=Type.choices)
    method = models.CharField(max_length=20, choices=Method.choices)
    description = models.TextField()

    # Recipient
    paid_to = models.CharField(max_length=200)
    paid_to_contact = models.CharField(max_length=100, blank=True, default='')

    # Reference
    reference = models.CharField(max_length=100, blank=True, default='')
    receipt = models.ImageField(upload_to='payments/receipts/', blank=True, null=True)
    invoice_number = models.CharField(max_length=50, blank=True, default='')

    # Status
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING, db_index=True)
    payment_date = models.DateField()
```

```
    # Approvals
    approved_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        related_name='approved_payments'
    )
    approved_at = models.DateTimeField(null=True, blank=True)

    # Completion
    completed_at = models.DateTimeField(null=True, blank=True)

    # Paystack reference (for online payments)
    paystack_reference = models.CharField(max_length=100, blank=True, default='')

    # Notes
    notes = models.TextField(blank=True, default='')

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        related_name='created_payments'
    )

    class Meta:
        db_table = 'payments'
        ordering = ['-payment_date']
        indexes = [
            models.Index(fields=['project', '-payment_date']),
            models.Index(fields=['type', 'status']),
        ]

    def __str__(self):
        return f"{self.get_type_display()} --- KES {self.amount} ({self.paid_to})"

    def approve(self, approver):
        """Approve payment."""
        self.status = self.Status.APPROVED
        self.approved_by = approver
        self.approved_at = timezone.now()
        self.save()

    def complete(self, completer, reference='', notes=''):
        """Complete payment."""
        self.status = self.Status.COMPLETED
        self.completed_at = timezone.now()
        if reference:
            self.reference = reference
        if notes:
            self.completion_notes = notes
        self.save()
```

Listing 14.1: Payment Model – finances/models/payment.py

14.3 Expense Model

```
from django.db import models
from django.conf import settings

class Expense(models.Model):
    """Expense record model."""

    class Category(models.TextChoices):
        LABOUR = 'LABOUR', 'Labour'
        MATERIALS = 'MATERIALS', 'Materials'
        EQUIPMENT = 'EQUIPMENT', 'Equipment'
        TRANSPORT = 'TRANSPORT', 'Transport'
        UTILITIES = 'UTILITIES', 'Utilities'
        OFFICE = 'OFFICE', 'Office'
        OTHER = 'OTHER', 'Other'

    project = models.ForeignKey(
        'projects.Project',
        on_delete=models.CASCADE,
        related_name='expenses'
    )

    amount = models.DecimalField(max_digits=12, decimal_places=2)
    category = models.CharField(max_length=20, choices=Category.choices)
    description = models.TextField()
    expense_date = models.DateField()

    receipt = models.ImageField(upload_to='expenses/receipts/', blank=True, null=True)
```

```
    receipt_number = models.CharField(max_length=50, blank=True, default='')

    # Vendor
    vendor_name = models.CharField(max_length=200, blank=True, default='')
    vendor_contact = models.CharField(max_length=100, blank=True, default='')

    # Notes
    notes = models.TextField(blank=True, default='')

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        related_name='created_expenses'
    )

    class Meta:
        db_table = 'expenses'
        ordering = ['-expense_date']
        indexes = [
            models.Index(fields=['project', '-expense_date']),
            models.Index(fields=['category']),
        ]

    def __str__(self):
        return f"{self.get_category_display()} --- KES {self.amount} ({self.expense_date})"
```

Listing 14.2: Expense Model – finances/models/expense.py

14.4 Budget Tracking

```
from django.db import models
from django.utils import timezone

class Budget(models.Model):
    """Project budget model."""

    class Category(models.TextChoices):
        MATERIALS = 'MATERIALS', 'Materials'
        LABOUR = 'LABOUR', 'Labour'
        EQUIPMENT = 'EQUIPMENT', 'Equipment'
        SUBCONTRACTOR = 'SUBCONTRACTOR', 'Subcontractor'
        TRANSPORT = 'TRANSPORT', 'Transport'
        CONTINGENCY = 'CONTINGENCY', 'Contingency'
        OTHER = 'OTHER', 'Other'

    project = models.ForeignKey(
        'projects.Project',
        on_delete=models.CASCADE,
        related_name='budgets'
    )

    category = models.CharField(max_length=20, choices=Category.choices)
    allocated = models.DecimalField(max_digits=15, decimal_places=2)
    spent = models.DecimalField(max_digits=15, decimal_places=2, default=0)
    committed = models.DecimalField(max_digits=15, decimal_places=2, default=0)

    description = models.TextField(blank=True, default='')

    @property
    def remaining(self):
        return self.allocated - self.spent - self.committed

    @property
    def spent_percentage(self):
        if self.allocated == 0:
            return 0
        return round((self.spent / self.allocated) * 100, 2)

    @property
    def is_over_budget(self):
        return self.spent > self.allocated

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'budgets'
        unique_together = ['project', 'category']

    def __str__(self):
        return f"{self.project.name} --- {self.get_category_display()}: KES {self.allocated}"

    def update_spent(self):
```

```
        """Update spent amount from expenses."""
        from finances.models.expense import Expense
        total = Expense.objects.filter(
            project=self.project,
            category=self.category
        ).aggregate(total=Sum('amount'))['total'] or 0
        self.spent = total
        self.save()
```

Listing 14.3: Budget Model – finances/models/budget.py

14.5 Financial Service

```
from django.db.models import Sum, Count
from django.utils import timezone
from datetime import timedelta
from decimal import Decimal
from finances.models.payment import Payment
from finances.models.expense import Expense
from finances.models.budget import Budget

class FinancialService:
    """Service for financial management."""

    @classmethod
    def get_project_financial_summary(cls, project):
        """Get financial summary for a project."""
        payments = Payment.objects.filter(project=project, status='COMPLETED')
        expenses = Expense.objects.filter(project=project)
        budgets = Budget.objects.filter(project=project)

        total_payments = payments.aggregate(total=Sum('amount'))['total'] or 0
        total_expenses = expenses.aggregate(total=Sum('amount'))['total'] or 0
        total_budget = budgets.aggregate(total=Sum('allocated'))['total'] or 0
        total_spent = budgets.aggregate(total=Sum('spent'))['total'] or 0

        return {
            'project': project,
            'total_payments': float(total_payments),
            'total_expenses': float(total_expenses),
            'total_budget': float(total_budget),
            'total_spent': float(total_spent),
            'remaining_budget': float(total_budget - total_spent),
            'budget_used_percentage': round((total_spent / total_budget * 100), 1) if total_budget > 0 else 0,
            'payment_count': payments.count(),
            'expense_count': expenses.count(),
            'budget_categories': budgets.all(),
            'recent_payments': payments.order_by('-payment_date')[:10],
            'recent_expenses': expenses.order_by('-expense_date')[:10]
        }

    @classmethod
    def get_overall_financial_summary(cls):
        """Get overall financial summary for all projects."""
        projects = Project.objects.filter(is_archived=False)
        total_payments = Payment.objects.filter(status='COMPLETED').aggregate(total=Sum('amount'))['total'] or 0
        total_expenses = Expense.objects.aggregate(total=Sum('amount'))['total'] or 0
        total_budget = Budget.objects.aggregate(total=Sum('allocated'))['total'] or 0
        total_spent = Budget.objects.aggregate(total=Sum('spent'))['total'] or 0

        return {
            'total_projects': projects.count(),
            'total_payments': float(total_payments),
            'total_expenses': float(total_expenses),
            'total_budget': float(total_budget),
            'total_spent': float(total_spent),
            'remaining_budget': float(total_budget - total_spent),
            'budget_used_percentage': round((total_spent / total_budget * 100), 1) if total_budget > 0 else 0,
            'project_summaries': [cls.get_project_financial_summary(p) for p in projects[:5]]
        }

    @classmethod
    def get_payment_chart_data(cls, months=6):
        """Get payment trend chart data."""
        data = []
        for i in range(months):
            month_start = timezone.now().replace(day=1) - timedelta(days=30 * i)
            month_end = (month_start + timedelta(days=32)).replace(day=1) - timedelta(days=1)

            payments = Payment.objects.filter(
                payment_date__gte=month_start.date(),
                payment_date__lt=month_end.date(),
                status='COMPLETED'
            )

            total = payments.aggregate(total=Sum('amount'))['total'] or 0
```

```
            data.append({
                'month': month_start.strftime('%b %Y'),
                'total': float(total),
                'count': payments.count()
            })
        return data[::-1]
```

Listing 14.4: Financial Service – finances/services/financial-service.py

Chapter Summary

Table 14.1: Financial Management — Key Specifications

Property	**Value**
Payment Types	5 (Material, Salary, Equipment, Subcontractor, Other)
Payment Methods	5 (Cash, M-Pesa, Bank, Cheque, Paystack)
Payment Status	4 (Pending, Approved, Completed, Cancelled)
Expense Categories	7 (Labour, Materials, Equipment, Transport, etc.)
Budget Tracking	Per-project with category breakdown
Receipt Upload	Cloudinary storage
Reports	Project financial summary, expense breakdown
Charts	Payment trends, expense breakdown
Audit Trail	All financial transactions logged

Chapter 15

Client Management

15.1 Overview

The Client Management module provides a centralized database for tracking clients and leads. It integrates with the contact form and quote request system to capture new leads automatically.

> **KEY INSIGHT** Core Principle: Every client is more than a name. They have projects, orders, and a history. Caldan Construction tracks it all.

15.2 Client Model

```
from django.db import models
from django.utils import timezone

class Client(models.Model):
    """Client and lead tracking model."""

    class Type(models.TextChoices):
        INDIVIDUAL = 'INDIVIDUAL', 'Individual'
        COMPANY = 'COMPANY', 'Company'
        INSTITUTION = 'INSTITUTION', 'Institution'
        OTHER = 'OTHER', 'Other'

    class Status(models.TextChoices):
        LEAD = 'LEAD', 'Lead'
        ACTIVE = 'ACTIVE', 'Active Client'
        INACTIVE = 'INACTIVE', 'Inactive'
        CONVERTED = 'CONVERTED', 'Converted'

    # Identity
    name = models.CharField(max_length=200)
    type = models.CharField(max_length=20, choices=Type.choices, default=Type.INDIVIDUAL)
    company_name = models.CharField(max_length=200, blank=True, default='')

    # Contact
    email = models.EmailField()
    phone = models.CharField(max_length=15)
    alternative_phone = models.CharField(max_length=15, blank=True, default='')

    # Address
    address = models.TextField(blank=True, default='')
    county = models.CharField(max_length=100, blank=True, default='')
    town = models.CharField(max_length=100, blank=True, default='')

    # Status
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.LEAD)

    # Source
    source = models.CharField(max_length=50, blank=True, default='')  # Website, Referral, Direct

    # Projects linked
    projects = models.ManyToManyField('projects.Project', related_name='clients', blank=True)

    # Orders linked
    orders = models.ManyToManyField('shop.Order', related_name='clients', blank=True)

    # Notes
    notes = models.TextField(blank=True, default='')

    # Timestamps
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    last_contact = models.DateTimeField(null=True, blank=True)
```

```
    class Meta:
        db_table = 'clients'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['email']),
            models.Index(fields=['phone']),
            models.Index(fields=['status']),
        ]

    def __str__(self):
        return f"{self.name} ({self.get_status_display()})"

    def convert_to_active(self):
        """Convert lead to active client."""
        self.status = self.Status.ACTIVE
        self.save()

    def add_project(self, project):
        """Add project to client."""
        self.projects.add(project)
        self.last_contact = timezone.now()
        self.save()
```

Listing 15.1: Client Model – clients/models/client.py

15.3 Lead Capture

When a contact form or quote request is submitted, a lead is automatically created:

```
from clients.models.client import Client
from notifications.services.email_service import EmailService

class LeadService:
    """Service for handling leads from website forms."""

    @classmethod
    def create_from_contact_form(cls, name, email, phone, message, subject=''):
        """Create a lead from contact form submission."""
        client, created = Client.objects.get_or_create(
            email=email,
            defaults={
                'name': name,
                'phone': phone,
                'source': 'Website Contact Form',
                'status': Client.Status.LEAD,
                'notes': f"Message: {message[:500]}"
            }
        )

        if not created:
            client.notes += f"\n\n{timezone.now()}: New contact form submission:\n{message[:200]}"
            client.save()

        # Notify admin
        EmailService.send_admin_contact_alert(name, email, phone, message)

        return client

    @classmethod
    def create_from_quote_request(cls, name, email, phone, project_type, budget, timeline, details):
        """Create a lead from quote request form."""
        client, created = Client.objects.get_or_create(
            email=email,
            defaults={
                'name': name,
                'phone': phone,
                'source': 'Website Quote Request',
                'status': Client.Status.LEAD,
                'notes': f"Project Type: {project_type}\nBudget: {budget}\nTimeline: {timeline}\n\nDetails:\n{details
    ↪ }"
            }
        )

        if not created:
            client.notes += f"\n\n{timezone.now()}: New quote request:\nProject Type: {project_type}\nBudget: {budget
    ↪ }"
            client.save()

        # Notify admin
        EmailService.send_admin_quote_alert(name, email, phone, project_type, budget, details)

        return client
```

Listing 15.2: Lead Service – clients/services/lead-service.py

Chapter Summary

Table 15.1: Client Management — Key Specifications

Property	Value
Client Types	4 (Individual, Company, Institution, Other)
Client Status	4 (Lead, Active, Inactive, Converted)
Lead Capture	Automatic from contact forms and quote requests
Project Linking	Many-to-many with projects
Order Linking	Many-to-many with shop orders
Notes	Free-text notes field
Audit Trail	All client interactions logged

Chapter 16

Team Management

16.1 Overview

The Team Management module handles team member profiles and management. It allows the Platform Admin to add, edit, and display team members on the public website.

> **KEY INSIGHT** Core Principle: A company is only as good as its people. The team management module puts the human face of Caldan Construction front and center.

16.2 Team Member Model

```
from django.db import models
from django.utils import timezone

class TeamMember(models.Model):
    """Team member profile model."""

    class Role(models.TextChoices):
        DIRECTOR = 'DIRECTOR', 'Director'
        MANAGER = 'MANAGER', 'Manager'
        ENGINEER = 'ENGINEER', 'Engineer'
        SUPERVISOR = 'SUPERVISOR', 'Supervisor'
        ADMIN = 'ADMIN', 'Administration'
        OTHER = 'OTHER', 'Other'

    name = models.CharField(max_length=150)
    role = models.CharField(max_length=20, choices=Role.choices)
    title = models.CharField(max_length=100, blank=True, default='')
    bio = models.TextField(blank=True, default='')
    photo = models.ImageField(upload_to='team/', blank=True, null=True)

    # Contact (optional, for internal use)
    email = models.EmailField(blank=True, default='')
    phone = models.CharField(max_length=15, blank=True, default='')

    # Display order
    display_order = models.PositiveSmallIntegerField(default=0)

    # Status
    is_active = models.BooleanField(default=True)
    is_featured = models.BooleanField(default=False)

    # Social links
    linkedin = models.URLField(blank=True, default='')
    twitter = models.URLField(blank=True, default='')

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'team_members'
        ordering = ['display_order', 'name']

    def __str__(self):
        return f"{self.name} --- {self.get_role_display()}"
```

Listing 16.1: Team Member Model – team/models/team-member.py

16.3 Team Management Views

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from accounts.permissions.decorators import admin_required
from team.models import TeamMember

@login_required
@admin_required
def team_list(request):
    """List all team members."""
    members = TeamMember.objects.all().order_by('display_order')
    return render(request, 'dashboard/admin/team/list.html', {'members': members})

@login_required
@admin_required
def team_create(request):
    """Create a new team member."""
    if request.method == 'POST':
        form = TeamMemberForm(request.POST, request.FILES)
        if form.is_valid():
            member = form.save()
            messages.success(request, f"Team member {member.name} created.")
            return redirect('team:list')
    else:
        form = TeamMemberForm()

    return render(request, 'dashboard/admin/team/create.html', {'form': form})

@login_required
@admin_required
def team_edit(request, member_id):
    """Edit a team member."""
    member = get_object_or_404(TeamMember, id=member_id)

    if request.method == 'POST':
        form = TeamMemberForm(request.POST, request.FILES, instance=member)
        if form.is_valid():
            form.save()
            messages.success(request, f"Team member {member.name} updated.")
            return redirect('team:list')
    else:
        form = TeamMemberForm(instance=member)

    return render(request, 'dashboard/admin/team/edit.html', {'form': form, 'member': member})

@login_required
@admin_required
def team_delete(request, member_id):
    """Delete a team member."""
    member = get_object_or_404(TeamMember, id=member_id)

    if request.method == 'POST':
        member.delete()
        messages.success(request, f"Team member {member.name} deleted.")
        return redirect('team:list')

    return render(request, 'dashboard/admin/team/delete.html', {'member': member})
```

Listing 16.2: Team Management Views – team/views/-admin-views.py

16.4 Public Team Display

The team members are displayed on the About page and can be filtered by role:

```
<section class="team-section">
    <div class="container">
        <h2 class="section-title">Meet the Team</h2>
        <p class="section-subtitle">The people behind Caldan Construction</p>

        <div class="team-grid">
            {% for member in team_members %}
            <div class="team-card">
                {% if member.photo %}
                <img src="{{ member.photo.url }}" alt="{{ member.name }}" class="team-photo">
                {% else %}
                <div class="team-photo-placeholder">
                    {{ member.name|first|upper }}
                </div>
                {% endif %}
                <h3>{{ member.name }}</h3>
                <p class="team-role">{{ member.get_role_display }}</p>
                {% if member.title %}
```

```
                <p class="team-title">{{ member.title }}</p>
                {% endif %}
                {% if member.bio %}
                <p class="team-bio">{{ member.bio|truncatewords:30 }}</p>
                {% endif %}
            </div>
            {% endfor %}
        </div>
    </div>
</section>
```

Listing 16.3: Public Team Display – website/templates/website/about.html

Chapter Summary

Table 16.1: Team Management — Key Specifications

Property	**Value**
Team Roles	6 (Director, Manager, Engineer, Supervisor, Admin, Other)
Fields	Name, Role, Title, Bio, Photo, Contact, Social Links
Public Display	On About page, sorted by display order
Featured Members	Can be marked as featured
CRUD	Full CRUD from admin dashboard
Audit Trail	All team member changes logged

Chapter 17

Services Management

17.1 Overview

The Services Management module allows the Platform Admin to manage the services offered by Caldan Construction. Services are displayed on the public website and can be linked to projects.

> **KEY INSIGHT** Core Principle: Services are the foundation of what Caldan offers. They must be easy to manage and display clearly to potential clients.

17.2 Service Model

```
from django.db import models
from django.utils.text import slugify

class Service(models.Model):
    """Construction services offered by Caldan."""

    class Category(models.TextChoices):
        DESIGN = 'DESIGN', 'Design & Construction'
        RENOVATION = 'RENOVATION', 'Renovations'
        INTERIOR = 'INTERIOR', 'Interior Design'
        WELDING = 'WELDING', 'Welding & Fabrication'
        CONCRETE = 'CONCRETE', 'Concrete Products'
        INDUSTRIAL = 'INDUSTRIAL', 'Industrial Floor Solutions'
        WATERPROOFING = 'WATERPROOFING', 'Waterproofing Works'
        INFRASTRUCTURE = 'INFRASTRUCTURE', 'Roads & Stormwater Infrastructure'
        CHEMICALS = 'CHEMICALS', 'Construction Chemicals'

    name = models.CharField(max_length=200)
    category = models.CharField(max_length=20, choices=Category.choices, db_index=True)
    slug = models.SlugField(unique=True)
    description = models.TextField()
    icon = models.CharField(max_length=50, help_text="Font Awesome icon class")
    image = models.ImageField(upload_to='services/', blank=True, null=True)

    # Additional details
    long_description = models.TextField(blank=True, default='')
    features = models.JSONField(default=list, blank=True)  # List of feature strings

    is_active = models.BooleanField(default=True)
    display_order = models.PositiveSmallIntegerField(default=0)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'services'
        ordering = ['display_order', 'name']

    def __str__(self):
        return self.name

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)
```

Listing 17.1: Service Model – services/models/service.py

17.3 Service Management Views

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from accounts.permissions.decorators import admin_required
from services.models import Service

@login_required
@admin_required
def service_list(request):
    """List all services."""
    services = Service.objects.all().order_by('display_order')
    return render(request, 'dashboard/admin/services/list.html', {'services': services})

@login_required
@admin_required
def service_create(request):
    """Create a new service."""
    if request.method == 'POST':
        form = ServiceForm(request.POST, request.FILES)
        if form.is_valid():
            service = form.save()
            messages.success(request, f"Service {service.name} created.")
            return redirect('services:list')
    else:
        form = ServiceForm()

    return render(request, 'dashboard/admin/services/create.html', {'form': form})

@login_required
@admin_required
def service_edit(request, service_id):
    """Edit a service."""
    service = get_object_or_404(Service, id=service_id)

    if request.method == 'POST':
        form = ServiceForm(request.POST, request.FILES, instance=service)
        if form.is_valid():
            form.save()
            messages.success(request, f"Service {service.name} updated.")
            return redirect('services:list')
    else:
        form = ServiceForm(instance=service)

    return render(request, 'dashboard/admin/services/edit.html', {'form': form, 'service': service})

@login_required
@admin_required
def service_delete(request, service_id):
    """Delete a service."""
    service = get_object_or_404(Service, id=service_id)

    if request.method == 'POST':
        service.delete()
        messages.success(request, f"Service {service.name} deleted.")
        return redirect('services:list')

    return render(request, 'dashboard/admin/services/delete.html', {'service': service})
```

Listing 17.2: Service Management Views – services/views/-admin-views.py

Chapter Summary

Table 17.1: Services Management — Key Specifications

Property	Value
Service Categories	9
Fields	Name, Category, Slug, Description, Icon, Image, Features
Public Display	On Services page, sorted by display order
Project Linking	Many-to-many with projects
CRUD	Full CRUD from admin dashboard
Audit Trail	All service changes logged

Part V

Online Shop

Chapter 18

Shop & Product Catalog

18.1 Overview

The Online Shop module transforms Caldan Construction's material inventory into an e-commerce platform. Clients can browse materials, view products, and place orders online.

KEY INSIGHT Core Principle: The shop integrates seamlessly with inventory. When a client places an order, stock is automatically deducted from inventory. No manual updates required.

18.2 Product Model

```
from django.db import models
from django.utils.text import slugify
from django.utils import timezone

class Product(models.Model):
    """Construction materials for online shop."""

    class Category(models.TextChoices):
        CEMENT = 'CEMENT', 'Cement'
        STEEL = 'STEEL', 'Steel'
        SAND = 'SAND', 'Sand'
        AGGREGATE = 'AGGREGATE', 'Aggregate'
        BLOCKS = 'BLOCKS', 'Blocks'
        PAINT = 'PAINT', 'Paint'
        TIMBER = 'TIMBER', 'Timber'
        PLUMBING = 'PLUMBING', 'Plumbing'
        ELECTRICAL = 'ELECTRICAL', 'Electrical'
        TOOLS = 'TOOLS', 'Tools'
        OTHER = 'OTHER', 'Other'

    class Unit(models.TextChoices):
        BAG = 'BAG', 'Bag'
        TONNE = 'TONNE', 'Tonne'
        PIECE = 'PIECE', 'Piece'
        LITRE = 'LITRE', 'Litre'
        METRE = 'METRE', 'Metre'
        SQUARE_METRE = 'SQUARE_METRE', 'Square Metre'
        CUBIC_METRE = 'CUBIC_METRE', 'Cubic Metre'
        KILOGRAM = 'KILOGRAM', 'Kilogram'
        OTHER = 'OTHER', 'Other'

    # Basic info
    name = models.CharField(max_length=200)
    slug = models.SlugField(unique=True)
    category = models.CharField(max_length=20, choices=Category.choices, db_index=True)
    unit = models.CharField(max_length=20, choices=Unit.choices)

    # Description
    description = models.TextField()
    short_description = models.CharField(max_length=200, blank=True, default='')

    # Pricing
    price = models.DecimalField(max_digits=10, decimal_places=2)
    compare_price = models.DecimalField(max_digits=10, decimal_places=2, null=True, blank=True)

    # Stock (linked to inventory)
    inventory_material = models.ForeignKey(
        'inventory.Material',
        on_delete=models.SET_NULL,
        null=True,
```

```
        blank=True,
        related_name='shop_products'
    )
    stock_quantity = models.DecimalField(max_digits=10, decimal_places=2, default=0)

    # Images
    image = models.ImageField(upload_to='shop/products/')
    additional_images = models.JSONField(default=list, blank=True)

    # Status
    is_active = models.BooleanField(default=True)
    is_featured = models.BooleanField(default=False)
    in_stock = models.BooleanField(default=True)

    # Meta
    meta_title = models.CharField(max_length=200, blank=True, default='')
    meta_description = models.CharField(max_length=200, blank=True, default='')

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'shop_products'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['category', 'is_active']),
            models.Index(fields=['is_featured']),
            models.Index(fields=['in_stock']),
        ]

    def __str__(self):
        return f"{self.name} --- KES {self.price}"

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.name)
        super().save(*args, **kwargs)

    def update_stock(self):
        """Update stock from linked inventory material."""
        if self.inventory_material:
            self.stock_quantity = self.inventory_material.current_quantity
            self.in_stock = self.stock_quantity > 0
            self.save(update_fields=['stock_quantity', 'in_stock'])

    @property
    def image_url(self):
        """Return image URL or placeholder."""
        if self.image:
            return self.image.url
        return '/static/images/placeholder-product.png'
```

Listing 18.1: Product Model – shop/models/product.py

18.3 Product Views

```
from django.shortcuts import render, get_object_or_404
from django.core.paginator import Paginator
from shop.models.product import Product

def shop_home(request):
    """Shop homepage with product catalog."""
    products = Product.objects.filter(is_active=True)

    # Filters
    category = request.GET.get('category')
    search = request.GET.get('search')

    if category:
        products = products.filter(category=category)
    if search:
        products = products.filter(
            models.Q(name__icontains=search) |
            models.Q(description__icontains=search)
        )

    # Pagination
    paginator = Paginator(products, 20)
    page = request.GET.get('page', 1)
    products = paginator.get_page(page)

    # Categories for filter
    categories = Product.Category.choices

    context = {
        'products': products,
```

```
        'categories': categories,
        'selected_category': category,
        'search': search,
    }
    return render(request, 'shop/index.html', context)

def product_detail(request, slug):
    """Product detail page."""
    product = get_object_or_404(Product, slug=slug, is_active=True)

    # Related products (same category)
    related = Product.objects.filter(
        category=product.category,
        is_active=True
    ).exclude(id=product.id)[:4]

    context = {
        'product': product,
        'related_products': related,
    }
    return render(request, 'shop/product_detail.html', context)
```

Listing 18.2: Product Views – shop/views/-public-views.py

Chapter Summary

Table 18.1: Shop & Product Catalog — Key Specifications

Property	**Value**
Product Categories	10 (Cement, Steel, Sand, etc.)
Product Units	8 (Bag, Tonne, Piece, Litre, etc.)
Inventory Integration	Linked via inventory_material
Stock Tracking	Automatic from inventory
Images	Primary + additional (JSON)
Search	Category filter + text search
Pagination	20 products per page
Related Products	Same category

Chapter 19

Cart & Checkout

19.1 Overview

The Cart and Checkout system provides a seamless shopping experience for clients. The cart is session-based, allowing clients to browse and add items without creating an account.

> **KEY INSIGHT** Core Principle: Frictionless checkout = higher conversion. The cart and checkout process is designed to be simple, fast, and secure.

19.2 Cart Models

```
from django.db import models
from django.utils import timezone

class Cart(models.Model):
    """Client shopping cart."""
    session_key = models.CharField(max_length=40, unique=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'shop_carts'

    def __str__(self):
        return f"Cart {self.session_key}"

    @property
    def total_items(self):
        return self.items.aggregate(total=Sum('quantity'))['total'] or 0

    @property
    def subtotal(self):
        return self.items.aggregate(
            total=Sum(models.F('quantity') * models.F('unit_price'))
        )['total'] or 0

    def clear(self):
        self.items.all().delete()

class CartItem(models.Model):
    """Individual cart items."""
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE, related_name='items')
    product = models.ForeignKey('shop.Product', on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField(default=1)
    unit_price = models.DecimalField(max_digits=10, decimal_places=2)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'shop_cart_items'
        unique_together = ['cart', 'product']

    def __str__(self):
        return f"{self.quantity} x {self.product.name}"

    @property
    def total_price(self):
        return self.quantity * self.unit_price
```

Listing 19.1: Cart Models – shop/models/cart.py

19.3 Cart Service

```
from shop.models.cart import Cart, CartItem
from shop.models.product import Product

class CartService:
    """Service for cart operations."""

    @classmethod
    def get_or_create_cart(cls, session_key):
        """Get cart by session key or create new."""
        cart, created = Cart.objects.get_or_create(session_key=session_key)
        return cart

    @classmethod
    def add_item(cls, cart, product_id, quantity=1):
        """Add item to cart."""
        product = Product.objects.get(id=product_id, is_active=True)

        cart_item, created = CartItem.objects.get_or_create(
            cart=cart,
            product=product,
            defaults={
                'quantity': quantity,
                'unit_price': product.price
            }
        )

        if not created:
            cart_item.quantity += quantity
            cart_item.save()

        cart.updated_at = timezone.now()
        cart.save()

        return cart_item

    @classmethod
    def update_quantity(cls, cart_item_id, quantity):
        """Update item quantity."""
        cart_item = CartItem.objects.get(id=cart_item_id)

        if quantity <= 0:
            cart_item.delete()
            return None

        cart_item.quantity = quantity
        cart_item.save()
        return cart_item

    @classmethod
    def remove_item(cls, cart_item_id):
        """Remove item from cart."""
        cart_item = CartItem.objects.get(id=cart_item_id)
        cart_item.delete()

    @classmethod
    def clear_cart(cls, cart):
        """Clear all items from cart."""
        cart.clear()
```

Listing 19.2: Cart Service – shop/services/cart-service.py

19.4 Checkout Views

```
from django.shortcuts import render, redirect
from django.contrib import messages
from django.views import View
from shop.services.cart_service import CartService
from shop.services.checkout_service import CheckoutService

class CheckoutView(View):
    """Checkout page."""

    def get(self, request):
        cart = CartService.get_or_create_cart(request.session.session_key)

        if cart.total_items == 0:
            messages.warning(request, "Your cart is empty.")
            return redirect('shop:cart')

        context = {
            'cart': cart,
```

```
            'subtotal': cart.subtotal,
            'delivery_fee': 500,  # Flat rate
            'total': cart.subtotal + 500,
        }
        return render(request, 'shop/checkout.html', context)

    def post(self, request):
        """Process checkout."""
        cart = CartService.get_or_create_cart(request.session.session_key)

        if cart.total_items == 0:
            messages.error(request, "Your cart is empty.")
            return redirect('shop:cart')

        # Create order
        order = CheckoutService.create_order(
            cart=cart,
            client_name=request.POST.get('name'),
            client_email=request.POST.get('email'),
            client_phone=request.POST.get('phone'),
            delivery_address=request.POST.get('address'),
            delivery_county=request.POST.get('county'),
            delivery_town=request.POST.get('town'),
            notes=request.POST.get('notes', '')
        )

        # Initialize payment
        payment_data = CheckoutService.initiate_payment(order)

        return render(request, 'shop/payment.html', {
            'order': order,
            'paystack_public_key': settings.PAYSTACK_PUBLIC_KEY,
            'payment_data': payment_data,
        })
```

Listing 19.3: Checkout Views – shop/views/-checkout-views.py

19.5 Checkout Service

```
from django.db import transaction
from django.utils import timezone
from shop.models.order import Order
from shop.models.order_item import OrderItem
from inventory.services.stock_service import StockService
from notifications.services.email_service import EmailService

class CheckoutService:
    """Service for checkout operations."""

    @classmethod
    @transaction.atomic
    def create_order(cls, cart, client_name, client_email, client_phone,
                     delivery_address, delivery_county, delivery_town, notes=''):
        """Create order from cart."""
        order = Order.objects.create(
            order_number=cls.generate_order_number(),
            client_name=client_name,
            client_email=client_email,
            client_phone=client_phone,
            delivery_address=delivery_address,
            delivery_county=delivery_county,
            delivery_town=delivery_town,
            notes=notes,
            subtotal=cart.subtotal,
            delivery_fee=500,
            total=cart.subtotal + 500,
            status=Order.Status.PENDING
        )

        # Create order items
        for cart_item in cart.items.all():
            OrderItem.objects.create(
                order=order,
                product=cart_item.product,
                quantity=cart_item.quantity,
                unit_price=cart_item.unit_price,
                total_price=cart_item.total_price
            )

        # Clear cart
        cart.clear()

        return order

    @classmethod
    def generate_order_number(cls):
```

```
        """Generate unique order number: ORD-2026-0001."""
        year = timezone.now().year
        count = Order.objects.filter(created_at__year=year).count() + 1
        return f"ORD-{year}-{count:04d}"

    @classmethod
    def initiate_payment(cls, order):
        """Initiate Paystack payment."""
        import requests
        from django.conf import settings

        url = "https://api.paystack.co/transaction/initialize"
        headers = {
            "Authorization": f"Bearer {settings.PAYSTACK_SECRET_KEY}",
            "Content-Type": "application/json"
        }
        payload = {
            "email": order.client_email,
            "amount": int(order.total * 100),
            "reference": f"ORD-{order.id}",
            "callback_url": f"{settings.BASE_URL}/shop/payment-callback/",
            "metadata": {
                "order_id": order.id,
                "order_number": order.order_number,
            }
        }

        response = requests.post(url, json=payload, headers=headers)
        return response.json().get('data', {})
```

Listing 19.4: Checkout Service – shop/services/checkout-service.py

Chapter Summary

Table 19.1: Cart & Checkout — Key Specifications

Property	**Value**
Cart Storage	Session-based (no account required)
Cart Operations	Add, Update, Remove, Clear
Checkout Fields	Name, Email, Phone, Address, County, Town
Order Number	ORD-YYYY-XXXX (auto-generated)
Delivery Fee	Flat rate (configurable)
Payment	Paystack (M-Pesa, Card, Bank)
Stock Deduction	Automatic on payment confirmation
Notifications	Order confirmation email

Chapter 20

Order Management

20.1 Overview

The Order Management module provides complete visibility and control over all orders placed through the online shop. The Platform Admin can view, update status, and manage orders from a single dashboard.

KEY INSIGHT Core Principle: Every order is tracked from placement to delivery. No order is ever lost or forgotten.

20.2 Order Models

```
from django.db import models
from django.conf import settings
from django.utils import timezone

class Order(models.Model):
    """Client order model."""

    class Status(models.TextChoices):
        PENDING = 'PENDING', 'Pending Payment'
        PAID = 'PAID', 'Paid'
        PROCESSING = 'PROCESSING', 'Processing'
        SHIPPED = 'SHIPPED', 'Shipped'
        DELIVERED = 'DELIVERED', 'Delivered'
        COMPLETED = 'COMPLETED', 'Completed'
        CANCELLED = 'CANCELLED', 'Cancelled'

    order_number = models.CharField(max_length=20, unique=True, db_index=True)

    # Client details
    client_name = models.CharField(max_length=150)
    client_email = models.EmailField()
    client_phone = models.CharField(max_length=15)

    # Delivery
    delivery_address = models.TextField()
    delivery_county = models.CharField(max_length=100)
    delivery_town = models.CharField(max_length=100)
    delivery_notes = models.TextField(blank=True, default='')

    # Financial
    subtotal = models.DecimalField(max_digits=12, decimal_places=2)
    delivery_fee = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    total = models.DecimalField(max_digits=12, decimal_places=2)

    # Status
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING, db_index=True)

    # Payment
    paystack_reference = models.CharField(max_length=100, blank=True, default='')
    paid_at = models.DateTimeField(null=True, blank=True)

    # Tracking
    tracking_number = models.CharField(max_length=100, blank=True, default='')
    estimated_delivery = models.DateField(null=True, blank=True)

    # Notes
    notes = models.TextField(blank=True, default='')
    admin_notes = models.TextField(blank=True, default='')

    # Timestamps
```

```
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'shop_orders'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['order_number']),
            models.Index(fields=['status']),
            models.Index(fields=['client_email']),
        ]

    def __str__(self):
        return f"{self.order_number} --- {self.client_name}"

    def mark_paid(self, reference):
        """Mark order as paid."""
        self.status = self.Status.PAID
        self.paid_at = timezone.now()
        self.paystack_reference = reference
        self.save()

        # Deduct stock
        for item in self.items.all():
            if item.product.inventory_material:
                StockService.record_stock_out(
                    material_id=item.product.inventory_material.id,
                    quantity=item.quantity,
                    performed_by=None,
                    notes=f"Order {self.order_number}"
                )

    def update_status(self, new_status, notes=''):
        """Update order status."""
        self.status = new_status
        if notes:
            self.admin_notes += f"\n\n{timezone.now()}: {notes}"
        self.save()

class OrderItem(models.Model):
    """Individual order items."""
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name='items')
    product = models.ForeignKey('shop.Product', on_delete=models.PROTECT)
    quantity = models.PositiveIntegerField()
    unit_price = models.DecimalField(max_digits=10, decimal_places=2)
    total_price = models.DecimalField(max_digits=12, decimal_places=2)

    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'shop_order_items'

    def __str__(self):
        return f"{self.quantity} x {self.product.name}"
```

Listing 20.1: Order Models – shop/models/order.py

20.3 Order Management Views

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib import messages
from django.contrib.auth.decorators import login_required
from django.core.paginator import Paginator
from accounts.permissions.decorators import admin_required
from shop.models.order import Order

@login_required
@admin_required
def order_list(request):
    """List all orders with filters."""
    queryset = Order.objects.all()

    # Filters
    status = request.GET.get('status')
    search = request.GET.get('search')
    date_from = request.GET.get('date_from')
    date_to = request.GET.get('date_to')

    if status:
        queryset = queryset.filter(status=status)
    if search:
        queryset = queryset.filter(
            models.Q(order_number__icontains=search) |
            models.Q(client_name__icontains=search) |
            models.Q(client_email__icontains=search)
        )
```

```
    if date_from:
        queryset = queryset.filter(created_at__date__gte=date_from)
    if date_to:
        queryset = queryset.filter(created_at__date__lte=date_to)

    # Stats
    stats = {
        'total': Order.objects.count(),
        'pending': Order.objects.filter(status='PENDING').count(),
        'processing': Order.objects.filter(status='PROCESSING').count(),
        'shipped': Order.objects.filter(status='SHIPPED').count(),
        'delivered': Order.objects.filter(status='DELIVERED').count(),
        'cancelled': Order.objects.filter(status='CANCELLED').count(),
        'total_revenue': Order.objects.filter(status__in=['PAID', 'PROCESSING', 'SHIPPED', 'DELIVERED']).aggregate(
            total=Sum('total')
        )['total'] or 0,
    }

    # Pagination
    paginator = Paginator(queryset, 20)
    page = request.GET.get('page', 1)
    orders = paginator.get_page(page)

    context = {
        'orders': orders,
        'stats': stats,
        'active_status': status,
        'search': search,
        'date_from': date_from,
        'date_to': date_to,
    }
    return render(request, 'dashboard/admin/orders/list.html', context)

@login_required
@admin_required
def order_detail(request, order_id):
    """Order detail view."""
    order = get_object_or_404(Order, id=order_id)
    return render(request, 'dashboard/admin/orders/detail.html', {'order': order})

@login_required
@admin_required
def order_update_status(request, order_id):
    """Update order status."""
    order = get_object_or_404(Order, id=order_id)

    if request.method == 'POST':
        new_status = request.POST.get('status')
        notes = request.POST.get('notes', '')

        if new_status in Order.Status.values:
            order.update_status(new_status, notes)
            messages.success(request, f"Order {order.order_number} updated to {new_status}.")
        else:
            messages.error(request, "Invalid status.")

        return redirect('shop_admin:order_detail', order_id=order.id)

    return redirect('shop_admin:order_detail', order_id=order.id)
```

Listing 20.2: Order Management Views – shop/views/-admin-views.py

20.4 Order Status Workflow

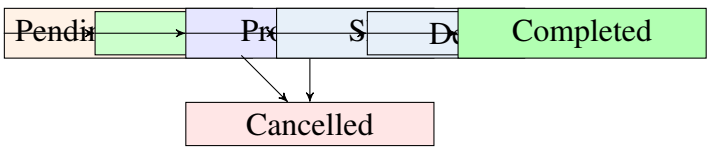


Figure 20.1: Order Status Workflow

20.5 Order Tracking (Client Side)

Clients can track their orders without creating an account:

```
def order_tracking(request):
```

```
    """Order tracking page."""
    order_number = request.GET.get('order_number')

    if order_number:
        try:
            order = Order.objects.get(order_number=order_number)
            context = {
                'order': order,
                'found': True,
            }
        except Order.DoesNotExist:
            context = {
                'found': False,
                'error': 'Order not found. Please check your order number.'
            }
    else:
        context = {'found': False}

    return render(request, 'shop/order_tracking.html', context)
```

Listing 20.3: Order Tracking – shop/views/-order-views.py

Chapter Summary

Table 20.1: Order Management — Key Specifications

Property	**Value**
Order Statuses	7 (Pending, Paid, Processing, Shipped, Delivered, Completed, Cancelled)
Order Number	ORD-YYYY-XXXX (auto-generated)
Stock Deduction	Automatic on payment confirmation
Admin Actions	View, Update Status, Add Notes
Client Tracking	By order number (no account required)
Notifications	Status update emails
Analytics	Revenue stats, order counts by status
Audit Trail	All status changes logged

Part VI

Admin Dashboard

Chapter 21

Dashboard Architecture

21.1 Overview

The Caldan Construction dashboard system provides a comprehensive, role-based interface for the Platform Admin. The architecture is designed to be modular, maintainable, and consistent across all pages.

KEY INSIGHT Design Philosophy: Every dashboard page is built from the same base template. The sidebar, topbar, and layout are shared—only the content changes based on the page. This ensures consistency and reduces maintenance overhead.

21.2 Base Dashboard Layout

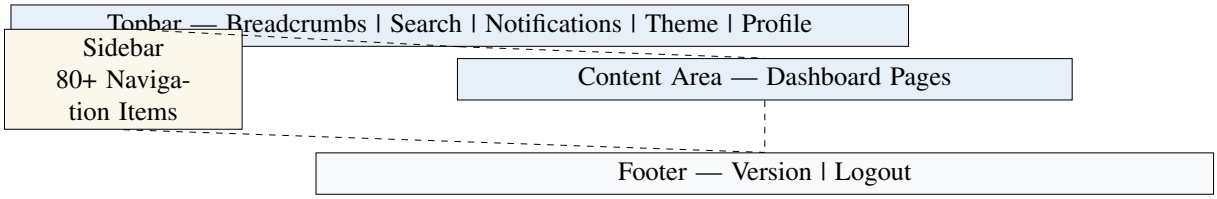


Figure 21.1: Base Dashboard Layout

21.3 Context Processors

Context processors inject global data into every dashboard template:

```
from dashboard.services.stats_service import StatsService
from notifications.services.notification_service import NotificationService

def dashboard_context(request):
    """Inject dashboard-wide context variables into all templates."""
    context = {}

    if request.user.is_authenticated:
        user = request.user

        # User info
        context['user_name'] = user.get_short_name()
        context['user_full_name'] = user.full_name
        context['user_email'] = user.email
        context['user_phone'] = user.phone

        # Admin ID
        context['admin_id'] = getattr(user, 'admin_id', 'ADMIN-001')

        # Theme
        profile = getattr(user, 'profile', None)
        if profile:
```

```
            context['theme'] = profile.theme
            context['user_theme_class'] = f"theme-{profile.theme}"

        # Notifications
        context['unread_notifications'] = NotificationService.get_unread_count(user)
        context['recent_notifications'] = NotificationService.get_recent(user, limit=5)

        # Stats
        context['stats'] = StatsService.get_admin_stats(user)

        # Permissions
        if profile and hasattr(profile, 'permissions'):
            context['admin_permissions'] = profile.permissions

    return context
```

Listing 21.1: Dashboard Context Processor – dashboard/context-processors.py

21.4 Base Dashboard Template

```
<!DOCTYPE html>
<html lang="en"
      data-theme="{% if user.profile.theme == 'device' %}{% else %}{{ user.profile.theme }}{% endif %}"
      {% if user.profile.theme == 'device' %}data-device-theme{% endif %}>
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Caldan Construction{% endblock %}</title>

    <!-- Theme CSS (inline) -->
    {% include "dashboard/partials/_css_inline/_admin_css.html" %}
    {% block extra_css %}{% endblock %}

    <!-- Font Awesome -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css">
</head>
<body>
    <!-- Sidebar -->
    {% include dashboard_sidebar_template %}

    <!-- Main Content -->
    <div class="main-content">
        <!-- Topbar -->
        {% include dashboard_topbar_template %}

        <!-- Content -->
        <div class="content-area">
            {% block dashboard_content %}{% endblock %}
        </div>
    </div>

    <!-- Toast Container -->
    <div id="toastContainer"></div>

    <!-- JavaScript (inline) -->
    {% include "dashboard/partials/_javascript_inline/_admin_js.html" %}
    {% block extra_js %}{% endblock %}
</body>
</html>
```

Listing 21.2: Base Dashboard Template – dashboard/templates/dashboard/base-dashboard.html

21.5 Sidebar Navigation

The sidebar contains 80+ navigation items organized into sections:

```
# Platform Admin Sidebar - 80+ Items
ADMIN_MENU = {
    'sections': [
        {
            'label': 'Overview',
            'icon': 'fa-th-large',
            'items': [
                {'label': 'Dashboard', 'icon': 'fa-tachometer-alt', 'url': 'dashboard:overview'},
                {'label': 'Analytics', 'icon': 'fa-chart-line', 'url': 'dashboard:analytics'},
                {'label': 'Activity Feed', 'icon': 'fa-history', 'url': 'dashboard:activity'},
                {'label': 'Notifications', 'icon': 'fa-bell', 'url': 'dashboard:notifications'},
            ]
        },
```

```
        {
            'label': 'Projects',
            'icon': 'fa-building',
            'items': [
                {'label': 'All Projects', 'icon': 'fa-building', 'url': 'projects:list'},
                {'label': 'Create Project', 'icon': 'fa-plus-circle', 'url': 'projects:create'},
                {'label': 'Project Progress', 'icon': 'fa-chart-bar', 'url': 'projects:progress'},
                {'label': 'Milestones', 'icon': 'fa-flag-checkered', 'url': 'projects:milestones'},
                {'label': 'Timeline', 'icon': 'fa-calendar-alt', 'url': 'projects:timeline'},
                {'label': 'Budget', 'icon': 'fa-money-bill-wave', 'url': 'projects:budget'},
            ]
        },
        {
            'label': 'Inventory',
            'icon': 'fa-boxes',
            'items': [
                {'label': 'Materials', 'icon': 'fa-boxes', 'url': 'inventory:materials'},
                {'label': 'Stock In', 'icon': 'fa-arrow-down', 'url': 'inventory:stock_in'},
                {'label': 'Stock Out', 'icon': 'fa-arrow-up', 'url': 'inventory:stock_out'},
                {'label': 'Alerts', 'icon': 'fa-exclamation-triangle', 'url': 'inventory:alerts'},
                {'label': 'Suppliers', 'icon': 'fa-truck', 'url': 'inventory:suppliers'},
                {'label': 'Deliveries', 'icon': 'fa-truck-loading', 'url': 'inventory:deliveries'},
            ]
        },
        {
            'label': 'Shop',
            'icon': 'fa-shopping-cart',
            'items': [
                {'label': 'Products', 'icon': 'fa-box', 'url': 'shop:products'},
                {'label': 'Orders', 'icon': 'fa-shopping-bag', 'url': 'shop_admin:orders'},
                {'label': 'Create Product', 'icon': 'fa-plus-circle', 'url': 'shop:product_create'},
                {'label': 'Order Reports', 'icon': 'fa-file-invoice', 'url': 'shop_admin:reports'},
            ]
        },
        {
            'label': 'Finances',
            'icon': 'fa-money-bill-wave',
            'items': [
                {'label': 'Payments', 'icon': 'fa-money-bill-wave', 'url': 'finances:payments'},
                {'label': 'Expenses', 'icon': 'fa-receipt', 'url': 'finances:expenses'},
                {'label': 'Budgets', 'icon': 'fa-chart-pie', 'url': 'finances:budgets'},
            ]
        },
        {
            'label': 'Clients',
            'icon': 'fa-users',
            'items': [
                {'label': 'All Clients', 'icon': 'fa-users', 'url': 'clients:list'},
                {'label': 'Create Client', 'icon': 'fa-user-plus', 'url': 'clients:create'},
                {'label': 'Leads', 'icon': 'fa-user-tag', 'url': 'clients:leads'},
            ]
        },
        {
            'label': 'Team',
            'icon': 'fa-user-tie',
            'items': [
                {'label': 'Team Members', 'icon': 'fa-user-tie', 'url': 'team:list'},
                {'label': 'Add Member', 'icon': 'fa-user-plus', 'url': 'team:create'},
            ]
        },
        {
            'label': 'Services',
            'icon': 'fa-tools',
            'items': [
                {'label': 'All Services', 'icon': 'fa-tools', 'url': 'services:list'},
                {'label': 'Add Service', 'icon': 'fa-plus-circle', 'url': 'services:create'},
                {'label': 'Categories', 'icon': 'fa-tags', 'url': 'services:categories'},
            ]
        },
        {
            'label': 'Reports',
            'icon': 'fa-chart-bar',
            'items': [
                {'label': 'Reports Dashboard', 'icon': 'fa-chart-bar', 'url': 'reports:index'},
                {'label': 'Project Report', 'icon': 'fa-file-alt', 'url': 'reports:project'},
                {'label': 'Inventory Report', 'icon': 'fa-boxes', 'url': 'reports:inventory'},
                {'label': 'Financial Report', 'icon': 'fa-file-invoice', 'url': 'reports:financial'},
                {'label': 'Export Center', 'icon': 'fa-file-export', 'url': 'reports:export'},
                {'label': 'Backup Manager', 'icon': 'fa-database', 'url': 'reports:backup'},
            ]
        },
        {
            'label': 'Settings',
            'icon': 'fa-cog',
            'items': [
                {'label': 'General', 'icon': 'fa-cog', 'url': 'settings:general'},
                {'label': 'Appearance', 'icon': 'fa-palette', 'url': 'settings:appearance'},
                {'label': 'Profile', 'icon': 'fa-user', 'url': 'settings:profile'},
                {'label': 'Security', 'icon': 'fa-shield-alt', 'url': 'settings:security'},
                {'label': 'Notifications', 'icon': 'fa-bell', 'url': 'settings:notifications'},
                {'label': 'Services', 'icon': 'fa-tools', 'url': 'settings:services'},
                {'label': 'Admins', 'icon': 'fa-users-cog', 'url': 'settings:admins'},
                {'label': 'Audit Log', 'icon': 'fa-history', 'url': 'settings:audit_log'},
                {'label': 'Data Management', 'icon': 'fa-database', 'url': 'settings:data_management'},
```

```
            ]
        },
    ]
}
```

Listing 21.3: Sidebar Menu Data – dashboard/data/sidebar-menu.py

21.6 Theme System

21.6.1 CSS Custom Properties

```
/* ======================================================================
   CALDAN CONSTRUCTION - THEME SYSTEM
   Light, Dark, and Device Default
   ====================================================================== */

/* LIGHT MODE (Default) */
:root, [data-theme="light"] {
    /* Backgrounds */
    --bg-primary: #FCF9F4;
    --bg-secondary: #F6F1E8;
    --bg-card: #FFFFFF;
    --bg-card-hover: #FAF8F5;
    --bg-sidebar: #0D2B36;
    --bg-sidebar-hover: #1A3A45;
    --bg-sidebar-active: #D69723;

    /* Text */
    --text-primary: #1F2937;
    --text-secondary: #6B7280;
    --text-tertiary: #9CA3AF;
    --text-inverse: #FFFFFF;
    --text-sidebar: #D1D5DB;
    --text-sidebar-active: #FFFFFF;

    /* Borders */
    --border-primary: #E5E7EB;
    --border-secondary: #D1D5DB;
    --border-light: #F0F1F3;

    /* Accents */
    --accent-primary: #0D2B36;
    --accent-secondary: #D69723;
    --accent-danger: #B9342B;
    --accent-success: #27AE60;
    --accent-warning: #F39C12;
    --accent-info: #2980B9;

    /* Shadows */
    --shadow-sm: 0 1px 2px rgba(0,0,0,0.05);
    --shadow-md: 0 4px 6px -1px rgba(0,0,0,0.07);
    --shadow-lg: 0 10px 15px -3px rgba(0,0,0,0.08);
    --shadow-xl: 0 20px 25px -5px rgba(0,0,0,0.1);

    /* Layout */
    --sidebar-width: 270px;
    --topbar-height: 70px;
    --content-max-width: 1400px;

    /* Radius */
    --radius-sm: 4px;
    --radius-md: 8px;
    --radius-lg: 12px;
    --radius-xl: 16px;
    --radius-full: 9999px;

    /* Transitions */
    --transition-fast: 150ms ease;
    --transition-base: 250ms ease;
    --transition-slow: 350ms ease;
}

/* DARK MODE */
[data-theme="dark"] {
    /* Backgrounds */
    --bg-primary: #0D1F2A;
    --bg-secondary: #1A3A3A;
    --bg-card: #1E3A3A;
    --bg-card-hover: #2A4A4A;
    --bg-sidebar: #0A1A1A;
    --bg-sidebar-hover: #1A3A3A;
    --bg-sidebar-active: #D69723;

    /* Text */
    --text-primary: #F1F5F9;
```

```
    --text-secondary: #94A3B8;
    --text-tertiary: #64748B;
    --text-inverse: #0F172A;
    --text-sidebar: #94A3B8;
    --text-sidebar-active: #FFFFFF;

    /* Borders */
    --border-primary: #334155;
    --border-secondary: #475569;
    --border-light: #273549;

    /* Accents */
    --accent-primary: #2A5A5A;
    --accent-secondary: #D69723;
    --accent-danger: #E74C3C;
    --accent-success: #2ECC71;
    --accent-warning: #F1C40F;
    --accent-info: #3498DB;

    /* Shadows */
    --shadow-sm: 0 1px 2px rgba(0,0,0,0.3);
    --shadow-md: 0 4px 6px -1px rgba(0,0,0,0.4);
    --shadow-lg: 0 10px 15px -3px rgba(0,0,0,0.5);
    --shadow-xl: 0 20px 25px -5px rgba(0,0,0,0.6);
}

/* DEVICE DEFAULT */
@media (prefers-color-scheme: dark) {
    [data-device-theme] {
        /* Copy dark mode variables */
        --bg-primary: #0D1F2A;
        --bg-secondary: #1A3A3A;
        --bg-card: #1E3A3A;
        /* ... etc */
    }
}
```

Listing 21.4: Theme System CSS – static/css/theme.css

Chapter Summary

Table 21.1: Dashboard Architecture — Key Specifications

Property	**Value**
Base Template	Single base with sidebar + topbar includes
Sidebar Roles	1 (Platform Admin with 80+ items)
Topbar	Role-aware with appropriate actions
Context Processor	Injects stats, notifications, theme into all templates
Theme System	Light/Dark/Device with CSS variables
Responsive	Mobile-first, collapses to drawer on small screens
CSS Architecture	CSS custom properties (no hardcoded colors)
JS Architecture	Inline partials, no external dashboard JS files

Chapter 22

Dashboard Pages & Navigation

22.1 Overview

The Platform Manager Dashboard is the most comprehensive view in Caldan Construction. It provides full visibility and control over all projects, inventory, shop, finances, clients, team, services, reports, and system settings.

> **KEY INSIGHT** Core Principle: The Platform Admin sees everything, controls everything, and manages everything. No other user has this level of access.

22.2 Complete Page Inventory

The dashboard contains 80+ pages organized into 10 sections:

Table 22.1: Complete Dashboard Page Inventory

Section	Page Name	URL	Icon
Overview	Dashboard Home	`/dashboard/`	fa-tachometer-alt
Overview	Analytics	`/dashboard/analytics/`	fa-chart-line
Overview	Activity Feed	`/dashboard/activity/`	fa-history
Overview	Notifications	`/dashboard/notifications/`	fa-bell
Projects	All Projects	`/dashboard/projects/`	fa-building
Projects	Create Project	`/dashboard/projects/create/`	fa-plus-circle
Projects	Project Detail	`/dashboard/projects/<id>/`	fa-info-circle
Projects	Edit Project	`/dashboard/projects/<id>/edit/`	fa-edit
Projects	Project Progress	`/dashboard/projects/<id>/progress/`	fa-chart-bar
Projects	Milestones	`/dashboard/projects/<id>/milestones/`	fa-flag-checkered
Projects	Timeline	`/dashboard/projects/<id>/timeline/`	fa-calendar-alt
Projects	Project Budget	`/dashboard/projects/<id>/budget/`	fa-money-bill-wave
Inventory	All Materials	`/dashboard/inventory/materials/`	fa-boxes
Inventory	Create Material	`/dashboard/inventory/materials/create/`	fa-plus-circle
Inventory	Edit Material	`/dashboard/inventory/materials/<id>/edit/`	fa-edit
Inventory	Material Detail	`/dashboard/inventory/materials/<id>/`	fa-info-circle
Inventory	Stock In	`/dashboard/inventory/stock-in/`	fa-arrow-down
Inventory	Stock Out	`/dashboard/inventory/stock-out/`	fa-arrow-up
Inventory	Stock Transactions	`/dashboard/inventory/transactions/`	fa-exchange-alt
Inventory	Low Stock Alerts	`/dashboard/inventory/alerts/`	fa-exclamation-triangle
Inventory	Suppliers	`/dashboard/inventory/suppliers/`	fa-truck
Inventory	Create Supplier	`/dashboard/inventory/suppliers/create/`	fa-plus-circle
Inventory	Supplier Detail	`/dashboard/inventory/suppliers/<id>/`	fa-info-circle
Inventory	Deliveries	`/dashboard/inventory/deliveries/`	fa-truck-loading
Shop	Products	`/dashboard/shop/products/`	fa-box
Shop	Create Product	`/dashboard/shop/products/create/`	fa-plus-circle
Shop	Edit Product	`/dashboard/shop/products/<id>/edit/`	fa-edit
Shop	Product Detail	`/dashboard/shop/products/<id>/`	fa-info-circle
Shop	All Orders	`/dashboard/shop/orders/`	fa-shopping-bag
Shop	Order Detail	`/dashboard/shop/orders/<id>/`	fa-file-invoice
Shop	Update Order Status	`/dashboard/shop/orders/<id>/update/`	fa-sync
Shop	Order Reports	`/dashboard/shop/reports/`	fa-chart-bar
Finances	All Payments	`/dashboard/finances/payments/`	fa-money-bill-wave
Finances	Create Payment	`/dashboard/finances/payments/create/`	fa-plus-circle
Finances	Payment Detail	`/dashboard/finances/payments/<id>/`	fa-info-circle
Finances	All Expenses	`/dashboard/finances/expenses/`	fa-receipt
Finances	Create Expense	`/dashboard/finances/expenses/create/`	fa-plus-circle
Finances	Budgets	`/dashboard/finances/budgets/`	fa-chart-pie
Clients	All Clients	`/dashboard/clients/`	fa-users
Clients	Create Client	`/dashboard/clients/create/`	fa-user-plus
Clients	Client Detail	`/dashboard/clients/<id>/`	fa-info-circle
Clients	Edit Client	`/dashboard/clients/<id>/edit/`	fa-edit
Clients	Leads	`/dashboard/clients/leads/`	fa-user-tag
Team	Team Members	`/dashboard/team/`	fa-user-tie
Team	Add Member	`/dashboard/team/create/`	fa-user-plus
Team	Edit Member	`/dashboard/team/<id>/edit/`	fa-edit
Team	Member Detail	`/dashboard/team/<id>/`	fa-info-circle
Services	All Services	`/dashboard/services/`	fa-tools

22.3 Dashboard Overview Page

```
{% extends "dashboard/base_dashboard.html" %}
{% load static %}

{% block title %}Dashboard --- Caldan Construction{% endblock %}

{% block dashboard_content %}
<div class="dashboard-page">
    <!-- Welcome Banner -->
    <div class="welcome-banner">
        <div class="welcome-text">
            <h1>Welcome back, {{ user_name }}!</h1>
            <p class="subtitle">Here's what's happening across your construction operations</p>
        </div>
        <div class="welcome-date">
            <i class="fas fa-calendar-alt"></i>
            <span>{{ current_date|date:"l, F j, Y" }}</span>
        </div>
    </div>

    <!-- Stats Cards -->
    <div class="stats-grid">
        <div class="stat-card">
            <div class="stat-icon projects">
                <i class="fas fa-building"></i>
            </div>
            <div class="stat-info">
                <div class="stat-value">{{ stats.total_projects }}</div>
                <div class="stat-label">Active Projects</div>
            </div>
            <div class="stat-change positive">
                <i class="fas fa-arrow-up"></i> {{ stats.projects_change }}%
            </div>
        </div>

        <div class="stat-card">
            <div class="stat-icon inventory">
                <i class="fas fa-boxes"></i>
            </div>
            <div class="stat-info">
                <div class="stat-value">{{ stats.low_stock_alerts }}</div>
                <div class="stat-label">Low Stock Alerts</div>
            </div>
            <div class="stat-change negative">
                <i class="fas fa-arrow-up"></i> {{ stats.alerts_change }}%
            </div>
        </div>

        <div class="stat-card">
            <div class="stat-icon orders">
                <i class="fas fa-shopping-bag"></i>
            </div>
            <div class="stat-info">
                <div class="stat-value">{{ stats.pending_orders }}</div>
                <div class="stat-label">Pending Orders</div>
            </div>
            <div class="stat-change positive">
                <i class="fas fa-arrow-up"></i> {{ stats.orders_change }}%
            </div>
        </div>

        <div class="stat-card">
            <div class="stat-icon payments">
                <i class="fas fa-money-bill-wave"></i>
            </div>
            <div class="stat-info">
                <div class="stat-value">KES {{ stats.monthly_revenue|floatformat:0|intcomma }}</div>
                <div class="stat-label">Monthly Revenue</div>
            </div>
            <div class="stat-change positive">
                <i class="fas fa-arrow-up"></i> {{ stats.revenue_change }}%
            </div>
        </div>
    </div>

    <!-- Charts Row -->
    <div class="charts-row">
        <div class="card chart-card">
            <div class="card-header">
                <h3><i class="fas fa-chart-bar"></i> Revenue Trend</h3>
            </div>
            <div class="card-body">
                <canvas id="revenueChart" height="200"></canvas>
            </div>
        </div>

        <div class="card chart-card">
            <div class="card-header">
                <h3><i class="fas fa-chart-pie"></i> Order Status</h3>
            </div>
```

```
            <div class="card-body">
                <canvas id="orderChart" height="200"></canvas>
            </div>
        </div>
    </div>

    <!-- Quick Actions -->
    <div class="quick-actions">
        <a href="{% url 'projects:create' %}" class="action-card">
            <i class="fas fa-plus-circle"></i>
            <span>New Project</span>
        </a>
        <a href="{% url 'inventory:stock_in' %}" class="action-card">
            <i class="fas fa-arrow-down"></i>
            <span>Stock In</span>
        </a>
        <a href="{% url 'shop:product_create' %}" class="action-card">
            <i class="fas fa-box"></i>
            <span>Add Product</span>
        </a>
        <a href="{% url 'reports:export' %}" class="action-card">
            <i class="fas fa-file-export"></i>
            <span>Export Data</span>
        </a>
    </div>

    <!-- Activity Feed -->
    <div class="card activity-card">
        <div class="card-header">
            <h3><i class="fas fa-history"></i> Recent Activity</h3>
            <a href="{% url 'dashboard:activity' %}" class="btn-link">View All</a>
        </div>
        <div class="card-body">
            <div class="activity-feed">
                {% for activity in recent_activities %}
                <div class="activity-item activity-{{ activity.type|lower }}">
                    <div class="activity-icon">
                        <i class="fas {{ activity.icon }}"></i>
                    </div>
                    <div class="activity-content">
                        <p class="activity-title">{{ activity.title }}</p>
                        <p class="activity-description">{{ activity.description }}</p>
                        <span class="activity-time">{{ activity.created_at|timesince }} ago</span>
                    </div>
                </div>
                {% empty %}
                <div class="empty-state">
                    <i class="fas fa-inbox"></i>
                    <p>No recent activity</p>
                </div>
                {% endfor %}
            </div>
        </div>
    </div>
</div>

<script>
    // Revenue Chart
    const ctx1 = document.getElementById('revenueChart').getContext('2d');
    new Chart(ctx1, {
        type: 'line',
        data: {
            labels: {{ chart_labels|safe }},
            datasets: [{
                label: 'Revenue (KES)',
                data: {{ chart_data|safe }},
                borderColor: '#D69723',
                backgroundColor: 'rgba(214, 151, 35, 0.1)',
                fill: true,
                tension: 0.4
            }]
        },
        options: {
            responsive: true,
            maintainAspectRatio: false,
            plugins: {
                legend: { display: false }
            }
        }
    });

    // Order Status Chart
    const ctx2 = document.getElementById('orderChart').getContext('2d');
    new Chart(ctx2, {
        type: 'doughnut',
        data: {
            labels: {{ order_labels|safe }},
            datasets: [{
                data: {{ order_data|safe }},
                backgroundColor: ['#D69723', '#0D2B36', '#27AE60', '#F39C12', '#B9342B']
            }]
        },
        options: {
            responsive: true,
```

```
            maintainAspectRatio: false,
            plugins: {
                legend: {
                    position: 'bottom'
                }
            }
        }
    });
</script>
{% endblock %}
```

Listing 22.1: Dashboard Overview Template

Chapter Summary

Table 22.2: Dashboard Pages & Navigation — Key Specifications

Property	**Value**
Total Pages	80+
Sections	10
Sidebar Items	80+
Stats Cards	4 (Projects, Inventory, Orders, Revenue)
Charts	Revenue Trend, Order Status
Quick Actions	4 (New Project, Stock In, Add Product, Export)
Activity Feed	Real-time with pagination

Chapter 23

Data Visualization & Analytics

23.1 Overview

The Analytics module provides data visualization and business intelligence capabilities. It transforms raw operational data into actionable insights through charts, graphs, and interactive dashboards.

KEY INSIGHT Core Principle: Data is only valuable if it can be understood. The analytics module turns complex data into clear, visual insights that drive better decisions.

23.2 Analytics Dashboard

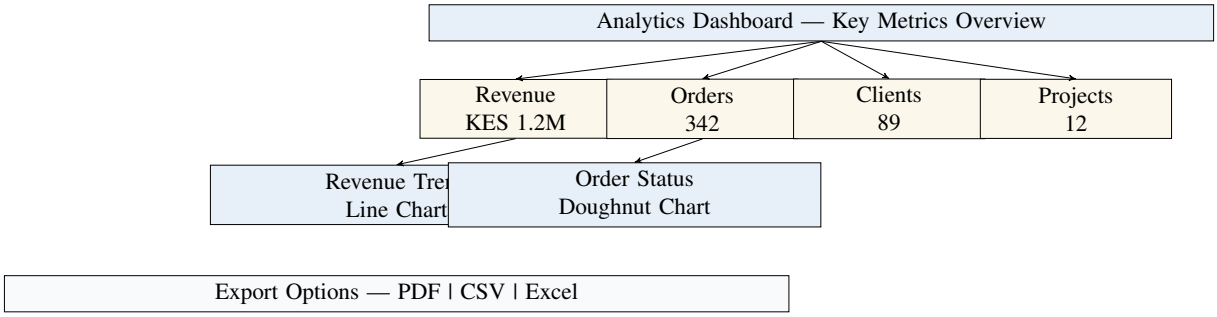


Figure 23.1: Analytics Dashboard Layout

23.3 Analytics Service

```
from django.db.models import Sum, Count, Avg
from django.utils import timezone
from datetime import timedelta, date
from decimal import Decimal

class AnalyticsService:
    """Service for data analytics and visualization."""

    @classmethod
    def get_dashboard_metrics(cls):
        """Get key metrics for analytics dashboard."""
        from projects.models import Project
        from shop.models import Order
        from finances.models import Payment
        from clients.models import Client

        # Revenue metrics
        month_start = timezone.now().replace(day=1)
        monthly_revenue = Payment.objects.filter(
            status='COMPLETED',
```

```
            payment_date__gte=month_start
        ).aggregate(total=Sum('amount'))['total'] or 0

        # Order metrics
        total_orders = Order.objects.count()
        pending_orders = Order.objects.filter(status='PENDING').count()
        completed_orders = Order.objects.filter(status='COMPLETED').count()

        # Client metrics
        total_clients = Client.objects.count()
        active_clients = Client.objects.filter(status='ACTIVE').count()
        new_clients_this_month = Client.objects.filter(
            created_at__gte=month_start
        ).count()

        # Project metrics
        total_projects = Project.objects.filter(is_archived=False).count()
        active_projects = Project.objects.filter(status='ACTIVE').count()
        completed_projects = Project.objects.filter(status='COMPLETED').count()

        return {
            'monthly_revenue': float(monthly_revenue),
            'total_orders': total_orders,
            'pending_orders': pending_orders,
            'completed_orders': completed_orders,
            'total_clients': total_clients,
            'active_clients': active_clients,
            'new_clients_this_month': new_clients_this_month,
            'total_projects': total_projects,
            'active_projects': active_projects,
            'completed_projects': completed_projects,
        }

    @classmethod
    def get_revenue_trend(cls, months=12):
        """Get monthly revenue trend for chart."""
        data = []
        today = timezone.now().date()

        for i in range(months):
            month_end = today.replace(day=1) - timedelta(days=1)
            month_start = month_end.replace(day=1)

            revenue = Payment.objects.filter(
                status='COMPLETED',
                payment_date__gte=month_start,
                payment_date__lte=month_end
            ).aggregate(total=Sum('amount'))['total'] or 0

            data.append({
                'month': month_start.strftime('%b %Y'),
                'revenue': float(revenue),
                'orders': Order.objects.filter(
                    created_at__date__gte=month_start,
                    created_at__date__lte=month_end
                ).count()
            })

            today = month_start - timedelta(days=1)

        return data[::-1]

    @classmethod
    def get_order_status_distribution(cls):
        """Get order status distribution for pie chart."""
        from shop.models import Order
        status_counts = {}
        for status in Order.Status.values:
            count = Order.objects.filter(status=status).count()
            if count > 0:
                status_counts[status] = count
        return status_counts
```

Listing 23.1: Analytics Service – dashboard/services/analytics-service.py

23.4 Analytics Views

```
from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from accounts.permissions.decorators import admin_required
from dashboard.services.analytics_service import AnalyticsService

@login_required
@admin_required
def analytics_dashboard(request):
    """Analytics dashboard view."""
```

```
    metrics = AnalyticsService.get_dashboard_metrics()
    revenue_trend = AnalyticsService.get_revenue_trend(months=12)
    order_status = AnalyticsService.get_order_status_distribution()

    context = {
        'metrics': metrics,
        'revenue_trend': revenue_trend,
        'order_status': order_status,
        'chart_labels': [d['month'] for d in revenue_trend],
        'chart_data': [d['revenue'] for d in revenue_trend],
        'order_labels': list(order_status.keys()),
        'order_data': list(order_status.values()),
        'page_title': 'Analytics Dashboard',
    }
    return render(request, 'dashboard/admin/analytics/index.html', context)
```

Listing 23.2: Analytics Views – dashboard/views/-analytics-views.py

23.5 Export Options

Every analytics dashboard includes export functionality:

Table 23.1: Export Options

Format	Use Case	Technology
PDF	Reports, presentations	Playwright
CSV	Spreadsheets, data analysis	Python csv module
Excel	Advanced data analysis	openpyxl
ZIP	Batch downloads	Python zipfile

Chapter Summary

Table 23.2: Data Visualization & Analytics — Key Specifications

Property	Value
Metrics Displayed	Revenue, Orders, Clients, Projects
Chart Types	Line, Doughnut, Bar
Time Periods	12 months rolling
Export Formats	PDF, CSV, Excel, ZIP
Real-time Data	Yes (on page load)
Responsive	Yes (mobile-friendly charts)

Part VII

Notification System

Chapter 24

Notification System Overview

24.1 Overview

The Caldan Construction notification system is a multi-channel communication engine that delivers timely, relevant information to users. It spans three delivery channels — SMS, Email, and In-App notifications — with automated triggers, configurable preferences, and full audit logging.

KEY INSIGHT Core Principle: Notifications are the connective tissue of the platform. Stock alerts save money, order confirmations build trust, and audit reports ensure accountability.

24.2 Three-Channel Architecture

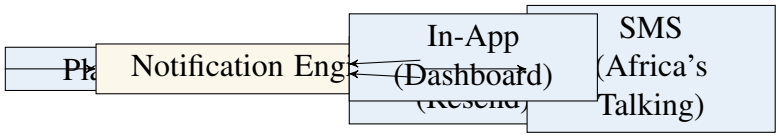


Figure 24.1: Three-Channel Notification Architecture

24.3 Notification Model

```
from django.db import models
from django.conf import settings
from django.utils import timezone

class Notification(models.Model):
    """In-app notification displayed in the dashboard."""

    class Priority(models.TextChoices):
        LOW = 'LOW', 'Low'
        MEDIUM = 'MEDIUM', 'Medium'
        HIGH = 'HIGH', 'High'
        URGENT = 'URGENT', 'Urgent'

    class Type(models.TextChoices):
        STOCK_ALERT = 'STOCK_ALERT', 'Stock Alert'
        ORDER_CONFIRMED = 'ORDER_CONFIRMED', 'Order Confirmed'
        ORDER_STATUS = 'ORDER_STATUS', 'Order Status Update'
        PROJECT_UPDATE = 'PROJECT_UPDATE', 'Project Update'
        PAYMENT_RECEIVED = 'PAYMENT_RECEIVED', 'Payment Received'
        SYSTEM = 'SYSTEM', 'System'
        REMINDER = 'REMINDER', 'Reminder'
        AUDIT = 'AUDIT', 'Audit Report'

    # Recipient
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
```

```
        related_name='notifications'
    )

    # Content
    title = models.CharField(max_length=200)
    message = models.TextField()
    type = models.CharField(max_length=20, choices=Type.choices, db_index=True)

    # Priority
    priority = models.CharField(max_length=10, choices=Priority.choices, default=Priority.MEDIUM)

    # Link (click to navigate)
    action_url = models.CharField(max_length=500, blank=True, default='')

    # Icon (Font Awesome)
    icon = models.CharField(max_length=50, default='fa-bell')

    # Read status
    is_read = models.BooleanField(default=False, db_index=True)
    read_at = models.DateTimeField(null=True, blank=True)

    # Metadata
    metadata = models.JSONField(default=dict, blank=True)

    # Timestamps
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'notifications'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['user', 'is_read', 'created_at']),
            models.Index(fields=['type', 'created_at']),
            models.Index(fields=['priority']),
        ]

    def __str__(self):
        return f"[{self.get_priority_display()}] {self.title} --- {self.user.email}"

    def mark_as_read(self):
        """Mark notification as read."""
        if not self.is_read:
            self.is_read = True
            self.read_at = timezone.now()
            self.save(update_fields=['is_read', 'read_at'])

    def mark_as_unread(self):
        """Mark notification as unread."""
        self.is_read = False
        self.read_at = None
        self.save(update_fields=['is_read', 'read_at'])
```

Listing 24.1: In-App Notification Model – notifications/models/notification.py

24.4 Notification Service

```
import logging
from django.utils import timezone
from notifications.models.notification import Notification

logger = logging.getLogger('caldan.notifications')

class NotificationService:
    """Service for creating and managing in-app notifications."""

    @classmethod
    def create(cls, user, title, message, notification_type='SYSTEM',
               priority='MEDIUM', icon='fa-bell', action_url='#', metadata=None):
        """Create an in-app notification for a user."""
        try:
            notification = Notification.objects.create(
                user=user,
                title=title,
                message=message,
                type=notification_type,
                priority=priority,
                icon=icon,
                action_url=action_url,
                is_read=False,
                metadata=metadata or {}
            )
            logger.info(f"Notification created for {user.email}: {title}")
            return notification
        except Exception as e:
            logger.error(f"Failed to create notification for {user.email}: {str(e)}")
```

```
            return None

    @classmethod
    def create_bulk(cls, users, title, message, notification_type='SYSTEM',
                    priority='MEDIUM', icon='fa-bell', action_url='#'):
        """Create notifications for multiple users at once."""
        notifications = []
        for user in users:
            notifications.append(
                Notification(
                    user=user,
                    title=title,
                    message=message,
                    type=notification_type,
                    priority=priority,
                    icon=icon,
                    action_url=action_url,
                    is_read=False
                )
            )

        created = Notification.objects.bulk_create(notifications)
        logger.info(f"Bulk created {len(created)} notifications")
        return len(created)

    @classmethod
    def mark_as_read(cls, notification_id, user):
        """Mark a notification as read."""
        try:
            notification = Notification.objects.get(id=notification_id, user=user)
            notification.is_read = True
            notification.read_at = timezone.now()
            notification.save(update_fields=['is_read', 'read_at'])
            return True
        except Notification.DoesNotExist:
            logger.warning(f"Notification {notification_id} not found for user {user.email}")
            return False

    @classmethod
    def mark_all_as_read(cls, user):
        """Mark all unread notifications as read for a user."""
        count = Notification.objects.filter(user=user, is_read=False).update(
            is_read=True,
            read_at=timezone.now()
        )
        logger.info(f"Marked {count} notifications as read for {user.email}")
        return count

    @classmethod
    def get_unread_count(cls, user):
        """Get the number of unread notifications for a user."""
        return Notification.objects.filter(user=user, is_read=False).count()

    @classmethod
    def get_recent(cls, user, limit=10):
        """Get recent notifications for a user."""
        return Notification.objects.filter(user=user).order_by('-created_at')[:limit]
```

Listing 24.2: Notification Service – notifications/services/notification-service.py

24.5 Notification Types

Table 24.1: Complete Notification Type Inventory

Type	Trigger	Channels	Priority
STOCK_ALERT	Material below threshold	In-App, Email, SMS	URGENT
ORDER_CONFIRMED	Order placed	In-App, Email	MEDIUM
ORDER_STATUS	Order status updated	In-App, Email	MEDIUM
PROJECT_UPDATE	Milestone updated	In-App, Email	MEDIUM
PAYMENT_RECEIVED	Payment confirmed	In-App, Email	LOW
SYSTEM	System notification	In-App, Email	HIGH
REMINDER	Scheduled reminder	In-App, Email	LOW
AUDIT	Bi-weekly audit report	Email	HIGH

Chapter Summary

Table 24.2: Notification System — Key Specifications

Property	Value
Channels	3: In-App, Email (Resend), SMS (Africa's Talking + Celcom)
Priority Levels	4: LOW, MEDIUM, HIGH, URGENT
Notification Types	8
In-App Delivery	Bell icon with badge count, mark as read
Email Provider	Resend (3,000/month free)
SMS Provider	Africa's Talking (primary) + Celcom (fallback)
Audit Trail	All notifications logged with status

Chapter 25

Email System (Resend)

25.1 Overview

The Caldan Construction email system uses Resend as the transactional email provider. This chapter documents the complete implementation including setup, configuration, and all email templates.

25.2 Resend Setup

25.2.1 Environment Configuration

```
# Resend Email Configuration
RESEND_API_KEY=re_cTuFEymB_EFDyucJ5J4YqkhKBaWb5W3Ra
SEND_REAL_EMAILS=True
```

Listing 25.1: .env Configuration for Resend

25.2.2 Django Settings

```
# Resend Configuration
RESEND_API_KEY = config('RESEND_API_KEY', default='')
SEND_REAL_EMAILS = config('SEND_REAL_EMAILS', default=False, cast=bool)

# Email settings for production
if SEND_REAL_EMAILS and RESEND_API_KEY:
    EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
    EMAIL_HOST = 'smtp.resend.com'
    EMAIL_PORT = 587
    EMAIL_USE_TLS = True
    EMAIL_HOST_USER = 'resend'
    EMAIL_HOST_PASSWORD = RESEND_API_KEY
    DEFAULT_FROM_EMAIL = 'Caldan Construction <noreply@caldanconstruction.com>'
else:
    # Development mode - emails print to console
    EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Listing 25.2: settings.py – Email Configuration

25.3 Email Service

```
import resend
import logging
from django.conf import settings
from django.template.loader import render_to_string
from django.utils import timezone
from notifications.models.email_log import EmailLog

logger = logging.getLogger('caldan.email')

# Initialize Resend
resend.api_key = settings.RESEND_API_KEY
```

```
class EmailService:
    """Handles all transactional email sending via Resend."""

    FROM_EMAIL = "Caldan Construction <noreply@caldanconstruction.com>"
    FROM_NAME = "Caldan Construction"

    @classmethod
    def send(cls, to_email, subject, html_content, to_name='', user=None, email_type='GENERAL'):
        """Send a transactional email."""
        # Check if real email sending is enabled
        if not getattr(settings, 'SEND_REAL_EMAILS', False):
            logger.info(f"[DEV MODE] Email would be sent to {to_email}: {subject}")
            cls._log_attempt(to_email, subject, 'DEV_MODE', user=user, email_type=email_type)
            return True

        try:
            params = {
                "from": cls.FROM_EMAIL,
                "to": [to_email],
                "subject": subject,
                "html": html_content,
            }

            response = resend.Emails.send(params)
            email_id = response.get('id', '')

            logger.info(f"Email sent to {to_email}: {email_id}")
            cls._log_attempt(to_email, subject, 'SENT', email_id=email_id, user=user, email_type=email_type)
            return True

        except Exception as e:
            logger.error(f"Email failed to {to_email}: {str(e)}")
            cls._log_attempt(to_email, subject, 'FAILED', error_message=str(e), user=user, email_type=email_type)
            return False

    @classmethod
    def send_template(cls, to_email, subject, template_name, context, to_name='', user=None, email_type='GENERAL'):
        """Send an email using a Django template."""
        # Add common context
        context.update({
            'brand_name': 'Caldan Construction',
            'tagline': 'Building Kenya\'s Future, One Project at a Time.',
            'current_year': timezone.now().year,
            'support_email': 'sir.mukoyakhisa@gmail.com',
        })

        html_content = render_to_string(template_name, context)
        return cls.send(to_email, subject, html_content, to_name, user, email_type)

    @classmethod
    def _log_attempt(cls, to_email, subject, status, email_id='', error_message='', user=None, email_type='GENERAL'):
        """Log email attempt to database."""
        try:
            EmailLog.objects.create(
                to_email=to_email,
                subject=subject[:200],
                status=status,
                email_id=email_id,
                error_message=error_message,
                user=user,
                email_type=email_type
            )
        except Exception as e:
            logger.error(f"Failed to log email attempt: {str(e)}")

    # --- Specific email methods ---

    @classmethod
    def send_otp(cls, user, otp_code, device_info='', ip_address=''):
        """Send OTP verification email."""
        return cls.send_template(
            to_email=user.email,
            subject="Your Caldan Construction Verification Code",
            template_name='email/otp_email.html',
            context={
                'user': user,
                'otp_code': otp_code,
                'device_info': device_info,
                'ip_address': ip_address,
                'expiry_minutes': 5,
            },
            user=user,
            email_type='OTP'
        )

    @classmethod
    def send_order_confirmation(cls, order):
        """Send order confirmation email."""
        return cls.send_template(
            to_email=order.client_email,
            subject=f"Order Confirmation --- {order.order_number}",
            template_name='email/order_confirmation.html',
            context={
```

```
                'order': order,
                'client_name': order.client_name,
            },
            email_type='ORDER_CONFIRMATION'
        )

    @classmethod
    def send_stock_alert(cls, material):
        """Send low stock alert email."""
        return cls.send_template(
            to_email=settings.ADMIN_EMAIL,
            subject=f"[WARNING] LOW STOCK ALERT: {material.name}",
            template_name='email/stock_alert.html',
            context={
                'material': material,
                'current_quantity': material.current_quantity,
                'minimum_threshold': material.minimum_threshold,
            },
            email_type='STOCK_ALERT'
        )

    @classmethod
    def send_admin_contact_alert(cls, name, email, phone, message):
        """Send admin alert for contact form submission."""
        return cls.send_template(
            to_email=settings.ADMIN_EMAIL,
            subject=f"New Contact Form Submission --- {name}",
            template_name='email/admin_contact_alert.html',
            context={
                'name': name,
                'email': email,
                'phone': phone,
                'message': message,
            },
            email_type='ADMIN_ALERT'
        )

    @classmethod
    def send_admin_quote_alert(cls, name, email, phone, project_type, budget, details):
        """Send admin alert for quote request."""
        return cls.send_template(
            to_email=settings.ADMIN_EMAIL,
            subject=f"New Quote Request --- {name}",
            template_name='email/admin_quote_alert.html',
            context={
                'name': name,
                'email': email,
                'phone': phone,
                'project_type': project_type,
                'budget': budget,
                'details': details,
            },
            email_type='ADMIN_ALERT'
        )
```

Listing 25.3: Email Service – notifications/services/email-service.py

25.4 Email Log Model

```
from django.db import models
from django.conf import settings

class EmailLog(models.Model):
    """Audit log for all emails sent through the platform."""

    class Status(models.TextChoices):
        SENT = 'SENT', 'Successfully Sent'
        FAILED = 'FAILED', 'Failed to Send'
        BOUNCED = 'BOUNCED', 'Bounced'
        DEV_MODE = 'DEV_MODE', 'Development Mode (Not Sent)'

    # Recipient
    to_email = models.EmailField()
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='email_logs',
    )

    # Content
    subject = models.CharField(max_length=200)
    email_type = models.CharField(max_length=50, db_index=True)

    # Delivery
```

```
    status = models.CharField(max_length=10, choices=Status.choices, default='SENT', db_index=True)
    email_id = models.CharField(max_length=100, blank=True, default='')
    error_message = models.TextField(blank=True, default='')

    # Timestamps
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        db_table = 'email_logs'
        verbose_name = 'Email Log'
        verbose_name_plural = 'Email Logs'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['status', 'created_at']),
            models.Index(fields=['user', 'created_at']),
            models.Index(fields=['email_type', 'created_at']),
        ]

    def __str__(self):
        return f"Email to {self.to_email} --- {self.subject[:50]}"
```

Listing 25.4: Email Log Model – notifications/models/email-log.py

25.5 Email Templates Inventory

Table 25.1: Email Template Inventory

Template	Purpose	Key Variables
otp_email.html	8-digit OTP verification	user, otp_code, expiry_minutes
welcome_admin.html	New admin welcome	user, temp_password, dashboard_url
admin_invitation.html	Admin invitation	invitation_link, expires_at
order_confirmation.html	Order confirmation	order, client_name, order_number
order_status_update.html	Order status update	order, new_status, tracking_number
stock_alert.html	Low stock alert	material, current_quantity, threshold
payment_receipt.html	Payment receipt	payment, amount, receipt_number
weekly_report.html	Weekly report	week_start, week_end, stats
bi_weekly_audit.html	Bi-weekly audit report	period, actions, stats
admin_contact_alert.html	Contact form alert	name, email, phone, message
admin_quote_alert.html	Quote request alert	name, project_type, budget, details

Chapter Summary

Table 25.2: Email System — Key Specifications

Property	Value
Email Provider	Resend
Free Tier	3,000 emails/month
Total Email Templates	11+
Email Types	Authentication, Orders, Stock, Reports, Admin Alerts
Development Mode	Console backend with SEND_REAL_EMAILS flag
Audit Trail	Complete EmailLog model with status tracking

Chapter 26

SMS System (Africa's Talking + Celcom)

26.1 Overview

The Caldan Construction SMS system provides reliable, multi-provider SMS delivery for critical notifications. Africa's Talking serves as the primary provider with Celcom Africa as a fallback.

26.2 Provider Strategy

Table 26.1: SMS Provider Strategy

Provider	Role	When Used
Africa's Talking	Primary	Development, sandbox testing, launch, normal transactional SMS
Celcom Africa	Fallback	Higher-volume operations, negotiated support, redundancy

26.3 SMS Service

```
import requests
import logging
from django.conf import settings
from notifications.models.sms_log import SMSLog

logger = logging.getLogger('caldan.sms')

class SMSService:
    """Handles SMS sending via Africa's Talking with Celcom fallback."""

    @classmethod
    def send(cls, phone_number, message, user=None, notification_type='GENERAL'):
        """Send SMS with automatic provider failover."""
        # Normalize phone number
        phone = cls._normalize_phone(phone_number)

        # Check if real SMS is enabled
        if not getattr(settings, 'SEND_REAL_SMS', False):
            logger.info(f"[DEV MODE] SMS would be sent to {phone}: {message[:50]}...")
            cls._log_attempt(phone, message, 'DEV_MODE', user=user, notification_type=notification_type)
            return True

        # Try primary provider
        provider = getattr(settings, 'SMS_PROVIDER', 'africastalking')

        if provider == 'celcom':
            result = cls._send_celcom(phone, message)
        else:
            result = cls._send_africastalking(phone, message)
```

```
        # Log attempt
        cls._log_attempt(
            phone, message,
            'SENT' if result['success'] else 'FAILED',
            provider=provider,
            provider_response=result.get('response', ''),
            user=user,
            notification_type=notification_type
        )

        # If failed, try fallback
        if not result['success'] and provider == 'africastalking':
            logger.warning(f"Africa's Talking failed, trying Celcom fallback")
            result = cls._send_celcom(phone, message)
            cls._log_attempt(
                phone, message,
                'SENT' if result['success'] else 'FAILED',
                provider='celcom',
                provider_response=result.get('response', ''),
                user=user,
                notification_type=notification_type
            )

        return result['success']

    @classmethod
    def _send_africastalking(cls, phone, message):
        """Send SMS via Africa's Talking."""
        try:
            url = "https://api.africastalking.com/version1/message"
            headers = {
                "apikey": settings.AFRICASTALKING_API_KEY,
                "Content-Type": "application/x-www-form-urlencoded",
                "Accept": "application/json",
            }
            data = {
                "username": settings.AFRICASTALKING_USERNAME,
                "to": phone,
                "message": message,
                "from": "CALDAN",
            }

            response = requests.post(url, headers=headers, data=data, timeout=30)
            result = response.json()

            if response.status_code == 200 and result.get('status') == 'success':
                return {'success': True, 'response': result}
            else:
                return {'success': False, 'response': result}

        except Exception as e:
            return {'success': False, 'response': str(e)}

    @classmethod
    def _send_celcom(cls, phone, message):
        """Send SMS via Celcom Africa."""
        try:
            url = settings.CELCOM_API_URL
            headers = {"Content-Type": "application/json"}
            data = {
                "api_key": settings.CELCOM_API_KEY,
                "partner_id": settings.CELCOM_PARTNER_ID,
                "shortcode": settings.CELCOM_SHORTCODE,
                "mobile": phone,
                "message": message,
            }

            response = requests.post(url, json=data, headers=headers, timeout=30)
            result = response.json()

            if response.status_code == 200 and result.get('status') == 'success':
                return {'success': True, 'response': result}
            else:
                return {'success': False, 'response': result}

        except Exception as e:
            return {'success': False, 'response': str(e)}

    @classmethod
    def _normalize_phone(cls, phone):
        """Normalize Kenyan phone number to international format."""
        phone = phone.strip().replace(' ', '').replace('-', '').replace('(', '').replace(')', '')

        if phone.startswith('07') or phone.startswith('01'):
            return '+254' + phone[1:]
        elif phone.startswith('7') or phone.startswith('1'):
            return '+254' + phone
        elif phone.startswith('254'):
            return '+' + phone
        elif phone.startswith('+254'):
            return phone
        else:
            return phone
```

```
    @classmethod
    def _log_attempt(cls, phone, message, status, provider='', provider_response='', user=None, notification_type='
     ↪ GENERAL'):
        """Log SMS attempt to database."""
        try:
            SMSLog.objects.create(
                recipient_phone=phone,
                message=message[:500],
                status=status,
                provider=provider,
                provider_response=provider_response[:500],
                user=user,
                notification_type=notification_type,
            )
        except Exception as e:
            logger.error(f"Failed to log SMS attempt: {str(e)}")
```

Listing 26.1: SMS Service – notifications/services/sms-service.py

26.4 SMS Log Model

```
from django.db import models
from django.conf import settings

class SMSLog(models.Model):
    """Audit log for all SMS messages sent."""

    class Status(models.TextChoices):
        SENT = 'SENT', 'Successfully Sent'
        FAILED = 'FAILED', 'Failed to Send'
        DEV_MODE = 'DEV_MODE', 'Development Mode'

    recipient_phone = models.CharField(max_length=15)
    message = models.TextField()
    status = models.CharField(max_length=10, choices=Status.choices, default='SENT', db_index=True)

    provider = models.CharField(max_length=50, blank=True, default='')
    provider_response = models.TextField(blank=True, default='')

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='sms_logs',
    )

    notification_type = models.CharField(max_length=50, db_index=True)

    created_at = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        db_table = 'sms_logs'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['status', 'created_at']),
            models.Index(fields=['notification_type', 'created_at']),
        ]

    def __str__(self):
        return f"SMS to {self.recipient_phone} --- {self.status}"
```

Listing 26.2: SMS Log Model – notifications/models/sms-log.py

Chapter Summary

Table 26.2: SMS System — Key Specifications

Property	**Value**
Primary SMS Provider	Africa's Talking
Fallback SMS Provider	Celcom Africa
Provider Selection	One configuration value: SMS_PROVIDER
Development Sandbox	Africa's Talking sandbox + SEND_REAL_SMS=False
Live Safety Switch	No real SMS until SEND_REAL_SMS=True
Audit Trail	Every attempt writes a provider-aware SMSLog record
Switching Providers	Update .env, restart app and workers, verify test SMS

Chapter 27

Notification Types & Templates

27.1 Overview

This chapter documents all notification types and templates across all three channels. Each notification has a specific purpose, priority, and template structure.

27.2 Complete Notification Type Inventory

Table 27.1: Complete Notification Type Inventory

Type	Trigger	Channels	Priority
STOCK_ALERT	Material below threshold	In-App, Email, SMS	URGENT
ORDER_CONFIRMED	Order placed	In-App, Email	MEDIUM
ORDER_STATUS	Order status updated	In-App, Email	MEDIUM
PROJECT_UPDATE	Milestone updated	In-App, Email	MEDIUM
PAYMENT_RECEIVED	Payment confirmed	In-App, Email	LOW
SYSTEM	System notification	In-App, Email	HIGH
REMINDER	Scheduled reminder	In-App, Email	LOW
AUDIT	Bi-weekly audit report	Email	HIGH

27.3 Email Template Inventory

Table 27.2: Email Template Inventory

Template	Purpose	Key Variables
\path{otp_email.html}	8-digit OTP verification	user, otp_code, expiry_minutes
\path{welcome_admin.html}	New admin welcome	user, temp_password, dashboard_url
\path{admin_invitation.html}	Admin invitation	invitation_link, expires_at
\path{order_confirmation.html}	Order confirmation	order, client_name, order_number
\path{order_status_update.html}	Order status update	order, new_status, tracking_number
\path{stock_alert.html}	Low stock alert	material, current_quantity, threshold
\path{payment_receipt.html}	Payment receipt	payment, amount, receipt_number
\path{weekly_report.html}	Weekly report	week_start, week_end, stats
\path{bi_weekly_audit.html}	Bi-weekly audit report	period, actions, stats
\path{admin_contact_alert.html}	Contact form alert	name, email, phone, message
\path{admin_quote_alert.html}	Quote request alert	name, project_type, budget, details

27.4 SMS Template Inventory

Table 27.3: SMS Template Inventory

Template	Purpose	Message Example
stock_alert	Low stock alert	"[ALERT] Low stock: material at quantity unit. Reorder now!"
order_confirmed	Order confirmation	"Order order_number confirmed. We'll notify you when shipped."
order_shipped	Order shipped	"Order order_number shipped! Tracking: tracking"
system_alert	Critical system alert	"[URGENT] System alert: message. Please check dashboard."

27.5 Email Template Example

```
{% extends "email/base_email.html" %}

{% block content %}
<div style="text-align: center;">
    <div style="width: 64px; height: 64px; background: #B9342B; color: white; border-radius: 50%; display: flex;
    ↪ align-items: center; justify-content: center; margin: 0 auto 24px; font-size: 32px;">
        <i class="fas fa-exclamation-triangle"></i>
    </div>
</div>

<h2 style="color: #0D2B36; font-size: 20px; margin: 0 0 16px; text-align: center;">Low Stock Alert</h2>

<p style="text-align: center; color: #4B5563;">The following material has fallen below the minimum threshold:</p>

<div style="background: #F6F1E8; border: 1px solid #E5E7EB; border-radius: 12px; padding: 20px; margin: 24px 0;">
    <div style="display: flex; justify-content: space-between; padding: 8px 0; border-bottom: 1px solid #E5E7EB;">
        <span style="font-weight: 600; color: #6B7280;">Material:</span>
        <span style="font-weight: 500; color: #1F2937;">{{ material.name }}</span>
    </div>
    <div style="display: flex; justify-content: space-between; padding: 8px 0; border-bottom: 1px solid #E5E7EB;">
        <span style="font-weight: 600; color: #6B7280;">Current Quantity:</span>
        <span style="font-weight: 700; color: #B9342B;">{{ material.current_quantity }} {{ material.unit }}</span>
    </div>
    <div style="display: flex; justify-content: space-between; padding: 8px 0;">
        <span style="font-weight: 600; color: #6B7280;">Minimum Threshold:</span>
        <span style="font-weight: 500; color: #1F2937;">{{ material.minimum_threshold }} {{ material.unit }}</span>
    </div>
</div>

<div style="text-align: center;">
    <a href="{{ dashboard_url }}" class="btn">View Inventory</a>
</div>
{% endblock %}
```

Listing 27.1: Stock Alert Email – templates/email/stock-alert.html

Chapter Summary

Table 27.4: Notification Types & Templates — Key Specifications

Property	Value
Notification Types	8
Email Templates	11+
SMS Templates	4
Template Variables	User, material, order, payment, etc.
Priority Mapping	URGENT -> SMS + Email + In-App
Branding	Caldan Construction colors and logo

Chapter 28

Reminder Engine & Automation

28.1 Overview

The Reminder Engine automates routine notifications and alerts. It runs daily checks for low stock, pending orders, and other critical events.

28.2 Reminder Service

```
from django.utils import timezone
from datetime import timedelta
from django.db.models import Q
from notifications.services.router_service import NotificationRouter
from inventory.models.material import Material
from shop.models.order import Order
import logging

logger = logging.getLogger('caldan.reminders')

class ReminderService:
    """Automated reminder engine for various events."""

    @classmethod
    def check_low_stock(cls):
        """Check and send low stock alerts."""
        low_stock_materials = Material.objects.filter(
            is_low_stock=True,
            is_active=True
        )

        alerts_sent = 0
        for material in low_stock_materials:
            # Send to admin
            from accounts.models.user import User
            admin = User.objects.filter(is_superuser=True).first()

            if admin:
                NotificationRouter.route(
                    user=admin,
                    title=f"[WARNING] LOW STOCK: {material.name}",
                    message=f"{material.name} is at {material.current_quantity} {material.unit}. "
                            f"Minimum threshold is {material.minimum_threshold} {material.unit}.",
                    notification_type='STOCK_ALERT',
                    priority='URGENT',
                    icon='fa-exclamation-triangle',
                    send_email=True,
                    send_sms=True,
                    metadata={'material_id': material.id}
                )
                alerts_sent += 1

        return alerts_sent

    @classmethod
    def check_pending_orders(cls):
        """Check for pending orders that need attention."""
        pending_orders = Order.objects.filter(
            status='PENDING',
            created_at__lt=timezone.now() - timedelta(hours=24)
        )

        notifications_sent = 0
        for order in pending_orders:
            from accounts.models.user import User
            admin = User.objects.filter(is_superuser=True).first()

            if admin:
                NotificationRouter.route(
```

```
                    user=admin,
                    title=f"[REMINDER] Pending Order: {order.order_number}",
                    message=f"Order {order.order_number} from {order.client_name} has been pending for over 24 hours
    ↪ .",
                    notification_type='REMINDER',
                    priority='HIGH',
                    icon='fa-clock',
                    send_email=True,
                    send_sms=False,
                    metadata={'order_id': order.id}
                )
                notifications_sent += 1

        return notifications_sent

    @classmethod
    def run_daily_reminders(cls):
        """Run all daily reminders."""
        results = {
            'low_stock_alerts': cls.check_low_stock(),
            'pending_orders': cls.check_pending_orders(),
        }
        logger.info(f"Daily reminders completed: {results}")
        return results
```

Listing 28.1: Reminder Service – notifications/services/reminder-service.py

28.3 Celery Task Configuration

```
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    # Daily reminders --- 8:00 AM EAT
    'run-daily-reminders': {
        'task': 'notifications.tasks.run_daily_reminders',
        'schedule': crontab(hour=8, minute=0),
    },

    # Bi-weekly audit report --- Monday 7:00 AM EAT (every 2 weeks)
    'send-bi-weekly-audit': {
        'task': 'notifications.tasks.send_bi_weekly_audit',
        'schedule': crontab(hour=7, minute=0, day_of_week=1),
    },

    # Database backup --- daily 2:00 AM EAT
    'database-backup': {
        'task': 'reports.tasks.database_backup',
        'schedule': crontab(hour=2, minute=0),
    },

    # Session cleanup --- daily midnight
    'session-cleanup': {
        'task': 'accounts.tasks.cleanup_sessions',
        'schedule': crontab(hour=0, minute=0),
    },
}
```

Listing 28.2: Celery Beat Schedule – celery.py

Chapter Summary

Table 28.1: Reminder Engine — Key Specifications

Property	Value
Daily Checks	Low stock, pending orders
Bi-Weekly Audit	Every 14 days (Monday)
Database Backup	Daily (2:00 AM EAT)
Session Cleanup	Daily (midnight)
Task Queue	Celery + Redis
Logging	All reminder actions logged

Chapter 29

Reports Engine

29.1 Overview

The Caldan Construction Reports Engine transforms raw operational data—projects, inventory, orders, payments—into actionable intelligence. Every report is available as both an interactive dashboard view and a downloadable export (PDF, CSV, or Excel).

KEY INSIGHT Design Philosophy: Reports are not a separate "analytics product"—they are embedded directly into the dashboard workflow. A Platform Admin checking project progress is one click away from a project report.

29.2 Report Architecture

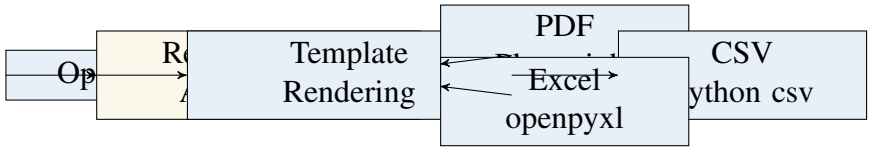


Figure 29.1: Report Generation Architecture

29.3 Report Inventory

Table 29.1: Complete Report Inventory

Report	Description	Export Formats	Access
Project Summary	Overall project status, progress, budget	PDF, CSV, Excel	Admin
Project Progress	Milestone progress, timeline	PDF, CSV	Admin
Project Budget	Budget vs actual spending	PDF, CSV, Excel	Admin
Inventory Report	All materials, current stock, alerts	PDF, CSV	Admin
Stock Transaction Report	IN/OUT history with details	PDF, CSV	Admin
Order Report	All orders, status, revenue	PDF, CSV, Excel	Admin
Financial Report	Payments, expenses, income	PDF, CSV, Excel	Admin
Client Report	Client list, active/inactive	PDF, CSV	Admin
Audit Report	System actions, user activity	PDF, CSV	Admin

29.4 Report Service

```
from django.db.models import Sum, Count, Q
from django.utils import timezone
from datetime import timedelta
from decimal import Decimal
from projects.models import Project
from inventory.models import Material, StockTransaction
from shop.models import Order
from finances.models import Payment, Expense

class ReportService:
    """Centralized report generation service."""

    @classmethod
    def get_project_summary(cls, project_id):
        """Generate comprehensive project summary report."""
        project = Project.objects.get(id=project_id)

        # Milestone stats
        milestones = project.milestones.all()
        total_milestones = milestones.count()
        completed_milestones = milestones.filter(status='COMPLETED').count()
        overdue_milestones = milestones.filter(status='OVERDUE').count()

        # Financial stats
        payments = Payment.objects.filter(project=project, status='COMPLETED')
        expenses = Expense.objects.filter(project=project)
        total_payments = payments.aggregate(total=Sum('amount'))['total'] or 0
        total_expenses = expenses.aggregate(total=Sum('amount'))['total'] or 0

        # Inventory stats (for this project)
        materials = Material.objects.filter(project=project)
        total_materials = materials.count()
        low_stock = materials.filter(is_low_stock=True).count()

        return {
            'project': project,
```

```
            'milestones': {
                'total': total_milestones,
                'completed': completed_milestones,
                'overdue': overdue_milestones,
                'completion_rate': round((completed_milestones / total_milestones * 100), 1) if total_milestones > 0
    ↪ else 0,
            },
            'financial': {
                'total_payments': float(total_payments),
                'total_expenses': float(total_expenses),
                'net_position': float(total_payments - total_expenses),
            },
            'inventory': {
                'total_materials': total_materials,
                'low_stock': low_stock,
            },
            'generated_at': timezone.now(),
        }

    @classmethod
    def get_inventory_report(cls):
        """Generate inventory report."""
        materials = Material.objects.filter(is_active=True)

        total_materials = materials.count()
        low_stock = materials.filter(is_low_stock=True).count()
        out_of_stock = materials.filter(current_quantity=0).count()
        total_value = materials.aggregate(total=Sum('total_cost'))['total'] or 0

        # By category
        category_breakdown = {}
        for category in Material.Category.choices:
            cat_materials = materials.filter(category=category[0])
            count = cat_materials.count()
            if count > 0:
                category_breakdown[category[1]] = {
                    'count': count,
                    'value': float(cat_materials.aggregate(total=Sum('total_cost'))['total'] or 0),
                    'low_stock': cat_materials.filter(is_low_stock=True).count(),
                }

        return {
            'total_materials': total_materials,
            'low_stock_count': low_stock,
            'out_of_stock_count': out_of_stock,
            'total_value': float(total_value),
            'category_breakdown': category_breakdown,
            'generated_at': timezone.now(),
        }

    @classmethod
    def get_order_report(cls):
        """Generate order report."""
        orders = Order.objects.all()

        total_orders = orders.count()
        pending_orders = orders.filter(status='PENDING').count()
        processing_orders = orders.filter(status='PROCESSING').count()
        completed_orders = orders.filter(status='COMPLETED').count()
        cancelled_orders = orders.filter(status='CANCELLED').count()

        total_revenue = orders.filter(
            status__in=['PAID', 'PROCESSING', 'SHIPPED', 'DELIVERED', 'COMPLETED']
        ).aggregate(total=Sum('total'))['total'] or 0

        return {
            'total_orders': total_orders,
            'pending_orders': pending_orders,
            'processing_orders': processing_orders,
            'completed_orders': completed_orders,
            'cancelled_orders': cancelled_orders,
            'total_revenue': float(total_revenue),
            'generated_at': timezone.now(),
        }
```

Listing 29.1: Report Service – reports/services/report-service.py

29.5 Report Views

```
from django.shortcuts import render
from django.contrib.auth.decorators import login_required
from accounts.permissions.decorators import admin_required
from reports.services.report_service import ReportService

@login_required
@admin_required
```

```
def report_index(request):
    """Reports dashboard."""
    # Get quick stats for all report types
    inventory_report = ReportService.get_inventory_report()
    order_report = ReportService.get_order_report()

    context = {
        'inventory_report': inventory_report,
        'order_report': order_report,
        'report_types': [
            {'name': 'Project Summary', 'icon': 'fa-building', 'url': 'reports:project'},
            {'name': 'Inventory Report', 'icon': 'fa-boxes', 'url': 'reports:inventory'},
            {'name': 'Order Report', 'icon': 'fa-shopping-bag', 'url': 'reports:orders'},
            {'name': 'Financial Report', 'icon': 'fa-money-bill-wave', 'url': 'reports:financial'},
            {'name': 'Client Report', 'icon': 'fa-users', 'url': 'reports:clients'},
            {'name': 'Audit Report', 'icon': 'fa-history', 'url': 'reports:audit'},
        ],
        'page_title': 'Reports Dashboard',
    }
    return render(request, 'dashboard/admin/reports/index.html', context)
```

Listing 29.2: Report Views – reports/views/-admin-views.py

Chapter Summary

Table 29.2: Reports Engine — Key Specifications

Property	Value
Report Types	9
Export Formats	3: PDF, CSV, Excel
PDF Engine	Playwright (headless Chromium)
CSV Engine	Python csv module
Excel Engine	openpyxl
Data Aggregation	Django ORM with annotations
Report Access	Admin only

Chapter 30

PDF Generation (Playwright)

30.1 Overview

Caldan Construction uses Playwright with headless Chromium for PDF generation. This provides complete control over PDF styling, supports modern CSS, and ensures consistent output across all document types.

> **INFORMATION** WeasyPrint is explicitly not part of the Caldan Construction PDF stack. All PDFs are rendered with Playwright using headless Chromium for maximum compatibility and styling control.

30.2 PDF Service

```
from playwright.sync_api import sync_playwright
from django.template.loader import render_to_string
from django.conf import settings
import os
from datetime import datetime

class PDFService:
    """Generate styled PDF exports using Playwright."""

    @classmethod
    def generate_pdf(cls, template_name, context, output_filename):
        """Generate PDF from HTML template."""
        # Render HTML
        html_string = render_to_string(template_name, context)

        # Ensure output directory exists
        output_dir = os.path.join(settings.MEDIA_ROOT, 'reports')
        os.makedirs(output_dir, exist_ok=True)

        output_path = os.path.join(output_dir, output_filename)

        # Generate PDF using Playwright
        with sync_playwright() as p:
            browser = p.chromium.launch(headless=True)
            page = browser.new_page()
            page.set_content(html_string, wait_until="networkidle")

            page.pdf(
                path=output_path,
                format='A4',
                print_background=True,
                margin={
                    'top': '1.5cm',
                    'bottom': '1.5cm',
                    'left': '1.5cm',
                    'right': '1.5cm',
                }
            )
            browser.close()

        return output_path

    @classmethod
    def generate_project_report_pdf(cls, project_data):
        """Generate project summary report PDF."""
        filename = f"project_report_{project_data['project'].project_code}_{datetime.now().strftime('%Y%m%d')}.pdf"
        return cls.generate_pdf(
            template_name='pdf/project_report.html',
```

```
            context=project_data,
            output_filename=filename
        )

    @classmethod
    def generate_inventory_report_pdf(cls, inventory_data):
        """Generate inventory report PDF."""
        filename = f"inventory_report_{datetime.now().strftime('%Y%m%d')}.pdf"
        return cls.generate_pdf(
            template_name='pdf/inventory_report.html',
            context={'report': inventory_data},
            output_filename=filename
        )

    @classmethod
    def generate_order_report_pdf(cls, order_data):
        """Generate order report PDF."""
        filename = f"order_report_{datetime.now().strftime('%Y%m%d')}.pdf"
        return cls.generate_pdf(
            template_name='pdf/order_report.html',
            context={'report': order_data},
            output_filename=filename
        )

    @classmethod
    def generate_audit_report_pdf(cls, audit_data):
        """Generate audit report PDF."""
        filename = f"audit_report_{datetime.now().strftime('%Y%m%d')}.pdf"
        return cls.generate_pdf(
            template_name='pdf/audit_report.html',
            context={'report': audit_data},
            output_filename=filename
        )

    @classmethod
    def generate_receipt_pdf(cls, payment):
        """Generate payment receipt PDF."""
        filename = f"receipt_{payment.receipt.receipt_number}.pdf"
        return cls.generate_pdf(
            template_name='pdf/receipt.html',
            context={
                'payment': payment,
                'receipt': payment.receipt,
            },
            output_filename=filename
        )

    @classmethod
    def generate_payslip_pdf(cls, payroll_data):
        """Generate payslip PDF."""
        filename = f"payslip_{payroll_data['worker'].full_name}_{datetime.now().strftime('%Y%m')}.pdf"
        return cls.generate_pdf(
            template_name='pdf/payslip.html',
            context=payroll_data,
            output_filename=filename
        )
```

Listing 30.1: PDF Service – reports/services/pdf-service.py

30.3 PDF Template Example

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <style>
        @page {
            size: A5;
            margin: 1.5cm;
        }
        body {
            font-family: 'Helvetica Neue', Arial, sans-serif;
            color: #1F2937;
            font-size: 11pt;
        }
        .receipt-header {
            border-bottom: 2px solid #D69723;
            padding-bottom: 15px;
            margin-bottom: 20px;
            text-align: center;
        }
        .receipt-logo {
            max-height: 40px;
            margin-bottom: 10px;
        }
        .receipt-title {
```

```
            font-size: 18pt;
            font-weight: bold;
            color: #0D2B36;
            margin: 5px 0;
        }
        .receipt-number {
            font-size: 10pt;
            color: #6B7280;
        }
        .detail-row {
            display: flex;
            justify-content: space-between;
            padding: 8px 0;
            border-bottom: 1px solid #E5E7EB;
        }
        .detail-label {
            font-weight: bold;
            color: #6B7280;
        }
        .detail-value {
            text-align: right;
        }
        .amount-highlight {
            font-size: 16pt;
            font-weight: bold;
            color: #0D2B36;
        }
        .receipt-footer {
            margin-top: 30px;
            padding-top: 15px;
            border-top: 1px solid #E5E7EB;
            font-size: 9pt;
            color: #6B7280;
            text-align: center;
        }
        .gold-accent {
            color: #D69723;
        }
    </style>
</head>
<body>
    <div class="receipt-header">
        <h1 class="receipt-title">CALDAN CONSTRUCTION</h1>
        <p class="receipt-number">Payment Receipt</p>
        <p class="receipt-number">#{{ receipt.receipt_number }}</p>
    </div>

    <div class="receipt-details">
        <div class="detail-row">
            <span class="detail-label">Client</span>
            <span class="detail-value">{{ payment.client_name }}</span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Order</span>
            <span class="detail-value">{{ payment.order.order_number }}</span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Amount</span>
            <span class="detail-value amount-highlight">KES {{ payment.amount|floatformat:2 }}</span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Payment Method</span>
            <span class="detail-value">{{ payment.get_method_display }}</span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Date</span>
            <span class="detail-value">{{ payment.payment_date|date:"d M Y" }}</span>
        </div>
        <div class="detail-row">
            <span class="detail-label">Status</span>
            <span class="detail-value gold-accent">Confirmed</span>
        </div>
    </div>

    <div class="receipt-footer">
        <p>This receipt was generated by <strong>Caldan Construction</strong></p>
        <p>Building Kenya's Future, One Project at a Time.</p>
        <p>{{ receipt.generated_at|date:"d/m/Y H:i" }} EAT</p>
    </div>
</body>
</html>
```

Listing 30.2: Receipt PDF Template – templates/pdf/receipt.html

30.4 Installation & Configuration

```
# PDF Generation
playwright>=1.40.0

# Install Chromium browser runtime during deployment
playwright install chromium
```

Listing 30.3: Playwright Installation – requirements.txt

Chapter Summary

Table 30.1: PDF Generation — Key Specifications

Property	**Value**
PDF Engine	Playwright + headless Chromium
Document Types	Receipts, Project Reports, Inventory Reports, Order Reports, Audit Reports, Payslips
Page Size	A4 (reports), A5 (receipts)
CSS Support	Full modern CSS
Print Background	Enabled
Margins	1.5cm (top, bottom, left, right)
File Storage	Cloudinary (media/reports/)

Chapter 31

Data Export & Backup

31.1 Overview

The Data Export and Backup module provides comprehensive data management capabilities—from exporting project data in multiple formats to creating complete backups and safely deleting data.

KEY INSIGHT Core Principle: Data is an asset. Caldan Construction provides all the tools to export, backup, and manage data responsibly. No data is ever permanently lost without explicit confirmation.

31.2 Data Export Center

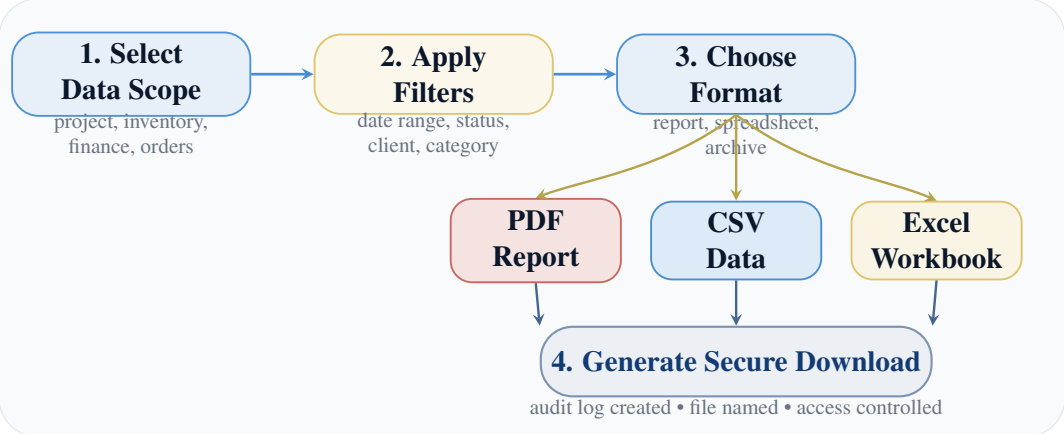


Figure 31.1: Data Export Flow

31.3 Export Service

```
import csv
import zipfile
from io import BytesIO, StringIO
from django.http import HttpResponse
from django.utils import timezone
from reports.services.pdf_service import PDFService

class ExportService:
    """Service for data export operations."""

    @classmethod
    def export_to_csv(cls, data, fields, filename):
        """Export data to CSV."""
```

```
        response = HttpResponse(content_type='text/csv')
        response['Content-Disposition'] = f'attachment; filename="{filename}"'

        writer = csv.writer(response)
        writer.writerow([field.replace('_', ' ').title() for field in fields])

        for item in data:
            row = []
            for field in fields:
                value = item.get(field, '')
                if isinstance(value, (int, float)):
                    row.append(str(value))
                else:
                    row.append(str(value) if value else '')
            writer.writerow(row)

        return response

    @classmethod
    def export_to_excel(cls, data, fields, filename):
        """Export data to Excel."""
        try:
            import openpyxl
            from openpyxl.styles import Font, PatternFill, Alignment
        except ImportError:
            raise ImportError("openpyxl is required for Excel export")

        wb = openpyxl.Workbook()
        ws = wb.active
        ws.title = "Data Export"

        # Header
        header_font = Font(bold=True, color="FFFFFF")
        header_fill = PatternFill(start_color="0D2B36", end_color="0D2B36", fill_type="solid")

        for col, field in enumerate(fields, 1):
            cell = ws.cell(row=1, column=col, value=field.replace('_', ' ').title())
            cell.font = header_font
            cell.fill = header_fill
            cell.alignment = Alignment(horizontal='center')

        # Data
        for row_idx, item in enumerate(data, 2):
            for col_idx, field in enumerate(fields, 1):
                value = item.get(field, '')
                ws.cell(row=row_idx, column=col_idx, value=value)

        # Auto-column width
        for column in ws.columns:
            max_length = 0
            column_letter = column[0].column_letter
            for cell in column:
                try:
                    if len(str(cell.value)) > max_length:
                        max_length = len(str(cell.value))
                except:
                    pass
            adjusted_width = min(max_length + 2, 50)
            ws.column_dimensions[column_letter].width = adjusted_width

        response = HttpResponse(
            content_type='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'
        )
        response['Content-Disposition'] = f'attachment; filename="{filename}"'
        wb.save(response)
        return response

    @classmethod
    def create_zip_archive(cls, files):
        """Create a ZIP archive from multiple files."""
        buffer = BytesIO()
        with zipfile.ZipFile(buffer, 'w', zipfile.ZIP_DEFLATED) as zip_file:
            for name, content in files.items():
                if isinstance(content, bytes):
                    zip_file.writestr(name, content)
                else:
                    zip_file.write(content, name)

        buffer.seek(0)
        response = HttpResponse(buffer, content_type='application/zip')
        response['Content-Disposition'] = f'attachment; filename="export_{timezone.now().strftime("%Y%m%d")}.zip"'
        return response
```

Listing 31.1: Export Service – reports/services/export-service.py

31.4 Backup Service

```
import os
import json
import zipfile
from django.utils import timezone
from django.conf import settings
from projects.models import Project
from accounts.services.audit_service import AuditService

class BackupService:
    """Service for project backup operations."""

    @classmethod
    def create_project_backup(cls, project_id, user):
        """Create a complete backup of a project."""
        project = Project.objects.get(id=project_id)

        # Create backup directory
        backup_dir = os.path.join(settings.MEDIA_ROOT, 'backups', project.project_code)
        os.makedirs(backup_dir, exist_ok=True)

        # Generate project data
        backup_data = {
            'project': {
                'id': project.id,
                'name': project.name,
                'project_code': project.project_code,
                'description': project.description,
                'location': project.location,
                'county': project.county,
                'town': project.town,
                'start_date': project.start_date.isoformat(),
                'end_date': project.end_date.isoformat() if project.end_date else None,
                'budget': float(project.budget),
                'spent': float(project.spent),
                'progress': float(project.progress),
                'status': project.status,
                'created_at': project.created_at.isoformat(),
            },
            'milestones': [],
            'payments': [],
            'expenses': [],
            'materials': [],
        }

        # Add related data
        for milestone in project.milestones.all():
            backup_data['milestones'].append({
                'id': milestone.id,
                'title': milestone.title,
                'description': milestone.description,
                'target_date': milestone.target_date.isoformat(),
                'completed_date': milestone.completed_date.isoformat() if milestone.completed_date else None,
                'completion_percentage': float(milestone.completion_percentage),
                'status': milestone.status,
            })

        for payment in project.payments.all():
            backup_data['payments'].append({
                'id': payment.id,
                'amount': float(payment.amount),
                'type': payment.type,
                'method': payment.method,
                'description': payment.description,
                'paid_to': payment.paid_to,
                'payment_date': payment.payment_date.isoformat(),
                'status': payment.status,
            })

        for expense in project.expenses.all():
            backup_data['expenses'].append({
                'id': expense.id,
                'amount': float(expense.amount),
                'category': expense.category,
                'description': expense.description,
                'expense_date': expense.expense_date.isoformat(),
                'vendor_name': expense.vendor_name,
            })

        for material in project.materials.all():
            backup_data['materials'].append({
                'id': material.id,
                'name': material.name,
                'category': material.category,
                'unit': material.unit,
                'current_quantity': float(material.current_quantity),
                'minimum_threshold': float(material.minimum_threshold),
                'unit_cost': float(material.unit_cost),
                'total_cost': float(material.total_cost),
            })

        # Save JSON file
        json_path = os.path.join(backup_dir, f"{project.project_code}_data.json")
        with open(json_path, 'w') as f:
```

```
            json.dump(backup_data, f, indent=2)

        # Create ZIP archive
        zip_path = os.path.join(settings.MEDIA_ROOT, 'backups', f"{project.project_code}_{timezone.now().strftime('%Y
    ↪ %m%d')}.zip")
        with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zip_file:
            zip_file.write(json_path, os.path.basename(json_path))

        # Clean up JSON file
        os.remove(json_path)

        # Log audit
        AuditService.log_action(
            user=user,
            action='PROJECT_BACKUP_CREATED',
            details={
                'project': project.name,
                'project_id': project.id,
                'backup_file': os.path.basename(zip_path),
            }
        )

        return zip_path

    @classmethod
    def restore_project(cls, backup_file, user):
        """Restore a project from backup."""
        # Extract ZIP
        import tempfile
        import shutil

        temp_dir = tempfile.mkdtemp()
        with zipfile.ZipFile(backup_file, 'r') as zip_ref:
            zip_ref.extractall(temp_dir)

        # Find JSON file
        json_files = [f for f in os.listdir(temp_dir) if f.endswith('.json')]
        if not json_files:
            raise ValueError("No JSON data found in backup")

        with open(os.path.join(temp_dir, json_files[0]), 'r') as f:
            data = json.load(f)

        # Create project from data
        from projects.models import Project
        project_data = data['project']

        project = Project.objects.create(
            name=project_data['name'],
            project_code=project_data['project_code'],
            description=project_data['description'],
            location=project_data['location'],
            county=project_data['county'],
            town=project_data['town'],
            start_date=project_data['start_date'],
            end_date=project_data['end_date'],
            budget=project_data['budget'],
            spent=project_data['spent'],
            progress=project_data['progress'],
            status=project_data['status'],
        )

        # Restore milestones
        for milestone_data in data['milestones']:
            project.milestones.create(
                title=milestone_data['title'],
                description=milestone_data['description'],
                target_date=milestone_data['target_date'],
                completed_date=milestone_data['completed_date'],
                completion_percentage=milestone_data['completion_percentage'],
                status=milestone_data['status'],
            )

        # Log audit
        AuditService.log_action(
            user=user,
            action='PROJECT_RESTORED_FROM_BACKUP',
            details={
                'project': project.name,
                'project_id': project.id,
                'backup_file': os.path.basename(backup_file),
            }
        )

        # Clean up
        shutil.rmtree(temp_dir)

        return project
```

Listing 31.2: Backup Service – reports/services/backup-service.py

31.5 Data Deletion

```
from django.db import transaction
from django.utils import timezone
from projects.models import Project
from accounts.services.audit_service import AuditService
from reports.services.backup_service import BackupService

class DeletionService:
    """Service for safe project deletion."""

    @classmethod
    def soft_delete_project(cls, project_id, user, reason=''):
        """Soft delete a project (archived, can be restored)."""
        project = Project.objects.get(id=project_id)

        if project.status == 'ARCHIVED':
            raise ValueError("Project is already archived")

        project.status = 'ARCHIVED'
        project.is_archived = True
        project.archived_at = timezone.now()
        project.save()

        # Log audit
        AuditService.log_action(
            user=user,
            action='PROJECT_SOFT_DELETED',
            details={
                'project': project.name,
                'project_id': project.id,
                'reason': reason,
            }
        )

        return project

    @classmethod
    def restore_project(cls, project_id, user):
        """Restore a soft-deleted project."""
        project = Project.objects.get(id=project_id, is_archived=True)

        project.status = 'ACTIVE'
        project.is_archived = False
        project.archived_at = None
        project.save()

        # Log audit
        AuditService.log_action(
            user=user,
            action='PROJECT_RESTORED',
            details={
                'project': project.name,
                'project_id': project.id,
            }
        )

        return project

    @classmethod
    @transaction.atomic
    def hard_delete_project(cls, project_id, user, confirm_backup=True):
        """Permanently delete a project (IRREVERSIBLE)."""
        project = Project.objects.get(id=project_id)

        # Check if backup exists (safety)
        if confirm_backup:
            import os
            from django.conf import settings
            backup_dir = os.path.join(settings.MEDIA_ROOT, 'backups')
            if not os.path.exists(os.path.join(backup_dir, f"{project.project_code}_*.zip")):
                raise ValueError("Please create a backup before permanent deletion")

        # Store project info for audit
        project_name = project.name
        project_code = project.project_code

        # Delete all related data
        project.milestones.all().delete()
        project.materials.all().delete()
        project.stock_transactions.all().delete()
        project.payments.all().delete()
        project.expenses.all().delete()
        project.budgets.all().delete()

        # Delete project
        project.delete()

        # Log audit
        AuditService.log_action(
            user=user,
```

```
            action='PROJECT_HARD_DELETED',
            details={
                'project_name': project_name,
                'project_code': project_code,
                'deleted_at': timezone.now().isoformat(),
            }
        )

        return True
```

Listing 31.3: Deletion Service – reports/services/deletion-service.py

Chapter Summary

Table 31.1: Data Export & Backup — Key Specifications

Property	**Value**
Export Formats	4: PDF, CSV, Excel, ZIP
Export Types	Single project, Bulk (multiple)
Backup Types	Full project backup (JSON + ZIP)
Backup Storage	Cloudinary (media/backups/)
Soft Delete	Archive with restore option
Hard Delete	Permanent deletion (requires backup)
Audit Trail	All export/backup/delete actions logged

Chapter 32

Bi-Weekly Audit Reports

32.1 Overview

The Bi-Weekly Audit Report is an automated security and compliance feature. Every 14 days, a comprehensive PDF report is generated and emailed to the Platform Admin, summarizing all system activity.

KEY INSIGHT Core Principle: You can't secure what you don't monitor. The bi-weekly audit report provides regular visibility into all system activity, enabling proactive security management.

32.2 Audit Report Contents

Table 32.1: Audit Report Contents

Section	Content
Executive Summary	Period covered, total actions, key metrics
Authentication Events	Successful logins, failed logins, account lockouts
User Management	New admins added, admins deleted, permission changes
Project Activity	Projects created, updated, completed, archived
Inventory Activity	Stock IN/OUT, low stock alerts, new materials
Shop Activity	Orders placed, order status changes, revenue
Financial Activity	Payments received, expenses recorded
Security Events	OTP failures, suspicious IP addresses, rate limit breaches
System Health	Backup status, error rates, performance metrics

32.3 Audit Report Service

```
from django.utils import timezone
from datetime import timedelta
from accounts.models.audit import AuditLog
from accounts.models.user import User
from reports.services.pdf_service import PDFService
from notifications.services.email_service import EmailService

class AuditReportService:
    """Service for generating bi-weekly audit reports."""

    @classmethod
    def generate_and_send(cls):
        """Generate and send bi-weekly audit report."""
        today = timezone.now().date()
        period_start = today - timedelta(days=14)
        period_end = today - timedelta(days=1)

        # Get all audit logs for the period
        audit_logs = AuditLog.objects.filter(
            created_at__date__gte=period_start,
            created_at__date__lte=period_end
        ).order_by('-created_at')

        # Get admin users
        admins = User.objects.filter(is_staff=True, is_active=True)

        # Prepare report data
        report_data = {
            'period_start': period_start,
            'period_end': period_end,
            'total_actions': audit_logs.count(),
            'unique_users': audit_logs.values('user').distinct().count(),
            'action_breakdown': cls._get_action_breakdown(audit_logs),
            'security_events': cls._get_security_events(audit_logs),
            'user_activity': cls._get_user_activity(audit_logs),
            'generated_at': timezone.now(),
        }

        # Generate PDF
        pdf_path = PDFService.generate_audit_report_pdf(report_data)

        # Send to all admins
        sent_count = 0
        for admin in admins:
            if admin.email:
                success = EmailService.send_template(
                    to_email=admin.email,
                    subject=f"Bi-Weekly Audit Report --- {period_start.strftime('%d/%m')} to {period_end.strftime('%d
 ↪ /%m/%Y')}",
                    template_name='email/bi_weekly_audit.html',
                    context={
                        'admin': admin,
                        'period_start': period_start,
                        'period_end': period_end,
                        'total_actions': report_data['total_actions'],
                        'unique_users': report_data['unique_users'],
                        'report_pdf_url': pdf_path,
                        'dashboard_url': '/dashboard/',
                    },
                    user=admin,
                    email_type='AUDIT_REPORT'
                )
                if success:
                    sent_count += 1

        return {'sent': sent_count, 'total_admins': admins.count(), 'pdf_path': pdf_path}

    @classmethod
    def _get_action_breakdown(cls, audit_logs):
        """Get breakdown of actions by type."""
        from django.db.models import Count
        return audit_logs.values('action').annotate(count=Count('id')).order_by('-count')[:10]

    @classmethod
    def _get_security_events(cls, audit_logs):
        """Get security-related events."""
        security_actions = ['LOGIN_FAILED', 'OTP_FAILED', 'ACCOUNT_LOCKED', 'PERMISSION_CHANGED']
        return audit_logs.filter(action__in=security_actions)

    @classmethod
    def _get_user_activity(cls, audit_logs):
        """Get activity breakdown by user."""
        from django.db.models import Count
        return audit_logs.values('user__email', 'user__full_name').annotate(
            count=Count('id')
        ).order_by('-count')[:10]
```

Listing 32.1: Audit Report Service – notifications/services/audit-report-service.py

32.4 Celery Task

```
from celery import shared_task
from notifications.services.audit_report_service import AuditReportService
import logging

logger = logging.getLogger('caldan.audit')

@shared_task
def send_bi_weekly_audit():
    """Celery task: Generate and send bi-weekly audit report."""
    logger.info("Starting bi-weekly audit report generation")
    result = AuditReportService.generate_and_send()
    logger.info(f"Bi-weekly audit report sent to {result['sent']} admins")
    return result
```

Listing 32.2: Bi-Weekly Audit Task – notifications/tasks.py

Chapter Summary

Table 32.2: Bi-Weekly Audit Reports — Key Specifications

Property	**Value**
Frequency	Every 14 days
Trigger	Celery beat scheduled task
Format	PDF + Email
Content	All system activity, security events, user actions
Delivery	Email to all Platform Admins
Storage	Archived in media/reports/audit/
Retention	90 days

Chapter 33

Security Architecture

33.1 Overview

Security is not an afterthought in Caldan Construction—it is embedded in every layer of the architecture. This chapter defines the comprehensive security measures implemented to protect user data and ensure platform integrity.

KEY INSIGHT Security Philosophy: Defense in depth. No single security control is relied upon exclusively. Multiple layers of protection work together to create a resilient security posture.

33.2 Threat Model

Table 33.1: Threat Model — Identified Risks

Threat Category	Examples	Mitigation
Unauthorized Access	Credential theft, session hijacking	OTP, rate limiting, HTTPS
Data Breach	SQL injection, exposed API keys	Parameterized queries, .env
Insider Threat	Malicious admin, compromised account	Audit logging, least privilege
Payment Fraud	Stolen cards, fake transactions	Paystack 3D Secure, webhook verification
Data Loss	Accidental deletion, hardware failure	Automated backups, soft delete

33.3 Network Security

33.3.1 HTTPS Everywhere

All traffic to caldanconstruction.com is encrypted via TLS 1.3:

```
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000  # 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')
```

Listing 33.1: HTTPS Configuration – settings.py

33.3.2 Content Security Policy

```
CSP_DEFAULT_SRC = ("'self'",)
CSP_SCRIPT_SRC = ("'self'", "'unsafe-inline'", "https://cdnjs.cloudflare.com", "https://unpkg.com")
CSP_STYLE_SRC = ("'self'", "'unsafe-inline'", "https://fonts.googleapis.com", "https://cdnjs.cloudflare.com")
CSP_IMG_SRC = ("'self'", "data:", "https://*.cloudinary.com")
CSP_FONT_SRC = ("'self'", "https://fonts.gstatic.com", "https://cdnjs.cloudflare.com")
CSP_CONNECT_SRC = ("'self'", "https://api.paystack.co", "https://nominatim.openstreetmap.org")
```

Listing 33.2: CSP Configuration – settings.py

33.4 Application Security

33.4.1 Authentication Security

Table 33.2: Authentication Security Measures

Measure	Implementation
Password Hashing	bcrypt (cost factor 12)
Password Requirements	Minimum 8 characters, mix of letters and numbers
Account Lockout	5 failed attempts -> 15-minute lockout
Session Management	HTTP-only cookies, SameSite=Lax, secure flag
Session Timeout	24-hour default; 30-day for "Remember Me"
OTP Security	8-digit code, 5-minute expiry, 3 attempts max
Trusted Device	SHA-256 fingerprint, 30-day expiry, 10 device limit

33.4.2 Input Validation

All user input is validated through Django forms:

```
from django import forms

class MaterialForm(forms.ModelForm):
    class Meta:
        model = Material
        fields = ['name', 'category', 'unit', 'current_quantity', 'minimum_threshold']

    def clean_current_quantity(self):
        quantity = self.cleaned_data['current_quantity']
        if quantity < 0:
            raise forms.ValidationError('Quantity cannot be negative')
        return quantity

    def clean_minimum_threshold(self):
        threshold = self.cleaned_data['minimum_threshold']
        if threshold < 0:
            raise forms.ValidationError('Threshold cannot be negative')
        return threshold
```

Listing 33.3: Input Validation Example

33.5 Data Security

33.5.1 Encryption at Rest

Table 33.3: Encryption at Rest

Data Type	Encryption Method	Notes
User Passwords	bcrypt hashing	Not reversible
Database	MySQL InnoDB encryption	Future implementation
Media Files	Cloudinary secure storage	Private access controls
Backups	GPG encryption	Future implementation

33.6 Audit Logging

All security events are logged:

Table 33.4: Security Events Logged

Event	Description
LOGIN_SUCCESS	Successful login attempt
LOGIN_FAILED	Failed login attempt
OTP_SENT	OTP sent to email
OTP_VERIFIED	OTP successfully verified
OTP_FAILED	OTP verification failed
ACCOUNT_LOCKED	Account locked due to failed attempts
PASSWORD_CHANGED	Password changed
PASSWORD_RESET	Password reset requested
USER_CREATED	New user created
USER_DELETED	User deleted
PERMISSION_CHANGED	Admin permissions changed
PROJECT_CREATED	New project created
PROJECT_DELETED	Project deleted
PAYMENT_RECORDED	Payment recorded
DATA_EXPORTED	Data exported
BACKUP_CREATED	Backup created
BACKUP_RESTORED	Backup restored

33.7 Incident Response Plan

Table 33.5: Incident Severity Levels

Level	Name	Examples
1	Critical	Data breach, payment system compromise, full service outage
2	High	Partial outage, unauthorized access (contained), data corruption
3	Medium	Performance degradation, minor data inconsistency
4	Low	Sporadic errors, cosmetic issues

Chapter Summary

Table 33.6: Security Architecture — Key Specifications

Property	Value
Encryption in Transit	TLS 1.3, HSTS preload
Encryption at Rest	Server disk encryption (hosting)
Authentication	Email + Password + OTP
Session Security	HTTP-only, secure, SameSite=Lax
Rate Limiting	5 failures -> 15-minute lockout
Incident Response	4 severity levels, documented plan
Security Testing	Weekly dependency scans, monthly DAST, annual pen test

Chapter 34

Audit & Compliance

34.1 Overview

The Caldan Construction audit system provides complete, immutable tracking of every significant action across the platform. Every login, every payment, every project change—all logged with user identity, timestamp, IP address, and detailed context.

> **KEY INSIGHT** Core Principle: If an action happens in Caldan Construction, it is logged. If it is logged, it is traceable. If it is traceable, it is auditable.

34.2 AuditLog Model

```
from django.db import models
from django.conf import settings
from django.utils import timezone
import hashlib

class AuditLog(models.Model):
    """Immutable audit trail for all significant platform actions."""

    class ActionType(models.TextChoices):
        # Authentication
        LOGIN_SUCCESS = 'LOGIN_SUCCESS', 'Login Successful'
        LOGIN_FAILED = 'LOGIN_FAILED', 'Login Failed'
        LOGOUT = 'LOGOUT', 'Logout'
        PASSWORD_CHANGED = 'PASSWORD_CHANGED', 'Password Changed'
        OTP_SENT = 'OTP_SENT', 'OTP Sent'
        OTP_VERIFIED = 'OTP_VERIFIED', 'OTP Verified'
        OTP_FAILED = 'OTP_FAILED', 'OTP Failed'
        ACCOUNT_LOCKED = 'ACCOUNT_LOCKED', 'Account Locked'

        # User Management
        USER_CREATED = 'USER_CREATED', 'User Created'
        USER_UPDATED = 'USER_UPDATED', 'User Updated'
        USER_DELETED = 'USER_DELETED', 'User Deleted'
        PERMISSION_CHANGED = 'PERMISSION_CHANGED', 'Permission Changed'
        ADMIN_INVITATION_SENT = 'ADMIN_INVITATION_SENT', 'Admin Invitation Sent'
        ADMIN_ACCOUNT_CREATED = 'ADMIN_ACCOUNT_CREATED', 'Admin Account Created'

        # Projects
        PROJECT_CREATED = 'PROJECT_CREATED', 'Project Created'
        PROJECT_UPDATED = 'PROJECT_UPDATED', 'Project Updated'
        PROJECT_DELETED = 'PROJECT_DELETED', 'Project Deleted'
        PROJECT_ARCHIVED = 'PROJECT_ARCHIVED', 'Project Archived'
        PROJECT_RESTORED = 'PROJECT_RESTORED', 'Project Restored'
        PROJECT_BACKUP_CREATED = 'PROJECT_BACKUP_CREATED', 'Backup Created'
        PROJECT_BACKUP_RESTORED = 'PROJECT_BACKUP_RESTORED', 'Backup Restored'

        # Inventory
        MATERIAL_CREATED = 'MATERIAL_CREATED', 'Material Created'
        MATERIAL_UPDATED = 'MATERIAL_UPDATED', 'Material Updated'
        MATERIAL_DELETED = 'MATERIAL_DELETED', 'Material Deleted'
        STOCK_IN = 'STOCK_IN', 'Stock In'
        STOCK_OUT = 'STOCK_OUT', 'Stock Out'
        STOCK_ALERT = 'STOCK_ALERT', 'Stock Alert'
        STOCK_ALERT_RESOLVED = 'STOCK_ALERT_RESOLVED', 'Stock Alert Resolved'

        # Shop
        ORDER_PLACED = 'ORDER_PLACED', 'Order Placed'
        ORDER_STATUS_UPDATED = 'ORDER_STATUS_UPDATED', 'Order Status Updated'
        ORDER_CANCELLED = 'ORDER_CANCELLED', 'Order Cancelled'
```

```
        # Payments
        PAYMENT_RECORDED = 'PAYMENT_RECORDED', 'Payment Recorded'
        PAYMENT_APPROVED = 'PAYMENT_APPROVED', 'Payment Approved'
        PAYMENT_COMPLETED = 'PAYMENT_COMPLETED', 'Payment Completed'
        PAYMENT_CANCELLED = 'PAYMENT_CANCELLED', 'Payment Cancelled'
        EXPENSE_RECORDED = 'EXPENSE_RECORDED', 'Expense Recorded'

        # Reports
        REPORT_EXPORTED = 'REPORT_EXPORTED', 'Report Exported'
        DATA_EXPORTED = 'DATA_EXPORTED', 'Data Exported'

        # System
        SYSTEM_ALERT = 'SYSTEM_ALERT', 'System Alert'
        SETTINGS_CHANGED = 'SETTINGS_CHANGED', 'Settings Changed'
        AUDIT_REPORT_GENERATED = 'AUDIT_REPORT_GENERATED', 'Audit Report Generated'

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name='audit_logs'
    )
    user_role = models.CharField(max_length=20, blank=True, default='')
    action = models.CharField(max_length=50, choices=ActionType.choices, db_index=True)
    entity_type = models.CharField(max_length=50, blank=True, default='')
    entity_id = models.CharField(max_length=50, blank=True, default='')
    details = models.JSONField(default=dict)
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    user_agent = models.TextField(blank=True, default='')

    # Cryptographic hash chain for immutability
    hash_previous = models.CharField(max_length=64, default='0' * 64)
    hash_current = models.CharField(max_length=64, default='0' * 64)

    created_at = models.DateTimeField(default=timezone.now, db_index=True)

    class Meta:
        db_table = 'audit_logs'
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['user', '-created_at']),
            models.Index(fields=['action', '-created_at']),
        ]

    def save(self, *args, **kwargs):
        if not self.hash_current or self.hash_current == '0' * 64:
            last_log = AuditLog.objects.order_by('-created_at').first()
            self.hash_previous = last_log.hash_current if last_log else '0' * 64

            hash_data = (
                f"{self.user_id or ''}"
                f"{self.action}"
                f"{self.entity_type}"
                f"{self.entity_id}"
                f"{self.created_at.isoformat()}"
                f"{self.hash_previous}"
            )
            self.hash_current = hashlib.sha256(hash_data.encode()).hexdigest()

        super().save(*args, **kwargs)
```

Listing 34.1: AuditLog Model – accounts/models/audit.py

34.3 Audit Service

```
class AuditService:
    """Centralized audit logging service."""

    @staticmethod
    def log_action(user, action, entity_type='', entity_id='', details=None, request=None):
        """Log an action to the immutable audit trail."""
        ip_address = None
        user_agent = ''

        if request:
            ip_address = AuditService._get_client_ip(request)
            user_agent = request.META.get('HTTP_USER_AGENT', '')[:500]

        return AuditLog.objects.create(
            user=user,
            user_role=user.role if user else 'SYSTEM',
            action=action,
            entity_type=entity_type,
            entity_id=str(entity_id) if entity_id else '',
            details=details or {},
```

```
            ip_address=ip_address,
            user_agent=user_agent,
        )

    @staticmethod
    def _get_client_ip(request):
        x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
        if x_forwarded_for:
            return x_forwarded_for.split(',')[0].strip()
        return request.META.get('REMOTE_ADDR')

    @staticmethod
    def verify_chain_integrity():
        """Verify the cryptographic hash chain hasn't been tampered with."""
        logs = AuditLog.objects.order_by('created_at')
        previous_hash = '0' * 64

        for log in logs:
            if log.hash_previous != previous_hash:
                return False, f"Chain broken at audit log ID {log.id}"

            hash_data = (
                f"{log.user_id or ''}"
                f"{log.action}"
                f"{log.entity_type}"
                f"{log.entity_id}"
                f"{log.created_at.isoformat()}"
                f"{log.hash_previous}"
            )
            calculated = hashlib.sha256(hash_data.encode()).hexdigest()

            if calculated != log.hash_current:
                return False, f"Hash mismatch at audit log ID {log.id}"

            previous_hash = log.hash_current

        return True, "Audit trail integrity verified"
```

Listing 34.2: Audit Service – accounts/services/audit-service.py

Chapter Summary

Table 34.1: Audit & Compliance — Key Specifications

Property	**Value**
Action Types	30+
Hash Algorithm	SHA-256 (cryptographic chain)
Immutability	Append-only with hash chaining
Fields Logged	User, role, action, entity, details, IP, user agent, hash
Bi-Weekly Audit Report	PDF emailed every 14 days
Export Formats	CSV and PDF
Retention Period	365 days

Chapter 35

Database Schema

35.1 Overview

The Caldan Construction database is designed on MySQL 8.0+ using the InnoDB storage engine for full transaction support, foreign key integrity, and row-level locking.

KEY INSIGHT Design Philosophy: Every table includes created_at and updated_at timestamps. Every foreign key uses appropriate on_delete behavior. All tables use utf8mb4 character set.

35.2 Entity-Relationship Overview

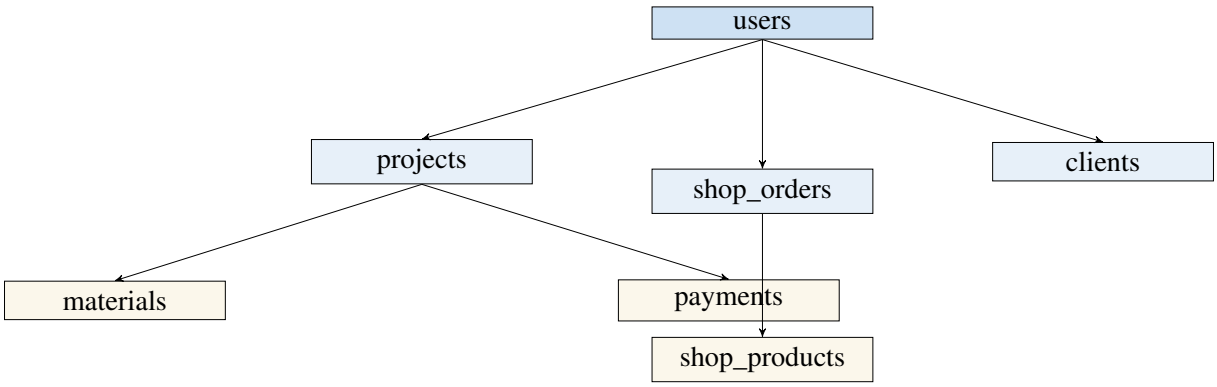


Figure 35.1: High-Level Entity Relationship Diagram

35.3 Core Tables

35.3.1 Users Table

Table 35.1: users Table

Column	Type	Description
id	INT	Primary key, AUTO_INCREMENT
email	VARCHAR(254)	Unique, NOT NULL, INDEX
password	VARCHAR(128)	NOT NULL (bcrypt hash)
full_name	VARCHAR(150)	NOT NULL
role	VARCHAR(20)	NOT NULL, INDEX
phone	VARCHAR(15)	NOT NULL
is_active	BOOLEAN	DEFAULT TRUE
is_staff	BOOLEAN	DEFAULT FALSE
is_superuser	BOOLEAN	DEFAULT FALSE
is_chief	BOOLEAN	DEFAULT FALSE
is_email_verified	BOOLEAN	DEFAULT FALSE
otp_code_hash	VARCHAR(64)	NULLABLE
otp_created_at	DATETIME	NULLABLE
otp_attempts	SMALLINT	DEFAULT 0
otp_resend_count	SMALLINT	DEFAULT 0
otp_last_resent	DATETIME	NULLABLE
trusted_devices	JSON	DEFAULT {}
failed_login_attempts	SMALLINT	DEFAULT 0
locked_until	DATETIME	NULLABLE
last_login	DATETIME	NULLABLE
date_joined	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	AUTO_UPDATE

35.3.2 Projects Table

Table 35.2: projects Table

Column	Type	Description
id	INT	Primary key, AUTO_INCREMENT
name	VARCHAR(200)	NOT NULL, INDEX
project_code	VARCHAR(20)	UNIQUE, NOT NULL
description	TEXT	DEFAULT ”
location	VARCHAR(200)	NOT NULL
county	VARCHAR(100)	NOT NULL, INDEX
town	VARCHAR(100)	NOT NULL, INDEX
latitude	DECIMAL(9,6)	NULLABLE
longitude	DECIMAL(9,6)	NULLABLE
start_date	DATE	NOT NULL
end_date	DATE	NULLABLE
estimated_completion	DATE	NULLABLE
actual_completion	DATE	NULLABLE
budget	DECIMAL(15,2)	DEFAULT 0
spent	DECIMAL(15,2)	DEFAULT 0
progress	DECIMAL(5,2)	DEFAULT 0
status	VARCHAR(20)	NOT NULL, INDEX
is_archived	BOOLEAN	DEFAULT FALSE, INDEX
archived_at	DATETIME	NULLABLE
cover_photo	VARCHAR(255)	NULLABLE
created_at	DATETIME	AUTO_NOW_ADD
updated_at	DATETIME	AUTO_UPDATE

35.3.3 Materials Table

Table 35.3: materials Table

Column	Type	Description
id	INT	Primary key, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL, INDEX
category	VARCHAR(20)	NOT NULL, INDEX
unit	VARCHAR(20)	NOT NULL
description	TEXT	DEFAULT ”
current_quantity	DECIMAL(10,2)	DEFAULT 0
minimum_threshold	DECIMAL(10,2)	DEFAULT 0
reorder_quantity	DECIMAL(10,2)	DEFAULT 0
unit_cost	DECIMAL(10,2)	DEFAULT 0
total_cost	DECIMAL(15,2)	DEFAULT 0
supplier_id	INT	FK to suppliers, NULLABLE
sku	VARCHAR(50)	DEFAULT ”
is_active	BOOLEAN	DEFAULT TRUE
is_low_stock	BOOLEAN	DEFAULT FALSE, INDEX
is_available_in_shop	BOOLEAN	DEFAULT FALSE
shop_price	DECIMAL(10,2)	NULLABLE
created_at	DATETIME	AUTO_NOW_ADD
updated_at	DATETIME	AUTO_UPDATE

35.3.4 Shop Orders Table

Table 35.4: shop_orders Table

Column	Type	Description
id	INT	Primary key, AUTO_INCREMENT
order_number	VARCHAR(20)	UNIQUE, NOT NULL, INDEX
client_name	VARCHAR(150)	NOT NULL
client_email	VARCHAR(254)	NOT NULL
client_phone	VARCHAR(15)	NOT NULL
delivery_address	TEXT	NOT NULL
delivery_county	VARCHAR(100)	NOT NULL
delivery_town	VARCHAR(100)	NOT NULL
subtotal	DECIMAL(12,2)	NOT NULL
delivery_fee	DECIMAL(10,2)	DEFAULT 0
total	DECIMAL(12,2)	NOT NULL
status	VARCHAR(20)	NOT NULL, INDEX
paystack_reference	VARCHAR(100)	DEFAULT ”
paid_at	DATETIME	NULLABLE
tracking_number	VARCHAR(100)	DEFAULT ”
estimated_delivery	DATE	NULLABLE
notes	TEXT	DEFAULT ”
admin_notes	TEXT	DEFAULT ”
created_at	DATETIME	AUTO_NOW_ADD
updated_at	DATETIME	AUTO_UPDATE

35.4 Global Indexes

Table 35.5: Critical Database Indexes

Index	Purpose
users_email_idx	Fast login lookup by email
users_role_idx	Filter users by role
projects_status_idx	Filter projects by status
projects_county_town_idx	Location-based filtering
materials_category_idx	Filter materials by category
materials_low_stock_idx	Quick low stock query
shop_orders_status_idx	Filter orders by status
shop_orders_email_idx	Find orders by client email
audit_user_action_idx	Audit log filtering
audit_created_idx	Date range queries

Chapter Summary

Table 35.6: Database Schema — Key Specifications

Property	Value
Database Engine	MySQL 8.0+ (InnoDB)
Character Set	utf8mb4
Total Tables	25+
Foreign Key Relationships	30+
JSON Fields	5+
Indexed Columns	35+
Migration System	Django migrations

Chapter 36

API Architecture

36.1 Overview

The Caldan Construction API provides programmatic access to platform data and functionality. It serves future mobile applications and enables third-party integrations.

KEY INSIGHT Design Philosophy: The API is built for the mobile app first, but designed for extensibility. Every endpoint is versioned, documented with OpenAPI, and secured with JWT authentication.

36.2 Technology Stack

Table 36.1: API Technology Stack

Component	Technology	Purpose
API Framework	Django REST Framework 3.14+	API endpoints, serialization
Authentication	JWT (Simple JWT)	Token-based authentication
Documentation	drf-spectacular (OpenAPI 3.0)	Swagger UI + Redoc
Serialization	DRF Serializers	Data validation, transformation
Pagination	PageNumberPagination	20 items per page
Rate Limiting	DRF throttling	1000 requests/hour per user

36.3 Authentication

```
from datetime import timedelta

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
        'rest_framework.authentication.SessionAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/day',
        'user': '1000/hour',
```

```
        'burst': '10/min',
    },
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 20,
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(hours=1),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
    'ALGORITHM': 'HS256',
    'SIGNING_KEY': SECRET_KEY,
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

Listing 36.1: JWT Configuration – settings.py

36.4 API Endpoints Reference

Table 36.2: API Endpoints Reference

Endpoint	Method	Purpose	Auth
/api/auth/login/	POST	Login with OTP	None
/api/auth/verify-otp/	POST	Verify 8-digit OTP	None
/api/auth/refresh/	POST	Refresh JWT	None
/api/auth/logout/	POST	Logout	JWT
/api/v1/projects/	GET	List projects	JWT
/api/v1/projects/	POST	Create project	JWT
/api/v1/projects/<id>/	GET	Get project	JWT
/api/v1/projects/<id>/	PUT	Update project	JWT
/api/v1/projects/<id>/	DELETE	Delete project	JWT
/api/v1/inventory/materials/	GET	List materials	JWT
/api/v1/inventory/materials/	POST	Create material	JWT
/api/v1/inventory/stock-in/	POST	Record stock in	JWT
/api/v1/inventory/stock-out/	POST	Record stock out	JWT
/api/v1/inventory/alerts/	GET	List alerts	JWT
/api/v1/shop/products/	GET	List products	None
/api/v1/shop/products/<id>/	GET	Get product	None
/api/v1/shop/orders/	GET	List orders	JWT
/api/v1/shop/orders/	POST	Create order	None
/api/v1/shop/orders/<id>/	GET	Get order	None
/api/v1/reports/project/<id>/	GET	Project report	JWT
/api/v1/reports/inventory/	GET	Inventory report	JWT

36.5 Error Handling

```
{
    "error": "Validation error",
    "code": "VALIDATION_ERROR",
    "timestamp": "2026-07-02T10:30:00Z",
    "details": {
        "quantity": ["Quantity must be greater than zero"]
    }
}
```

Listing 36.2: API Error Response Example

Chapter Summary

Table 36.3: API Architecture — Key Specifications

Property	**Value**
Framework	Django REST Framework 3.14+
Authentication	JWT (access: 1hr, refresh: 7 days)
Documentation	OpenAPI 3.0 (Swagger UI + Redoc)
Rate Limiting	User: 1000/hour, Anonymous: 100/day
Pagination	20 items per page
Versioning	URL path (/api/v1/, /api/v2/)

Chapter 37

Testing Strategy

37.1 Overview

Caldan Construction requires rigorous testing to ensure reliability, security, and correctness. Tests are written alongside production code, and the build fails if tests fail.

> **KEY INSIGHT** Testing Philosophy: Tests are not an afterthought—they are written alongside production code. Every new feature includes tests. Every bug fix includes a regression test.

37.2 Testing Pyramid

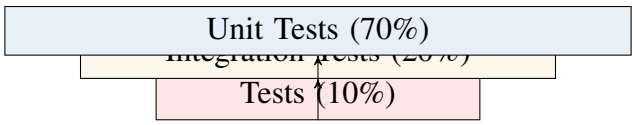


Figure 37.1: Testing Pyramid

37.3 Coverage Requirements

Table 37.1: Minimum Code Coverage by Module

Module	Target	Critical Paths
Core	90%	Base models, context processors
Accounts	85%	User model, authentication, OTP
Projects	80%	CRUD, progress, milestones
Inventory	85%	Stock IN/OUT, alerts
Shop	85%	Cart, checkout, orders
Finances	90%	Payment recording, budgets
Reports	70%	Data aggregation, exports

37.4 Unit Test Examples

```
from django.test import TestCase
from django.utils import timezone
from datetime import timedelta
from accounts.services.otp_service import OTPService
from accounts.models.user import User

class OTPServiceTests(TestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            email='test@example.com',
            password='testpass123',
            full_name='Test User',
            phone='0712345678'
        )

    def test_generate_otp(self):
        """OTP should be 8 digits and stored as hash."""
        otp = OTPService.generate_otp(self.user)

        self.assertEqual(len(otp), 8)
        self.assertTrue(otp.isdigit())
        self.assertIsNotNone(self.user.otp_code_hash)
        self.assertIsNotNone(self.user.otp_created_at)

    def test_verify_otp_success(self):
        """Valid OTP should verify successfully."""
        otp = OTPService.generate_otp(self.user)

        success, message = OTPService.verify_otp(self.user, otp)
        self.assertTrue(success)
        self.assertEqual(message, "OTP verified")
        self.assertTrue(self.user.is_email_verified)

    def test_verify_otp_expired(self):
        """Expired OTP should fail."""
        otp = OTPService.generate_otp(self.user)
        self.user.otp_created_at = timezone.now() - timedelta(minutes=6)
        self.user.save()

        success, message = OTPService.verify_otp(self.user, otp)
        self.assertFalse(success)
        self.assertEqual(message, "OTP expired")

    def test_verify_otp_max_attempts(self):
        """Max attempts should lock OTP."""
        otp = OTPService.generate_otp(self.user)

        for _ in range(3):
            OTPService.verify_otp(self.user, '00000000')

        success, message = OTPService.verify_otp(self.user, otp)
        self.assertFalse(success)
        self.assertEqual(message, "Too many attempts")
```

Listing 37.1: OTP Service Tests – accounts/tests/test-otp-service.py

37.5 Integration Tests

```
from rest_framework.test import APITestCase
from rest_framework import status
from django.urls import reverse
from accounts.models.user import User

class ProjectAPITests(APITestCase):

    def setUp(self):
        self.user = User.objects.create_user(
            email='admin@example.com',
            password='testpass123',
            full_name='Test Admin',
            role='ADMIN',
            phone='0712345678'
        )
        self.client.force_authenticate(user=self.user)

    def test_list_projects(self):
        """Authenticated admin can list projects."""
        url = reverse('api:project-list')
        response = self.client.get(url)
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_create_project(self):
        """POST /api/projects/ creates new project."""
        url = reverse('api:project-list')
        data = {
```

```
            'name': 'Test Project',
            'location': 'Nairobi',
            'county': 'Nairobi',
            'town': 'Nairobi',
            'start_date': '2026-01-15',
            'budget': '1000000.00'
        }
        response = self.client.post(url, data, format='json')
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertEqual(response.data['name'], 'Test Project')

    def test_create_project_unauthenticated(self):
        """Unauthenticated requests should return 401."""
        self.client.force_authenticate(user=None)
        url = reverse('api:project-list')
        response = self.client.get(url)
        self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)
```

Listing 37.2: API Integration Tests – api/tests/test-projects.py

37.6 End-to-End Tests (Playwright)

```
import { test, expect } from '@playwright/test';

test.describe('Login Flow', () => {
    test('Admin can login with OTP', async ({ page }) => {
        // Navigate to login page
        await page.goto('https://caldanconstruction.com/login');
        await expect(page).toHaveTitle(/Login/);

        // Fill credentials
        await page.fill('input[name="email"]', 'admin@example.com');
        await page.fill('input[name="password"]', 'testpass123');
        await page.click('button[type="submit"]');

        // Wait for OTP page
        await expect(page).toHaveURL(/.*\/verify-otp/);
        await expect(page.locator('.otp-input-group')).toBeVisible();

        // Enter OTP (simulated)
        await page.fill('input[name="otp_code"]', '12345678');
        await page.click('button[type="submit"]');

        // Verify dashboard access
        await expect(page).toHaveURL(/.*\/dashboard/);
        await expect(page.locator('.welcome-banner')).toBeVisible();
    });
});
```

Listing 37.3: Login Flow E2E – tests/e2e/login.spec.js

37.7 Continuous Integration

```
name: Caldan Construction CI

on:
    push:
        branches: [ main, develop ]
    pull_request:
        branches: [ main ]

jobs:
    test:
        runs-on: ubuntu-latest

        services:
            mysql:
                image: mysql:8.0
                env:
                    MYSQL_ROOT_PASSWORD: root
                    MYSQL_DATABASE: caldan_test
                ports:
                    - 3306:3306

        steps:
            - uses: actions/checkout@v4

            - name: Set up Python
              uses: actions/setup-python@v5
```

```
        with:
            python-version: '3.11'

    - name: Install dependencies
      run: |
          pip install -r requirements.txt
          pip install pytest pytest-django pytest-cov

    - name: Run tests with coverage
      run: |
          pytest --cov=. --cov-report=xml --junitxml=test-results.xml

    - name: Upload coverage
      uses: codecov/codecov-action@v3
      with:
          file: ./coverage.xml

    - name: Run security scan
      run: bandit -r . -ll -f json -o reports/bandit.json

    - name: Run Django checks
      run: python manage.py check --deploy
```

Listing 37.4: GitHub Actions CI – .github/workflows/ci.yml

Chapter Summary

Table 37.2: Testing Strategy — Key Specifications

Property	**Value**
Coverage Target	80% overall, 90% for critical paths
Unit Tests	Django TestCase
Integration Tests	API endpoints, database transactions
E2E Tests	Playwright (critical user journeys)
Load Testing	Locust (100 concurrent users)
Security Testing	Bandit (SAST), OWASP ZAP (DAST)
CI Pipeline	GitHub Actions

Chapter 38

Deployment & DevOps

38.1 Overview

The Caldan Construction deployment pipeline automates the process of moving code from development to production with zero downtime, automated testing, and rollback capabilities.

> **KEY INSIGHT** DevOps Philosophy: Infrastructure is code. Every server configuration, environment variable, and deployment step is documented, version-controlled, and repeatable.

38.2 Deployment Environments

Table 38.1: Deployment Environments

Environment	Purpose	Deployment Frequency
Local	Developer testing	On demand
Development	Integration testing	Every push to develop
Staging	QA, UAT, client preview	Every merge to main
Production	Live users	On release (manual approval)

38.3 Docker Configuration

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV DJANGO_SETTINGS_MODULE=caldan_site.settings

RUN apt-get update && apt-get install -y \
    gcc \
    libmariadb-dev \
    libpango-1.0-0 \
    libcairo2 \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
RUN python manage.py collectstatic --noinput

RUN useradd --create-home caldan && chown -R caldan:caldan /app
USER caldan
```

```
EXPOSE 8000

CMD ["gunicorn", "--bind", "0.0.0.0:8000", "caldan_site.wsgi:application"]
```

Listing 38.1: Dockerfile

38.4 Deployment Script

```
#!/bin/bash
set -e

ENVIRONMENT=$1
if [[ "$ENVIRONMENT" != "staging" && "$ENVIRONMENT" != "production" ]]; then
    echo "Usage: ./deploy.sh staging|production"
    exit 1
fi

echo "Deploying to $ENVIRONMENT..."

# Pull latest image
docker pull caldan/caldan-construction:latest

# Backup database
echo "Backing up database..."
ssh root@$DOMAIN "mysqldump caldan_construction | gzip > /backups/pre_deploy_$(date +%Y%m%d_%H%M%S).sql.gz"

# Run migrations
docker-compose -f docker-compose.$ENVIRONMENT.yml run --rm web python manage.py migrate

# Collect static files
docker-compose -f docker-compose.$ENVIRONMENT.yml run --rm web python manage.py collectstatic --noinput

# Restart web server
docker-compose -f docker-compose.$ENVIRONMENT.yml restart web

# Health check
sleep 10
if curl -s --head "https://$DOMAIN/health/" | grep "200 OK" > /dev/null; then
    echo "PASS: Health check passed"
else
    echo "FAIL: Health check failed -- rolling back"
    docker-compose -f docker-compose.$ENVIRONMENT.yml down
    docker-compose -f docker-compose.$ENVIRONMENT.yml up -d
    exit 1
fi

echo "Deployment to $ENVIRONMENT complete!"
```

Listing 38.2: Deployment Script – scripts/deploy.sh

38.5 Health Check Endpoint

```
from django.http import JsonResponse
from django.db import connections
from django.db.utils import OperationalError
import redis
from celery import current_app

def health_check(request):
    status = {
        'status': 'healthy',
        'timestamp': timezone.now().isoformat(),
        'checks': {}
    }

    # Database check
    try:
        with connections['default'].cursor() as cursor:
            cursor.execute("SELECT 1")
        status['checks']['database'] = 'healthy'
    except OperationalError:
        status['checks']['database'] = 'unhealthy'
        status['status'] = 'unhealthy'

    # Redis check
    try:
        r = redis.from_url(settings.REDIS_URL)
        r.ping()
```

```
        status['checks']['redis'] = 'healthy'
    except Exception:
        status['checks']['redis'] = 'unhealthy'
        status['status'] = 'unhealthy'

    # Celery check
    try:
        inspect = current_app.control.inspect()
        if inspect.ping():
            status['checks']['celery'] = 'healthy'
        else:
            status['checks']['celery'] = 'unhealthy'
            status['status'] = 'unhealthy'
    except Exception:
        status['checks']['celery'] = 'unhealthy'
        status['status'] = 'unhealthy'

    http_status = 200 if status['status'] == 'healthy' else 503
    return JsonResponse(status, status=http_status)
```

Listing 38.3: Health Check – caldan-site/urls.py

38.6 Rollback Strategy

```
#!/bin/bash
set -e

VERSION=$1
if [ -z "$VERSION" ]; then
    echo "Usage: ./rollback.sh <version-tag>"
    echo "Available versions:"
    docker images caldan/caldan-construction --format "table {{.Tag}}"
    exit 1
fi

echo "Rolling back to version $VERSION..."

# Stop current containers
docker-compose -f docker-compose.prod.yml down

# Revert to previous image
docker tag caldan/caldan-construction:$VERSION caldan/caldan-construction:latest

# Restore database from pre-deploy backup
BACKUP_FILE=$(ls -t /backups/pre_deploy_*.sql.gz | head -1)
echo "Restoring database from $BACKUP_FILE..."
gunzip -c $BACKUP_FILE | mysql caldan_construction

# Start containers
docker-compose -f docker-compose.prod.yml up -d

# Verify rollback
sleep 10
if curl -s --head "https://caldanconstruction.com/health/" | grep "200 OK" > /dev/null; then
    echo "PASS: Rollback successful"
else
    echo "FAIL: Rollback failed -- manual intervention required"
    exit 1
fi
```

Listing 38.4: Rollback Script – scripts/rollback.sh

38.7 Monitoring Metrics

Table 38.2: Production Monitoring Metrics

Metric	Source	Alert Threshold
CPU Usage	DigitalOcean metrics	>80% for 10 minutes
Memory Usage	DigitalOcean metrics	>85% for 5 minutes
Error Rate	Django logs	>1% of requests for 5 minutes
Response Time	New Relic	>3s for 95th percentile
Payment Success	Paystack logs	<95% for 15 minutes
SMS Delivery	SMS provider logs	<90% for 1 hour
Email Bounce	Resend webhook	>5% for 1 hour

Chapter Summary

Table 38.3: Deployment & DevOps — Key Specifications

Property	Value
Containerization	Docker
Orchestration	Docker Compose (staging), Docker Swarm (future)
CI/CD	GitHub Actions -> Docker Hub -> Railway
Environments	4 (Local, Development, Staging, Production)
Monitoring	Health checks, DigitalOcean metrics, Sentry
Rollback Time	<10 minutes (automated script)
Deployment Frequency	On demand (multiple times per day)

Final Statement

FINAL STATEMENT

This Version 1.0 specification is the complete technical blueprint for Caldan Construction.

*It covers **12 Django applications**, **80+ dashboard pages**, **30+ service classes**, **25+ database models**, **8 external integrations**, and a **phased implementation roadmap** spanning **26+ weeks** to production handover.*

Every feature, every model, every view, and every decision documented here is the product of exhaustive analysis of the Kenyan construction industry—an industry that has long relied on paper records, verbal communication, and manual tracking.

The gap between what existed and what is possible is now bridged.

Dr. Daniel Ogwoka Siringi
Director, Caldan Builder

Mukoya Khisa
Website Developer & Manager
Caldan Construction
Nairobi, Kenya
July 2026

"Building Kenya's Future, One Project at a Time."

Appendix A

Modern Visual Atlas

This appendix is no longer a decorative gallery. It keeps only the diagrams that add architectural or operational value and removes the repeated placeholder-style visuals. The detailed chapters already carry their own contextual figures; this appendix is now a concise reference set for the core system views.

A.1 Strategic and Brand Context

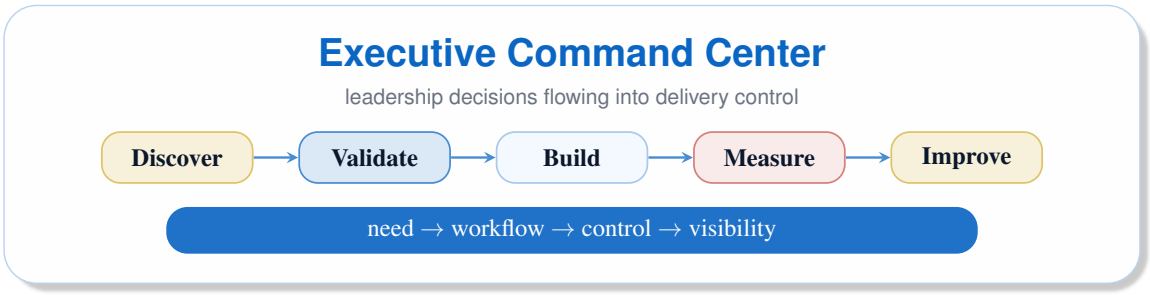


Figure A.1: Executive Command Center — leadership decisions flowing into delivery control

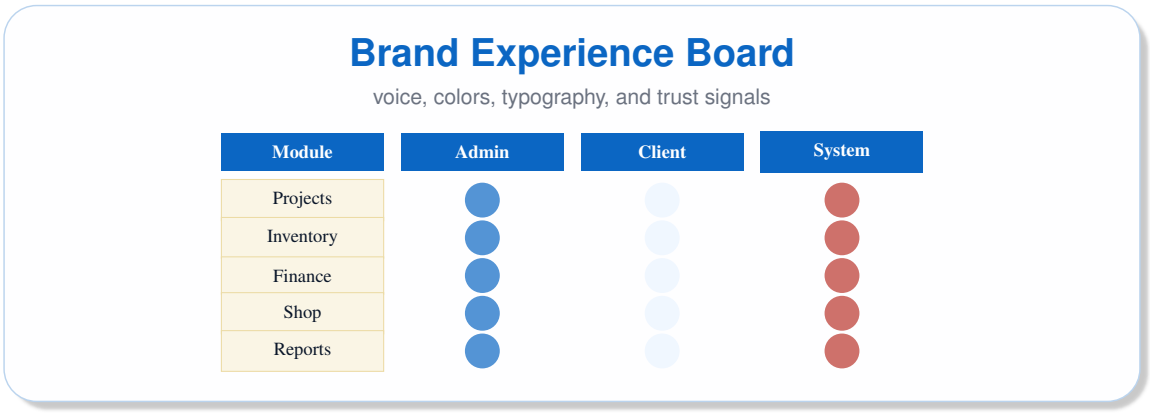


Figure A.2: Brand Experience Board — voice, colors, typography, and trust signals

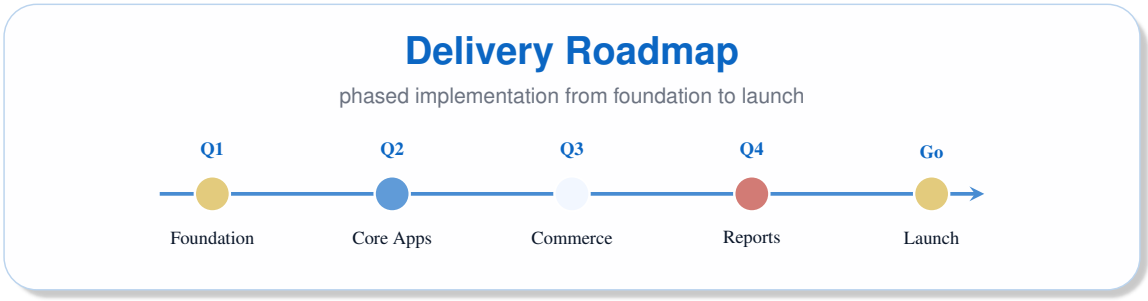


Figure A.3: Delivery Roadmap — phased implementation from foundation to launch

A.2 Platform Architecture

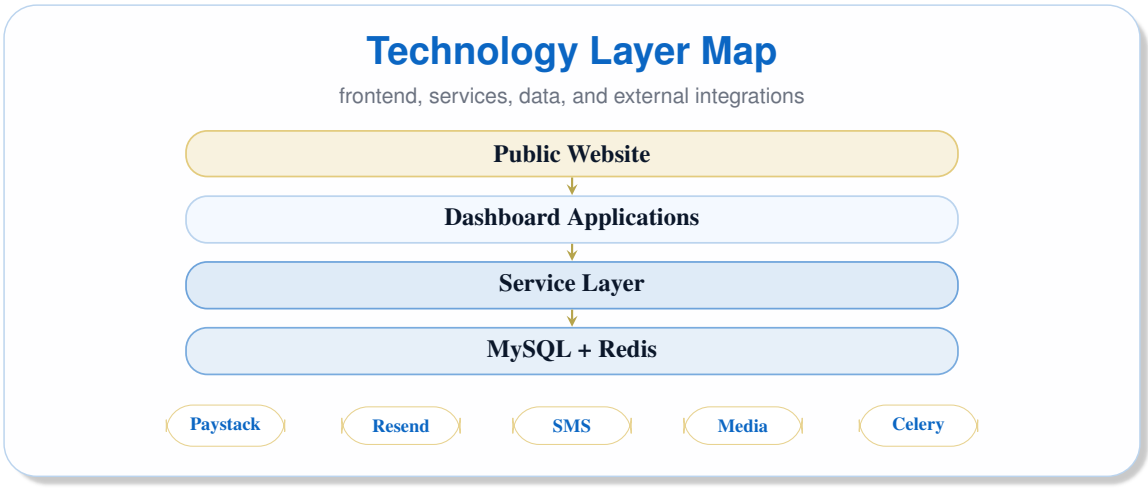


Figure A.4: Technology Layer Map — frontend, services, data, and external integrations

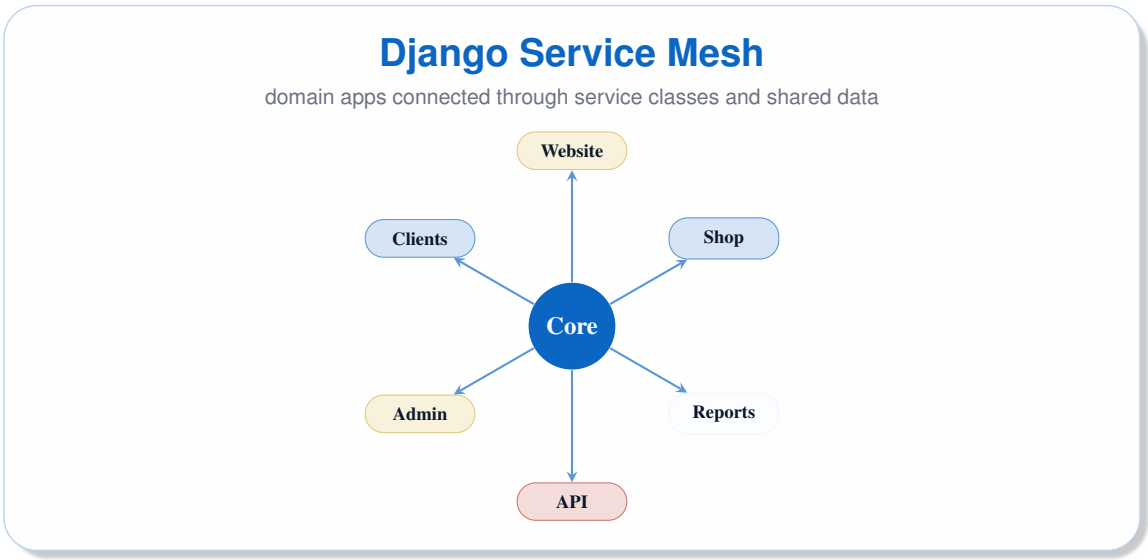


Figure A.5: Django Service Mesh — domain apps connected through service classes and shared data

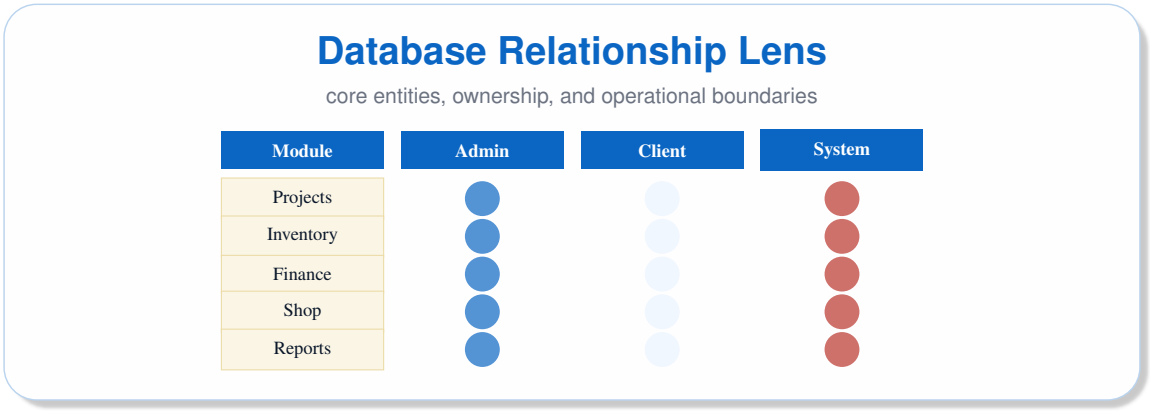


Figure A.6: Database Relationship Lens — core entities, ownership, and operational boundaries

A.3 Security and Governance

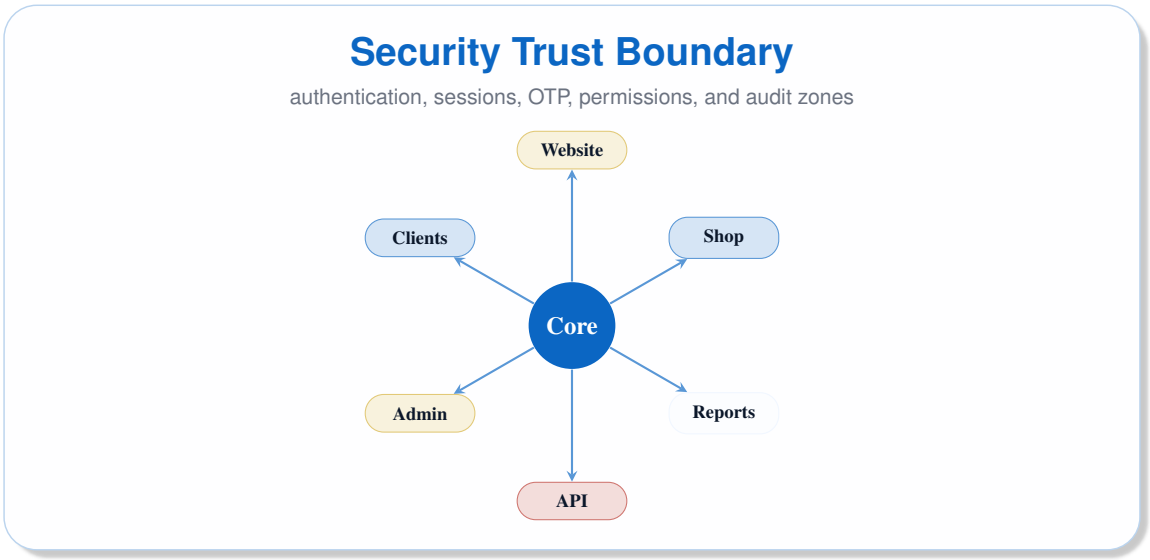


Figure A.7: Security Trust Boundary — authentication, sessions, OTP, permissions, and audit zones

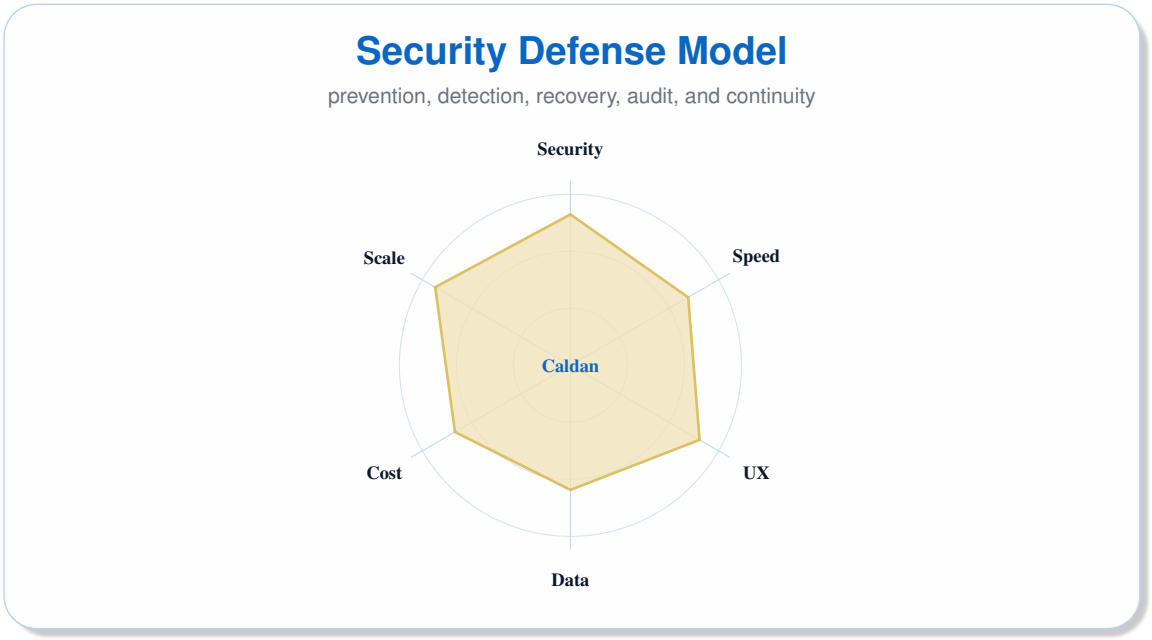


Figure A.8: Security Defense Model — prevention, detection, recovery, audit, and continuity

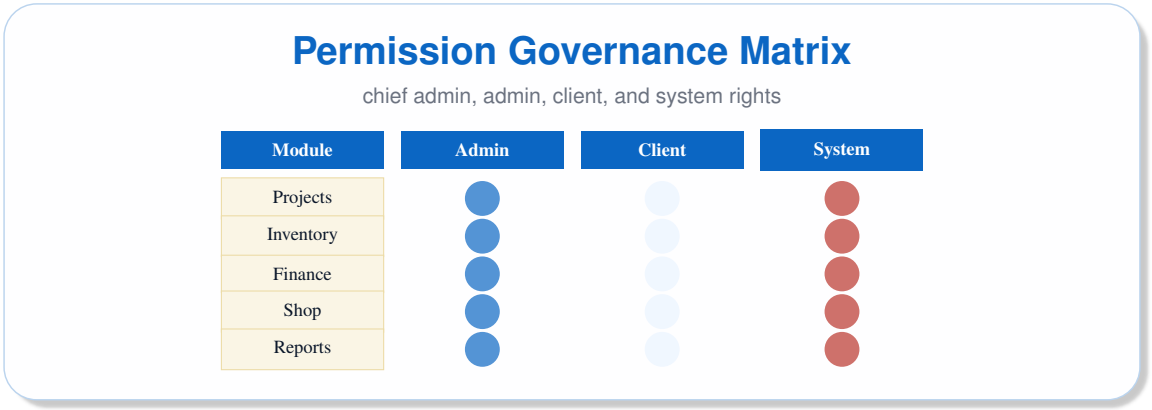


Figure A.9: Permission Governance Matrix — chief admin, admin, client, and system rights

A.4 Operations and Finance



Figure A.10: Inventory Control Tower — materials, thresholds, stock movement, and alerts



Figure A.11: Expense Review Board — expense capture, categorization, approval, reporting

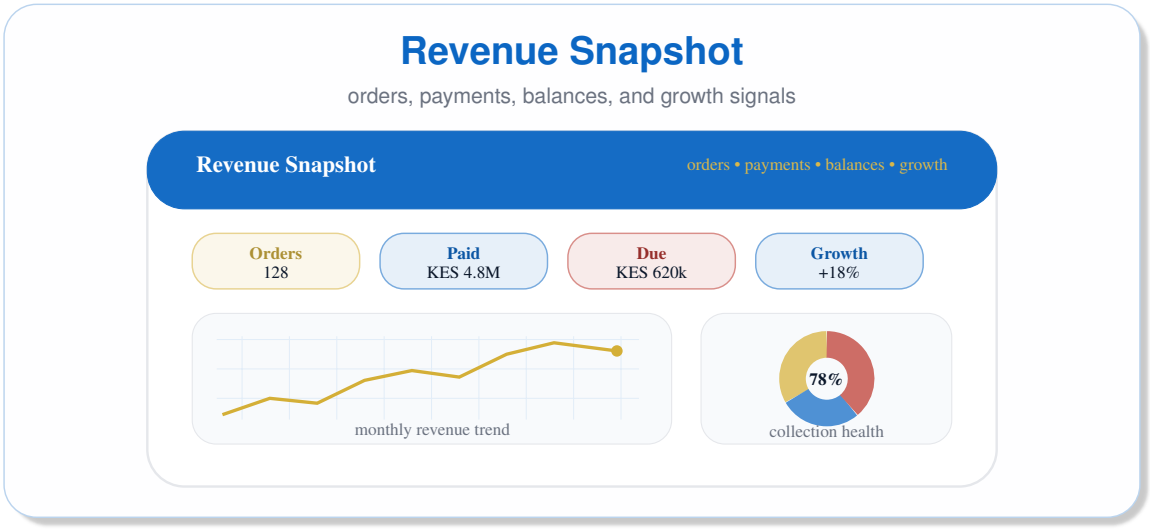


Figure A.12: Revenue Snapshot — orders, payments, balances, and growth signals

A.5 Delivery and Launch

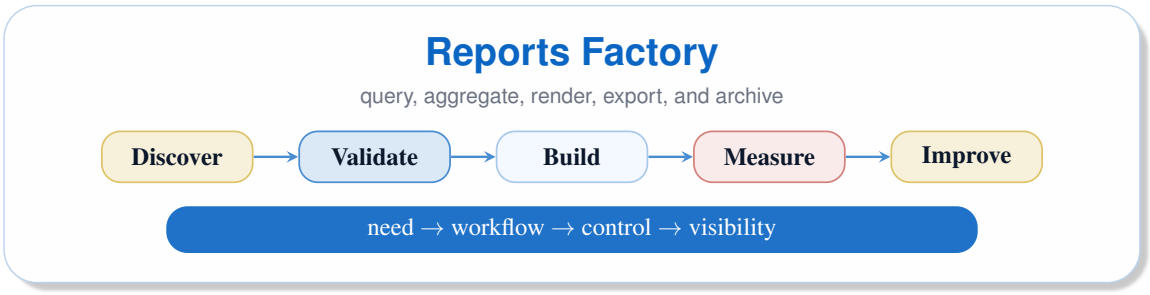


Figure A.13: Reports Factory — query, aggregate, render, export, and archive



Figure A.14: Deployment Runway — environment setup through production handover

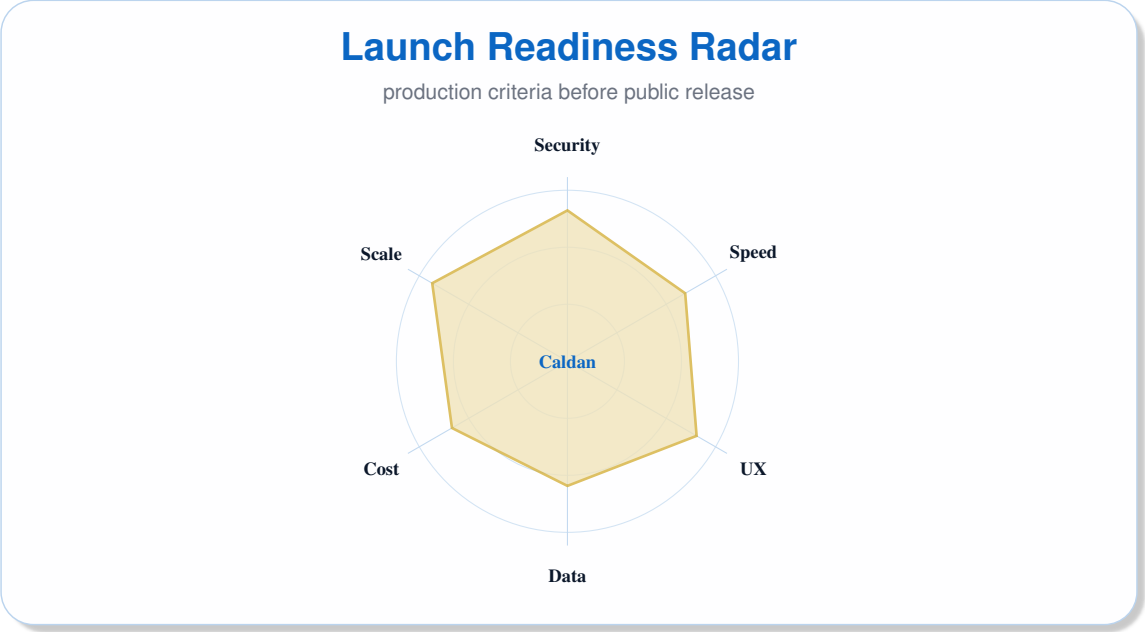


Figure A.15: Launch Readiness Radar — production criteria before public release

Appendix B

Complete Project Directory

B.1 Overview

This appendix provides the complete project directory structure for Caldan Construction, with file-by-file descriptions. The structure is modular, service-oriented, and follows the Salvins PMP pattern.

```
caldan_site/                                    # PROJECT ROOT
|
|-- .env                                        # Environment variables
|-- .gitignore                                  # Git ignore rules
|-- README.md                                   # Project overview
|-- manage.py                                   # Django management runner
|-- requirements.txt                            # All Python dependencies
|
|-- caldan_site/                                # PROJECT CONFIGURATION
|   |-- __init__.py
|   |-- settings.py                             # Django settings
|   |-- urls.py                                 # Root URL routing
|   |-- wsgi.py                                 # WSGI entry point
|   |-- asgi.py                                 # ASGI entry point
|   |-- celery.py                               # Celery configuration
|   `-- db_router.py                            # Database router
|
|-- accounts/                                   # AUTHENTICATION APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- user.py                             # Custom User (is_chief, is_superuser)
|   |   |-- profile.py                          # Admin profiles (permissions, theme)
|   |   |-- invitation.py                       # Secure invitation links (one-time, 48hr)
|   |   |-- session.py                          # Active session tracking
|   |   `-- audit.py                            # Immutable audit log (SHA-256 hash chain)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   |-- _setup_views.py                     # First-time setup URL (ONE-TIME USE)
|   |   |-- _auth_views.py                      # Login, OTP verification, logout
|   |   |-- _invitation_views.py                # Send/accept invitations
|   |   |-- _admin_views.py                     # Manage admins (list, delete, permissions)
|   |   |-- _profile_views.py                   # Profile management
|   |   `-- _password_views.py                  # Password reset, change password
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   |-- setup_form.py
|   |   |-- auth_form.py                        # LoginForm, OTPForm
|   |   |-- invitation_form.py
|   |   |-- profile_form.py
|   |   `-- password_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- auth_service.py                     # Login, logout, session
|   |   |-- otp_service.py                      # 8-digit OTP (5min expiry, 3 attempts)
|   |   |-- invitation_service.py               # Create/validate/accept invitations
|   |   |-- audit_service.py                    # Immutable audit logging
|   |   `-- permission_service.py               # Admin permission management
|   |
|   |-- middleware/
|   |   |-- __init__.py
|   |   |-- auth_middleware.py
|   |   |-- otp_middleware.py
|   |   `-- force_password_change_middleware.py
|   |
|   |-- management/
|   |   `-- commands/
|   |       |-- __init__.py
|   |       |-- create_chief_admin.py
|   |       `-- disable_setup.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   |-- 0001_initial.py
|   |   |-- 0002_user_is_chief.py
|   |   `-- 0003_admin_permissions.py
|   |
|   |-- templates/
|   |   `-- accounts/
|   |       |-- setup/
|   |       |   |-- setup.html
|   |       |   `-- setup_complete.html
|   |       |-- shared/
```

```
|   |       |   |-- login.html
|   |       |   |-- otp_verify.html
|   |       |   |-- change_password.html
|   |       |   |-- password_reset.html
|   |       |   |-- password_reset_confirm.html
|   |       |   `-- password_reset_complete.html
|   |       `-- admin/
|   |           |-- manage_admins.html              # (Sidebar Page)
|   |           |-- invite_admin.html               # (Sidebar Page)
|   |           |-- admin_detail.html               # (Sidebar Page)
|   |           `-- admin_permissions.html          # (Sidebar Page)
|   |
|   `-- tests/
|       |-- __init__.py
|       |-- test_setup.py
|       |-- test_auth_service.py
|       |-- test_invitation_service.py
|       `-- test_audit_service.py
|
|-- website/                                        # PUBLIC WEBSITE APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |-- context_processors.py
|   |-- sitemaps.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- contact.py                              # ContactSubmissions
|   |   |-- testimonial.py                          # Testimonials
|   |   `-- page.py                                 # Static page content
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   |-- _public_views.py                        # Home, About, Services, Contact
|   |   `-- _project_views.py                       # Public project gallery
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   |-- contact_form.py
|   |   `-- quote_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   `-- lead_service.py
|   |
|   |-- templatetags/
|   |   |-- __init__.py
|   |   `-- website_tags.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   |-- 0001_initial.py
|   |   `-- 0002_testimonials.py
|   |
|   |-- templates/
|   |   `-- website/
|   |       |-- base.html
|   |       |-- index.html
|   |       |-- about.html
|   |       |-- services.html
|   |       |-- service_detail.html
|   |       |-- projects.html
|   |       |-- project_detail.html
|   |       |-- contact.html
|   |       |-- quote_request.html
|   |       |-- thank_you.html
|   |       |
|   |       |-- partials/
|   |       |   |-- _navbar.html
|   |       |   |-- _footer.html
|   |       |   |-- _hero.html
|   |       |   |-- _cta.html
|   |       |   |-- _services_grid.html
|   |       |   |-- _projects_grid.html
|   |       |   |-- _project_card.html
|   |       |   |-- _testimonials.html
|   |       |   |-- _trust_counter.html
|   |       |   |-- _stats_bar.html
|   |       |   |-- _team_member.html
|   |       |   |-- _cookie_consent.html
|   |       |   `-- _whatsapp_button.html
|   |       |
|   |       `-- legal/
|   |           |-- privacy.html
|   |           |-- terms.html
|   |           |-- refunds.html
|   |           `-- cookies.html
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_public_views.py
|
|-- shop/                                           # ONLINE SHOP APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- product.py                              # Product (name, category, price, stock)
|   |   |-- cart.py                                 # Cart (session_key)
|   |   |-- cart_item.py                            # CartItem (cart, product, quantity)
|   |   |-- order.py                                # Order (order_number, client, items, total, status)
|   |   `-- wishlist.py                             # Wishlist (client, product)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   |-- _public_views.py                        # Shop home, product detail
|   |   |-- _cart_views.py                          # Cart CRUD
|   |   |-- _checkout_views.py                      # Checkout, payment
|   |   |-- _order_views.py                         # Order confirmation, tracking
|   |   `-- _admin_views.py                         # (Sidebar Pages) Manage orders
|   |
```

```
|   |-- forms/
|   |   |-- __init__.py
|   |   |-- checkout_form.py
|   |   `-- order_status_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- cart_service.py
|   |   |-- checkout_service.py
|   |   |-- payment_service.py                    # Paystack integration
|   |   `-- order_service.py
|   |
|   |-- tasks.py                                  # Celery tasks
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   |-- templates/
|   |   `-- shop/
|   |       |-- base_shop.html
|   |       |-- index.html
|   |       |-- product_detail.html
|   |       |-- cart.html
|   |       |-- checkout.html
|   |       |-- order_confirmation.html
|   |       |-- order_tracking.html
|   |       |-- order_history.html
|   |       |
|   |       `-- admin/
|   |           `-- dashboard/
|   |               `-- orders/
|   |                   |-- list.html             # (Sidebar Page)
|   |                   |-- detail.html           # (Sidebar Page)
|   |                   |-- update.html           # (Sidebar Page)
|   |                   `-- reports.html          # (Sidebar Page)
|   |
|   `-- tests/
|       |-- __init__.py
|       |-- test_cart_service.py
|       |-- test_checkout_service.py
|       `-- test_payment_service.py
|
|-- services/                                     # SERVICES MANAGEMENT APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   `-- service.py                            # Service (name, category, slug, icon, image)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   |-- _public_views.py                      # Services list, detail
|   |   `-- _admin_views.py                       # (Sidebar Pages) Manage services
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   `-- service_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   `-- service_service.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   |-- templates/
|   |   `-- dashboard/
|   |       `-- admin/
|   |           `-- services/
|   |               |-- list.html                 # (Sidebar Page)
|   |               |-- create.html               # (Sidebar Page)
|   |               |-- edit.html                 # (Sidebar Page)
|   |               |-- detail.html               # (Sidebar Page)
|   |               `-- categories.html           # (Sidebar Page)
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_service_service.py
|
|-- dashboard/                                    # ADMIN DASHBOARD APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |-- context_processors.py
|   |-- middleware.py
|   |
|   |-- data/
|   |   |-- __init__.py
|   |   |-- sidebar_menu.py                       # 80+ navigation items
|   |   `-- quick_actions.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- stats_service.py
|   |   `-- activity_service.py
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   |-- _overview_views.py                    # (Sidebar Page) Dashboard home
|   |   |-- _analytics_views.py                   # (Sidebar Page) Analytics
|   |   |-- _activity_views.py                    # (Sidebar Page) Activity feed
|   |   |-- _notification_views.py                # (Sidebar Page) Notifications
|   |   |-- _project_views.py                     # (Sidebar Pages) Projects CRUD
|   |   |-- _inventory_views.py                   # (Sidebar Pages) Inventory
|   |   |-- _supplier_views.py                    # (Sidebar Pages) Suppliers
|   |   |-- _delivery_views.py                    # (Sidebar Pages) Deliveries
|   |   |-- _finance_views.py                     # (Sidebar Pages) Finances
|   |   |-- _client_views.py                      # (Sidebar Pages) Clients
|   |   |-- _team_views.py                        # (Sidebar Pages) Team management
|   |   |-- _report_views.py                      # (Sidebar Pages) Reports
```

```
|   |   `-- _settings_views.py                      # (Sidebar Pages) Settings
|   |
|   |-- templates/
|   |   `-- dashboard/
|   |       |-- base_dashboard.html
|   |       |
|   |       |-- admin/
|   |       |   |-- overview/
|   |       |   |   `-- index.html                  # (Sidebar Page)
|   |       |   |-- analytics/
|   |       |   |   `-- index.html                  # (Sidebar Page)
|   |       |   |-- activity/
|   |       |   |   `-- index.html                  # (Sidebar Page)
|   |       |   |-- notifications/
|   |       |   |   `-- index.html                  # (Sidebar Page)
|   |       |   |
|   |       |   |-- projects/
|   |       |   |   |-- list.html                   # (Sidebar Page)
|   |       |   |   |-- create.html                 # (Sidebar Page)
|   |       |   |   |-- detail.html                 # (Sidebar Page)
|   |       |   |   |-- edit.html                   # (Sidebar Page)
|   |       |   |   |-- progress.html               # (Sidebar Page)
|   |       |   |   |-- milestones.html             # (Sidebar Page)
|   |       |   |   |-- timeline.html               # (Sidebar Page)
|   |       |   |   `-- budget.html                 # (Sidebar Page)
|   |       |   |
|   |       |   |-- inventory/
|   |       |   |   |-- materials.html              # (Sidebar Page)
|   |       |   |   |-- material_create.html        # (Sidebar Page)
|   |       |   |   |-- material_edit.html          # (Sidebar Page)
|   |       |   |   |-- material_detail.html        # (Sidebar Page)
|   |       |   |   |-- stock_in.html               # (Sidebar Page)
|   |       |   |   |-- stock_out.html              # (Sidebar Page)
|   |       |   |   |-- transactions.html           # (Sidebar Page)
|   |       |   |   `-- alerts.html                 # (Sidebar Page)
|   |       |   |
|   |       |   |-- suppliers/
|   |       |   |   |-- list.html                   # (Sidebar Page)
|   |       |   |   |-- create.html                 # (Sidebar Page)
|   |       |   |   |-- detail.html                 # (Sidebar Page)
|   |       |   |   `-- edit.html                   # (Sidebar Page)
|   |       |   |
|   |       |   |-- deliveries/
|   |       |   |   |-- list.html                   # (Sidebar Page)
|   |       |   |   |-- create.html                 # (Sidebar Page)
|   |       |   |   |-- detail.html                 # (Sidebar Page)
|   |       |   |   `-- edit.html                   # (Sidebar Page)
|   |       |   |
|   |       |   |-- finances/
|   |       |   |   |-- payments.html               # (Sidebar Page)
|   |       |   |   |-- payment_create.html         # (Sidebar Page)
|   |       |   |   |-- payment_detail.html         # (Sidebar Page)
|   |       |   |   |-- payment_edit.html           # (Sidebar Page)
|   |       |   |   |-- expenses.html               # (Sidebar Page)
|   |       |   |   |-- expense_create.html         # (Sidebar Page)
|   |       |   |   |-- budgets.html                # (Sidebar Page)
|   |       |   |   `-- budget_edit.html            # (Sidebar Page)
|   |       |   |
|   |       |   |-- clients/
|   |       |   |   |-- list.html                   # (Sidebar Page)
|   |       |   |   |-- create.html                 # (Sidebar Page)
|   |       |   |   |-- detail.html                 # (Sidebar Page)
|   |       |   |   `-- edit.html                   # (Sidebar Page)
|   |       |   |
|   |       |   |-- team/
|   |       |   |   |-- list.html                   # (Sidebar Page)
|   |       |   |   |-- create.html                 # (Sidebar Page)
|   |       |   |   |-- detail.html                 # (Sidebar Page)
|   |       |   |   `-- edit.html                   # (Sidebar Page)
|   |       |   |
|   |       |   |-- reports/
|   |       |   |   |-- index.html                  # (Sidebar Page)
|   |       |   |   |-- project.html                # (Sidebar Page)
|   |       |   |   |-- inventory.html              # (Sidebar Page)
|   |       |   |   |-- financial.html              # (Sidebar Page)
|   |       |   |   |-- export.html                 # (Sidebar Page)
|   |       |   |   |-- export_history.html         # (Sidebar Page)
|   |       |   |   |-- backup.html                 # (Sidebar Page)
|   |       |   |   `-- backup_history.html         # (Sidebar Page)
|   |       |   |
|   |       |   `-- settings/
|   |       |       |-- general.html                # (Sidebar Page)
|   |       |       |-- appearance.html             # (Sidebar Page)
|   |       |       |-- profile.html                # (Sidebar Page)
|   |       |       |-- profile_edit.html           # (Sidebar Page)
|   |       |       |-- change_password.html        # (Sidebar Page)
|   |       |       |-- security.html               # (Sidebar Page)
|   |       |       |-- notifications.html          # (Sidebar Page)
|   |       |       |-- services.html               # (Sidebar Page)
|   |       |       |-- team.html                   # (Sidebar Page)
|   |       |       |-- admins.html                 # (Sidebar Page)
|   |       |       |-- invite_admin.html           # (Sidebar Page)
|   |       |       |-- admin_detail.html           # (Sidebar Page)
|   |       |       |-- admin_permissions.html      # (Sidebar Page)
|   |       |       |-- audit_log.html              # (Sidebar Page)
|   |       |       |-- audit_export.html           # (Sidebar Page)
|   |       |       `-- data_management.html        # (Sidebar Page)
|   |       |
|   |       `-- partials/
|   |           |-- _head.html
|   |           |-- _breadcrumbs.html
|   |           |
|   |           |-- _components/
|   |           |   |-- _stats_cards.html
|   |           |   |-- _activity_feed.html
|   |           |   |-- _quick_actions.html
|   |           |   |-- _notification_bell.html
|   |           |   |-- _search_modal.html
|   |           |   |-- _theme_toggle.html
|   |           |   `-- _user_menu.html
|   |           |
|   |           |-- _css_inline/
|   |           |   |-- _admin_css.html
|   |           |   |-- _overview_css.html
|   |           |   |-- _projects_css.html
|   |           |   |-- _inventory_css.html
```

```
|   |           |   |-- _suppliers_css.html
|   |           |   |-- _deliveries_css.html
|   |           |   |-- _finances_css.html
|   |           |   |-- _clients_css.html
|   |           |   |-- _team_css.html
|   |           |   |-- _reports_css.html
|   |           |   `-- _settings_css.html
|   |           |
|   |           |-- _javascript_inline/
|   |           |   |-- _admin_js.html
|   |           |   |-- _overview_js.html
|   |           |   |-- _projects_js.html
|   |           |   |-- _inventory_js.html
|   |           |   |-- _suppliers_js.html
|   |           |   |-- _deliveries_js.html
|   |           |   |-- _finances_js.html
|   |           |   |-- _clients_js.html
|   |           |   |-- _team_js.html
|   |           |   |-- _reports_js.html
|   |           |   `-- _settings_js.html
|   |           |
|   |           |-- _modals/
|   |           |   |-- _change_password_modal.html
|   |           |   |-- _invite_admin_modal.html
|   |           |   |-- _delete_admin_modal.html
|   |           |   |-- _edit_permissions_modal.html
|   |           |   |-- _quick_add_modal.html
|   |           |   |-- _confirm_modal.html
|   |           |   `-- _toast_container.html
|   |           |
|   |           |-- _sidebars/
|   |           |   `-- _admin.html                 # 80+ navigation items
|   |           |
|   |           `-- _topbars/
|   |               `-- _admin_topbar.html
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_stats_service.py
|
|-- projects/                                       # PROJECT MANAGEMENT APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- project.py                              # Project (name, location, status, budget, progress)
|   |   |-- milestone.py                            # Milestone (title, target_date, status)
|   |   |-- gallery.py                              # Gallery (project, image, is_primary)
|   |   `-- project_service_link.py                 # Link projects to services
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   `-- _admin_views.py                         # (Sidebar Pages) Projects CRUD
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   |-- project_form.py
|   |   |-- milestone_form.py
|   |   `-- gallery_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- project_service.py
|   |   `-- progress_service.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_project_service.py
|
|-- inventory/                                      # INVENTORY APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- material.py                             # Material (name, category, unit, quantity, threshold)
|   |   |-- supplier.py                             # Supplier (name, contact, phone, email)
|   |   |-- stock_transaction.py                    # StockTransaction (material, type, quantity)
|   |   |-- stock_alert.py                          # StockAlert (material, message, status)
|   |   `-- delivery.py                             # Delivery (supplier, materials, date, receipt)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   `-- _admin_views.py                         # (Sidebar Pages) Inventory management
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   |-- material_form.py
|   |   |-- supplier_form.py
|   |   |-- stock_in_form.py
|   |   `-- stock_out_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- stock_service.py
|   |   `-- alert_service.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_stock_service.py
|
|-- finances/                                       # FINANCIAL MANAGEMENT APP
|   |-- __init__.py
```

```
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- payment.py                          # Payment (project, amount, type, method, status)
|   |   |-- expense.py                          # Expense (project, amount, category, description)
|   |   `-- budget.py                           # Budget (project, category, allocated, spent)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   `-- _admin_views.py                     # (Sidebar Pages) Financial management
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   |-- payment_form.py
|   |   `-- expense_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   `-- financial_service.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_financial_service.py
|
|-- clients/                                    # CLIENT MANAGEMENT APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   `-- client.py                           # Client (name, email, phone, company)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   `-- _admin_views.py                     # (Sidebar Pages) Client management
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   `-- client_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   `-- client_service.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_client_service.py
|
|-- team/                                       # TEAM MANAGEMENT APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   `-- team_member.py                      # TeamMember (name, role, bio, photo, order)
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   |-- _public_views.py                    # Team list (public website)
|   |   `-- _admin_views.py                     # (Sidebar Pages) Team management
|   |
|   |-- forms/
|   |   |-- __init__.py
|   |   `-- team_member_form.py
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   `-- team_service.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   |-- templates/
|   |   `-- dashboard/
|   |       `-- admin/
|   |           `-- team/
|   |               |-- list.html               # (Sidebar Page)
|   |               |-- create.html             # (Sidebar Page)
|   |               |-- detail.html             # (Sidebar Page)
|   |               `-- edit.html               # (Sidebar Page)
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_team_service.py
|
|-- reports/                                    # REPORTS & EXPORTS APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- scheduled_export.py
|   |   `-- backup_record.py
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   `-- _admin_views.py                     # (Sidebar Pages) Reports
|   |
```

```
|   |-- services/
|   |   |-- __init__.py
|   |   |-- report_service.py
|   |   |-- pdf_service.py                          # Playwright
|   |   |-- csv_service.py
|   |   |-- excel_service.py
|   |   |-- export_service.py
|   |   |-- backup_service.py
|   |   `-- deletion_service.py
|   |
|   |-- tasks.py                                    # Celery tasks (weekly audit, backups)
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   |-- templates/
|   |   `-- dashboard/
|   |       `-- admin/
|   |           `-- reports/
|   |               |-- index.html                  # (Sidebar Page)
|   |               |-- project.html                # (Sidebar Page)
|   |               |-- inventory.html              # (Sidebar Page)
|   |               |-- financial.html              # (Sidebar Page)
|   |               |-- export.html                 # (Sidebar Page)
|   |               |-- export_history.html         # (Sidebar Page)
|   |               |-- backup.html                 # (Sidebar Page)
|   |               `-- backup_history.html         # (Sidebar Page)
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_report_service.py
|
|-- notifications/                                  # NOTIFICATION SYSTEM APP
|   |-- __init__.py
|   |-- admin.py
|   |-- apps.py
|   |-- urls.py
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- notification.py                         # In-app notification
|   |   |-- sms_log.py                              # SMSLog
|   |   `-- email_log.py                            # EmailLog
|   |
|   |-- views/
|   |   |-- __init__.py
|   |   `-- _shared_views.py                        # Mark read, get unread count
|   |
|   |-- services/
|   |   |-- __init__.py
|   |   |-- notification_service.py
|   |   |-- email_service.py                        # Resend
|   |   |-- sms_service.py                          # Africa's Talking + Celcom
|   |   |-- router_service.py
|   |   `-- reminder_service.py
|   |
|   |-- tasks.py                                    # Celery tasks
|   |
|   |-- management/
|   |   `-- commands/
|   |       |-- __init__.py
|   |       |-- send_reminders.py
|   |       `-- send_bi_weekly_audit.py
|   |
|   |-- migrations/
|   |   |-- __init__.py
|   |   `-- 0001_initial.py
|   |
|   |-- templates/
|   |   `-- dashboard/
|   |       `-- admin/
|   |           `-- notifications/
|   |               `-- list.html                   # (Sidebar Page)
|   |
|   `-- tests/
|       |-- __init__.py
|       `-- test_notification_service.py
|
|-- static/                                         # STATIC FILES (MINIMAL)
|   |-- css/
|   |   |-- main.css                                # Base resets, variables, typography
|   |   `-- responsive.css                          # Global responsive breakpoints
|   |
|   |-- js/
|   |   |-- main.js                                 # Global utilities
|   |   |-- charts.js                               # Chart.js initialization
|   |   `-- pwa/
|   |       `-- serviceworker.js                    # PWA service worker
|   |
|   |-- images/
|   |   |-- logo.png
|   |   |-- logo_light.png
|   |   |-- logo_dark.png
|   |   |-- favicon.png
|   |   |-- og-image.png
|   |   |-- hero-bg.jpg
|   |   |-- icon-192.png
|   |   |-- icon-512.png
|   |   |-- marker-blue.png
|   |   `-- marker-gold.png
|   |
|   `-- vendors/
|       `-- fontawesome/
|           |-- css/
|           |   `-- all.min.css
|           `-- webfonts/
|               |-- fa-solid-900.woff2
|               |-- fa-regular-400.woff2
|               `-- fa-brands-400.woff2
|
|-- media/                                          # USER-UPLOADED FILES (Cloudinary)
|   |-- projects/
|   |-- services/
|   |-- shop/products/
|   |-- receipts/
```

```
|   |-- documents/
|   |-- profiles/
|   |-- team/                                   # Team member photos
|   `-- reports/
|
|-- templates/                                  # SHARED TEMPLATES
|   |-- 404.html
|   |-- 500.html
|   |
|   |-- email/
|   |   |-- base_email.html
|   |   |-- otp_email.html
|   |   |-- welcome_admin.html
|   |   |-- admin_invitation.html
|   |   |-- password_reset.html
|   |   |-- order_confirmation.html
|   |   |-- order_status_update.html
|   |   |-- stock_alert.html
|   |   |-- payment_receipt.html
|   |   |-- weekly_report.html
|   |   |-- bi_weekly_audit.html                # Bi-weekly audit report
|   |   |-- backup_completed.html
|   |   |-- quote_received.html
|   |   `-- contact_received.html
|   |
|   `-- pdf/
|       |-- project_report.html
|       |-- inventory_report.html
|       |-- financial_report.html
|       |-- receipt.html
|       |-- audit_report.html                   # Bi-weekly audit PDF
|       |-- order_invoice.html
|       |-- quote_pdf.html
|       `-- report_template.html
|
|-- scripts/                                    # DEPLOYMENT SCRIPTS
|   |-- deploy.sh
|   |-- rollback.sh
|   |-- backup_db.sh
|   |-- restore_db.sh
|   `-- verify_patch.py
|
|-- logs/                                       # APPLICATION LOGS
|   |-- debug.log
|   |-- error.log
|   |-- audit.log
|   |-- payments.log
|   |-- shop.log
|   |-- sms.log
|   `-- email.log
|
`-- tests/                                      # MAIN TEST SUITE
    |-- __init__.py
    |-- fixtures/
    |   |-- users.json
    |   |-- projects.json
    |   |-- materials.json
    |   `-- orders.json
    `-- e2e/
        |-- setup.spec.js
        |-- login.spec.js
        |-- invite_admin.spec.js
        |-- project_crud.spec.js
        |-- shop_checkout.spec.js
        `-- dashboard.spec.js
```

Listing B.1: Complete Project Directory

B.2 Directory Summary

Table B.1: Project Directory Summary

Category	Count
Django Apps	12
Dashboard Pages	80+
Public Pages	15+
Models	30+
Service Classes	30+
Email Templates	11+
PDF Templates	8
CSS Files (Static)	2
JS Files (Static)	3
CSS Files (Inline)	11
JS Files (Inline)	11
Total Files	800+

Appendix C

API Endpoints Reference

Table C.1: Complete API Endpoints Reference

Endpoint	Method	Purpose	Auth
Authentication			
/api/auth/login/	POST	Login with credentials	None
/api/auth/verify-otp/	POST	Verify 8-digit OTP	None
/api/auth/refresh/	POST	Refresh JWT	None
/api/auth/logout/	POST	Logout	JWT
/api/auth/password-reset/	POST	Request password reset	None
Projects			
/api/v1/projects/	GET	List projects	JWT
/api/v1/projects/	POST	Create project	JWT
/api/v1/projects/<id>/	GET	Get project	JWT
/api/v1/projects/<id>/	PUT	Update project	JWT
/api/v1/projects/<id>/	DELETE	Delete project	JWT
/api/v1/projects/<id> /progress/	GET	Get progress	JWT
/api/v1/projects/<id> /milestones/	GET	List milestones	JWT
Inventory			
/api/v1/inventory/materials/	GET	List materials	JWT
/api/v1/inventory/materials/	POST	Create material	JWT
/api/v1/inventory/materials/ <id>/	GET	Get material	JWT
/api/v1/inventory/materials/ <id>/	PUT	Update material	JWT
/api/v1/inventory/stock-in/	POST	Record stock in	JWT
/api/v1/inventory/stock-out/	POST	Record stock out	JWT
/api/v1/inventory/alerts/	GET	List alerts	JWT
Shop			
/api/v1/shop/products/	GET	List products	None
/api/v1/shop/products/<id>/	GET	Get product	None
/api/v1/shop/cart/	GET	Get cart	None
/api/v1/shop/cart/	POST	Add to cart	None
/api/v1/shop/cart/<id>/	PUT	Update cart item	None
/api/v1/shop/cart/<id>/	DELETE	Remove from cart	None
/api/v1/shop/orders/	GET	List orders	JWT
/api/v1/shop/orders/	POST	Create order	None
/api/v1/shop/orders/<id>/	GET	Get order	None
Reports			
/api/v1/reports/project/<id> /	GET	Project report	JWT
/api/v1/reports/inventory/	GET	Inventory report	JWT
/api/v1/reports/financial/	GET	Financial report	JWT
/api/v1/reports/orders/	GET	Order report	JWT

Appendix D

Email Template Inventory

Table D.1: Complete Email Template Inventory

Template	Purpose	Key Variables
otp_email.html	8-digit OTP verification	user, otp_code, expiry_minutes
welcome_admin.html	New admin welcome	user, temp_password, dashboard_url
admin_invitation.html	Admin invitation	invitation_link, expires_at
order_confirmation.html	Order confirmation	order, client_name, order_number
order_status_update.html	Order status update	order, new_status, tracking_number
stock_alert.html	Low stock alert	material, current_quantity, threshold
payment_receipt.html	Payment receipt	payment, amount, receipt_number
weekly_report.html	Weekly report	week_start, week_end, stats
bi_weekly_audit.html	Bi-weekly audit report	period, actions, stats
admin_contact_alert.html	Contact form alert	name, email, phone, message
admin_quote_alert.html	Quote request alert	name, project_type, budget, details
backup_completed.html	Backup completed	project, backup_file, size
quote_received.html	Quote confirmation	name, project_type, timeline
contact_received.html	Contact form confirmation	name, message

Appendix E

Permission Matrix

Table E.1: Complete Permission Matrix

Permission	Chief Admin	Platform Admin	Client
View dashboard	Yes	Yes	No
View all projects	Yes	Yes	No
Create projects	Yes	Yes	No
Edit projects	Yes	Yes	No
Delete projects	Yes	Configurable	No
View materials	Yes	Yes	No
Create materials	Yes	Yes	No
Edit materials	Yes	Yes	No
Record stock in	Yes	Yes	No
Record stock out	Yes	Yes	No
View alerts	Yes	Yes	No
View products	Yes	Yes	Yes
Place orders	Yes	Yes	Yes
View orders	Yes	Yes	Yes (own only)
Manage orders	Yes	Configurable	No
View payments	Yes	Yes	No
Record payments	Yes	Yes	No
View expenses	Yes	Yes	No
Record expenses	Yes	Yes	No
View clients	Yes	Yes	No
Create clients	Yes	Yes	No
View team members	Yes	Yes	No
Manage team members	Yes	Configurable	No
View services	Yes	Yes	No
Manage services	Yes	Configurable	No
View reports	Yes	Yes	No
Export data	Yes	Configurable	No
Backup projects	Yes	Configurable	No
Delete projects	Yes	Configurable	No
Manage admins	Yes	Configurable	No
Invite admins	Yes	Configurable	No
Manage settings	Yes	Configurable	No
View audit log	Yes	Yes	No
View notifications	Yes	Yes	Yes

Appendix F

Testing Checklist

Table F.1: Testing Checklist — Critical Paths

Feature	Test Type	Status	Notes
Setup URL (first admin)	E2E	Pending	One-time use
Login with OTP	E2E	Pending	5-minute expiry
Password reset	Integration	Pending	Email token
Admin invitation	Integration	Pending	48-hour expiry
Project CRUD	Unit	Pending	Full CRUD
Progress tracking	Integration	Pending	Milestone-based
Stock IN/OUT	Unit	Pending	Quantity update
Low stock alert	Integration	Pending	Email + In-App
Shop checkout	Integration	Pending	Paystack integration
Order status update	Integration	Pending	Status workflow
Payment recording	Unit	Pending	Amount validation
Report generation	Integration	Pending	PDF/CSV export
Bi-weekly audit	Integration	Pending	Email + PDF
Data export	Integration	Pending	CSV/Excel/PDF
Backup creation	Integration	Pending	Project backup
Soft delete	Unit	Pending	Archive + restore
Hard delete	Integration	Pending	Requires backup
Notification delivery	Integration	Pending	Email + SMS
Role permissions	Integration	Pending	Access control
API endpoints	Integration	Pending	JWT auth

Appendix G

Deployment Checklist

Table G.1: Deployment Checklist — Pre-Launch

Item	Status	Notes
Domain registered	Pending	caldanconstruction.com
SSL certificate	Pending	Let's Encrypt
Database configured	Pending	MySQL 8.0
Environment variables	Pending	.env file
Static files collected	Pending	collectstatic
Migrations applied	Pending	migrate
Paystack keys (sandbox)	Pending	Test mode
Paystack keys (production)	Pending	Live mode
Resend API key	Pending	Email
Africa's Talking API key	Pending	SMS primary
Celcom API key	Pending	SMS fallback
Cloudinary configuration	Pending	Media storage
Redis configuration	Pending	Cache + Celery
Celery configured	Pending	Tasks
Health check endpoint	Pending	/health/
Backup script	Pending	backup_db.sh
Monitoring alerts	Pending	DigitalOcean
Rollback script	Pending	rollback.sh

Appendix H

Caldan Builder Company Profile Details

This appendix consolidates the details supplied in the uploaded **Caldan Builder Company Profile** dated **April 2021**. These details are included to ensure that the technical specification also preserves the source company profile information used to describe the construction business context behind the Caldan Construction platform.

IMPORTANT The uploaded company profile contains several legacy or source-document inconsistencies, including references to **IVORYMART LIMITED**, **Bison Construction**, **NCA 4**, and **NCA 8**. They are retained here as source-profile notes for verification rather than silently overwritten.

H.1 Profile Header and Company Identity

Table H.1: Company Profile Header Details from April 2021 Source Profile

Item	Profile Detail
Profile title	Company Profile
Profile date	April 2021
Business name used	Caldan Builder
Business description	Dealers in design, construction, renovations, interior works, welding/fabrication, and concrete products
Primary postal address in header	P.O. Box 4215-20100, Nakuru
Header office location	Odera Street, Kaloka Building, opposite Nildeep
Header email	`ogwokadaniel@gmail.com`
Header contact	0703 653 010
Alternate header found in source	P.O. Box 4512-20100, Nakuru; Office: Odera Street, Kaloka Building, opposite Nildeep; Email: `info@bisonconstruction.co.ke`; Contacts: 0715792131, 0796 321480

H.2 Source Profile Contents and Annexures

Table H.2: Uploaded Company Profile Structure

Section	Source Page	Purpose
Introduction	3	Establishes company background and capability
Vision	3	Defines long-term professional services ambition
Mission Statement	3	Defines service quality, statutory practice, codes of conduct, and integrity
Services	3	Lists construction and engineering service categories
Ownership & Management	4	Captures leadership and management information
Approach	4	Describes teamwork, communication, and hands-on project control
Experience	4	Lists selected undertaken or ongoing projects
Resources	6	Captures directors, software, equipment, facilities, and specialist support
Contact Us	6	Provides postal, physical, telephone, email, and web contact fields
Conclusion	7	Summarizes capability and dedication

Table H.3: Uploaded Profile Annexures

Annexure	Description	Pages
A	Shareholding	1
B	Project / Employment References	1
C	Banking Details	1

H.3 Introduction, Vision, and Mission

Introduction. Caldan Builder was established in **2008**. The company profile states that the firm has since realized steady growth in its product portfolio and project range. It further states that Caldan Builder has invested heavily in machinery and trained manpower to achieve best-in-industry standards.

Caldan Builder is described as a developing multi-disciplinary practice offering clients specialist individual attention and solutions required by an ever-changing project environment. The profile emphasizes teamwork as a key project success element and states that the firm is duty-bound to ensure efficiency, professionalism, and timely project delivery for all clients.

The profile also states that Caldan Builder believes in a hands-on approach on all projects, with a director of the firm always in control of key project functions. It notes that the firm is equipped with current technology and has the staff and resources required to provide professional service at all times.

Vision. The stated vision is to grow the multi-disciplinary team in order to offer a broad spectrum of specialist engineering and construction services, becoming clients' preferred Professional Service Provider (PSP) through excellence and efficiency across the full project life cycle. The profile states that, whether the assignment is a building project or a civil engineering project, Caldan Builder seeks to exceed expectations and set new standards.

Mission. The stated mission is to offer excellent service in each professional discipline in accordance with statutory practices, codes of conduct, and integrity. The mission further commits the firm to developing its team and providing a leading platform from which to serve the built environment and valued clients.

H.4 Services

Table H.4: Caldan Builder Service Categories from Source Profile

Service Category	Description / Scope
General design & construction	Design and construction services for building and civil works
Pre-cast concrete products	Concrete products forming part of the firm's product portfolio
Industrial floor solutions	Flooring and industrial floor works
Water proofing works	Waterproofing services for building and construction projects
Roads and storm water infrastructure	Civil infrastructure works including roads and drainage-related works
Construction chemicals	Construction chemical products or applications
Interior works	Interior construction and renovation works referenced in the profile header
Welding/fabrication	Welding and fabrication services referenced in the profile header
Renovations	Renovation works referenced in the profile header

H.5 Ownership, Management, and Approach

The uploaded profile states that affirmative action is important to **IVORYMART LIMITED**, with a focus on recruitment and training of previously or historically disadvantaged individu-

als. Because this name differs from Caldan Builder, this detail is retained as a source-profile inconsistency requiring business verification.

The source profile also states that the company has an **NCA 4 recognition level** and is headed by a dynamic, motivated, and efficient team led by Managing Director **LAWRENCE NJOROGE (Eng.)**, assisted by **Mr. GERISHON NGUGI** and **Mr. JAMES NGUGI (Civil Eng.)** and staff. This management description is preserved as a source-profile note and should be verified against the Caldan Builder directors listed elsewhere in the same profile.

Caldan Builder's approach is described as teamwork-driven. The profile states that teamwork on every project is a key success ingredient and that the overall project manager is duty-bound to ensure effective communication with all stakeholders at all levels. Teams are described as being built by combining individual strengths and refining skills to meet and exceed client expectations.

The approach section also repeats the hands-on operating model: a director of the firm remains in control of key project functions, while the firm uses current technology, staff, and resources to provide professional service.

H.6 Experience and Project References

Table H.5: Selected Projects Listed in the Uploaded Company Profile

Project	**Client**	**Value / Financial Year Detail**
Munanda Secondary School	NCDF of Gilgil	2 million
25 Units Apartment in Nakuru – ongoing	Kennedy Onyancha	35 million
18 Units Apartment	Dr. Nyangara Naphtali	22 million
Oyarai Primary School	NG CDF	1.1 million
Oljorai Secondary School	Ministry of Education	2.1 million
Gilgil Secondary School	Koto Housing	6.5 million
Massonnaitte	James Mwangi	4.2 million

The source profile states that Caldan Builder has a combined professional experience of approximately **10 years** among its principals and members. Annexure A separately states that Dr. Daniel O. Siringi has **over 30 years of experience in construction**; both details are preserved here because they appear in different parts of the uploaded profile.

The profile includes pictorial headings for **Designing**, **Flooring works**, **Concrete repair works**, **Tank installation**, **Building works**, and **Civil/earth works**. These categories are retained as source-profile evidence categories for project-gallery or portfolio functionality in the platform.

H.7 Resources

Table H.6: Resource Details from Uploaded Company Profile

Resource Area	Source Profile Detail
Directors	Dr. Daniel O. Siringi – Building construction; Mrs. Callen K. Manoti – Project Administration
Design/software platforms	Caldan Builder has acquired current design programs that support planning, design, estimates, bills of quantities, and general operations. The profile states that the firm is IT compliant.
Equipment/facilities	Permanent office address in Nakuru Town, flight offices in various towns, company vehicles, necessary tools, cameras, laptops, printers, and construction equipment.
Project readiness	The firm states that it is equipped to handle project-related facilities and equipment needed to satisfy client requirements at any time.
Specialist support	The firm can undertake building, structural, health and safety, and environmental categories through specialist service providers with established working relationships.
Administrative support	The firm has accounts with support and administrative organizations including stationary, courier services, and travel services for project-related expenses.

H.8 Contact Details

Table H.7: Contact Details from Uploaded Company Profile

Contact Field	Source Profile Detail
Postal address in contact section	P.O. Box 4512-20100, Nakuru Town
Physical address in contact section	Along Munanda Building, Nakuru County
Contact numbers	+254 (0) 703 653 010; +254 (0) 703 653 010
Email	ogwokadaniel@gmail.com
Web	Blank in source profile
Header postal address	P.O. Box 4215-20100, Nakuru
Header office address	Odera Street, Kaloka Building, opposite Nildeep

H.9 Conclusion from Source Profile

The uploaded profile concludes that Caldan Builder is a capable and dedicated multi-disciplinary building and civil construction firm with a solution-based approach to meeting and exceeding client expectations. It states that the experienced and focused team is equipped with the experience and qualifications required to perform its functions with diligence and skill.

The source profile further states that it is Caldan Builder's vision to serve prospective clients in the private, professional, and public sectors. It asks that all enquiries and requests for additional information be directed to Caldan Builder using the details provided in the profile.

H.10 Shareholding, Credentials, References, and Banking Details

Table H.8: Shareholding Details from Annexure A

Name	Competence	Shares	Voting
Dr. Daniel O. Siringi	Over 30 years experience in construction	50%	50%
Mrs. Callen K. Manoti	Administration	50%	50%
Totals	–	100%	100%

Table H.9: Credentials, References, and Banking Details from Annexures

Area	Source Profile Detail
Government registration	Certificate of incorporation
Industry certification	NCA 8
Accreditations	Heading present; detailed entries not provided in extracted source text
Project / employment references	Annexure C provides columns for Client, Project, Reference, and Tel No.; entries are blank in the extracted source text and refer readers to the Experience section
Bank	Blank / dash in source profile
Branch	Blank / dash in source profile
Branch code	Blank / dash in source profile
Account number	Blank / dash in source profile
Account name	Blank / dash in source profile
Branch contact number	Blank / dash in source profile
Banking annexure source name	The banking annexure is titled **IVORYMART LIMITED; ANNEXURE D – BANKING DETAILS**, which conflicts with the Caldan Builder profile name and should be verified

Final Statement

FINAL STATEMENT

This Version 1.0 specification is the complete technical blueprint for **Caldan Construction**. It covers **12 Django applications**, **80+ dashboard pages**, **30+ service classes**, **25+ database models**, **8 external integrations**, and a **phased implementation roadmap** spanning **26+ weeks** to production handover.

Every feature, every model, every view, and every decision documented here is the product of exhaustive analysis of the Kenyan construction industry—an industry that has long relied on paper records, verbal communication, and manual tracking.

The gap between what existed and what is possible is now bridged.

Dr. Daniel Ogwoka Siringi
Director, Caldan Builder

Mukoya Khisa
Website Developer & Manager
Caldan Construction
Nairobi, Kenya
July 2026

"Building Kenya's Future, One Project at a Time."

CALDAN CONSTRUCTION

Confidential Technical Specification

12	80+	26+
Django Apps	Dashboard Pages	Delivery Weeks

“Building Kenya’s Future, One Project at a Time.”

Nairobi, Kenya | Document ID: CAL-SPEC-2026-07-V1.0